

从零理解 simpleKafka

一门面向源码的中文渐进式课程

simpleKafka 教学项目

课程版本：Java 21 · 31 个可运行步骤

写在开始之前

本书用一章课程地图和八个实现章节，展开 31 步。读者会亲自完成记录格式、日志、网络请求、消费位点、消费组、受控副本切换，以及跨 producer、broker、consumer 的一致性实验；不是通过连接真实 Kafka 来“假装实现”，也不是把生产级 Kafka 缩成几个空洞名词。每一步都对应一个明确的 Java 方法、行为测试、失败边界和可重复命令。请从第一步开始按序推进；本书正文、课程清单与测试类共用相同编号。本书面向 Kafka 零基础读者：不要求了解 Kafka、消息队列或分布式系统。开始前只需能读基本 Java（类、方法、集合、异常）并在终端运行命令；遇到二进制、文件 I/O、网络、并发和时钟时，本书会在首次需要它们的步骤中解释。

课程代码刻意分成 `provided/`、`exercises/`、`reference/` 三个源根。学生入口编译前两者，参考入口编译第一和第三者；两套实现具有相同全限定类名，绝不可把两个答案源根放进同一次编译。参考实现用于在尝试后对照契约，并非学生测试 classpath 的后门。详细的开发环境、测试读取方法和晋级规则见本书第一章。

目录

第一章 课程地图、工具与学习方式	1
1.1 一条消息怎样经过系统	1
1.2 本课程实现什么（以及不是什么）	2
1.3 准备 Java 21 与课程命令	2
1.4 读懂二进制与文件 I/O	3
1.5 测试失败的阅读法	3
1.6 已学概念小结与后续导航	4
第二章 记录与单段持久化	5
2.1 第 1 步：记录编解码	5
2.2 第 2 步：日志追加	8
2.3 第 3 步：按位点读取	9
2.4 第 4 步：崩溃恢复	11
第三章 分段、索引、保留与截断	14
3.1 第 5 步：稀疏索引	14
3.2 第 6 步：分段日志	16
3.3 第 7 步：日志保留	18
3.4 第 8 步：尾部截断	20
第四章 分区与真实网络 broker	23
4.1 第 9 步：分区路由	23
4.2 第 10 步：主题元数据	25
4.3 第 11 步：TCP 帧协议	27
4.4 第 12 步：单机 broker	29
第五章 生产消费、位点与重复投递	33
5.1 第 13 步：批量生产者	33

5.2	第 14 步：拉取消费者	35
5.3	第 15 步：消费位点持久化	37
5.4	第 16 步：至少一次处理	39
第六章	消费组、重平衡与提交围栏	42
6.1	第 17 步：分区分配	42
6.2	第 18 步：成员与重平衡	44
6.3	第 19 步：心跳与过期	46
6.4	第 20 步：消费组围栏	48
第七章	拉取副本、ISR、高水位与确认	51
7.1	第 21 步：拉取复制	51
7.2	第 22 步：ISR 与高水位	53
7.3	第 23 步：生产确认	55
7.4	第 24 步：并发副本 broker	57
第八章	epoch 围栏、干净选举、分叉修复与重新入组	60
8.1	第 25 步：干净选举与 epoch	60
8.2	第 26 步：分叉日志修复	62
8.3	第 27 步：元数据路由刷新	64
8.4	第 28 步：副本重新入 ISR	67
第九章	生产者、broker 与消费者的一致性	70
9.1	第 29 步：三层一致性故障实验	70
9.2	第 30 步：幂等生产者	74
9.3	第 31 步：消费副作用幂等	79
附录 A	格式、错误码与接口索引	82
A.1	磁盘与网络格式	82
A.1.1	记录与索引	82
A.1.2	TCP frame 与请求体	83
A.1.3	消息 body 字段顺序	83
A.1.4	标识符与值类型	85
A.2	幂等追加与持久化副作用	85
A.3	固定错误码	85
A.4	三十一歩与编辑入口	86

目录	iv
A.5 课程简化与 Kafka 的差异	88
A.6 官方资料与延伸阅读	88
A.7 排版与构建说明	89
术语索引	90

第一章 课程地图、工具与学习方式

1.1 一条消息怎样经过系统

先从一笔订单开始：用户付款后，商店要把“订单已付款”交给后续处理程序，例如库存服务。若付款程序直接调用库存程序，接收方停机时消息可能丢失；消息系统让发送方先交出一条消息，由系统保存，再由接收方按自己的速度读取。Apache Kafka 是一种分布式消息流平台；本书用可运行的小型教学系统解释它所涉及的基本问题，不假定读者已知道 Kafka，也不把课程协议说成官方 Kafka 协议。

本书把一条可保存、可读取的数据称为**消息或记录 (record)**。**生产者 (producer)** 发送记录，**broker** 接收请求并保存记录，**消费者 (consumer)** 主动请求并处理记录。一个消息系统是负责在它们之间传递和保存记录的软件。发送请求成功、记录被消费者读到、业务处理完成，是三个不同的事件；后面的章节会分别保存和检查它们的状态。

topic (主题) 是给相关消息取的分类名称，例如 **orders**。一个 topic 可以拆成多条各自有序的 **partition (分区)** 日志；本课程用分区让不同记录可以独立追加和读取。记录的 **key** 可用于关联或路由，**value** 是实际内容，**timestamp (时间戳)** 是随记录保存的时间值；时间戳不替代决定分区内顺序的 **offset (位点)**。每条记录的 offset 是它在所属分区里的编号，从 0 开始；不同分区各自编号，不能跨分区比较，也不存在一个代表整个 topic 的全局顺序。

topic	partition	记录的 offset 顺序
orders	0	0, 1
orders	1	0, 1

表中两个分区的 offset 0 是两条不同记录；本书能保证各分区内部的顺序，不给两个分区之间排出先后。

一条记录的生命周期是：生产者选择目标分区；broker 把记录追加到该分区日志；消费者主动拉取记录；程序执行处理回调；处理成功后再保存消费进度。读取不会删除日志中的记录，所以另一消费者或重启后的消费者仍可能再次读取它。日志保留决定旧记录何时被删；消费进度只说明某个消费者从哪里继续，两者不是同一件事。第 4 章会解释处理成功与保存进度之间的失败窗口。

章节与步骤	要解决的问题
第 1 章, 1-4	怎样编码记录、追加和读取文件, 并识别崩溃留下的不完整尾部?
第 2 章, 5-8	怎样用索引加速查找、分段保存, 并分别删除旧头部或截断尾部?
第 3 章, 9-12	怎样选择分区、发现 broker 的位置, 并把 TCP 字节还原成完整请求?
第 4 章, 13-16	怎样批量发送、读取记录、保存消费进度, 以及理解失败重放?
第 5 章, 17-20	多个消费者如何分配分区, 旧成员的请求如何被拒绝?
第 6 章, 21-24	broker 间怎样复制日志, 哪些记录可以被普通消费者看见?
第 7 章, 25-28	leader 故障时怎样选择副本、修复分叉并重新接纳副本?
第 8 章, 29-31	如何观察跨层故障、识别重复发送, 并让本地业务效果可重放?

1.2 本课程实现什么（以及不是什么）

本项目把上述路径做成一个小而可运行的系统：磁盘上的分区日志、TCP 请求、生产与消费、消费位点、单 topic 消费组，以及多个本机 broker 之间的拉取复制和受控切换。它不是 Kafka 官方客户端，也不使用 Kafka wire protocol；不能把官方 `KafkaProducer` 或 `KafkaConsumer` 指向本程序。

三个 Java source root 是课程设计的一部分。`provided/` 放数据类型、I/O、传输、序列化样板、生命周期和测试夹具；它让注意力集中在算法而非重复搭框架。`exercises/` 是学生唯一需要逐步补全的工作区，所有步骤的类型和公开签名预先可见，尚未到达的方法会抛出带步骤号的 `ExerciseNotImplementedException`。`reference/` 是隔离的完整实现，供尝试后对照契约和课程验证；它与练习代码类名相同，因此学生和参考源根绝不能同次编译。测试在两种模式下复用同一套黑盒契约；学生运行时 classpath 不含参考 classes。

这不是生产级 Kafka：没有 Kafka 线协议兼容、事务、压缩、compaction、完整控制器共识或 controller HA。第 8 章的有限幂等生产者协议和本地持久化效果账本不是 Kafka 生产协议，也不构成端到端事务。它们使用单 JVM 内可信的教学控制面；虽有多台 broker 的 TCP server、线程和独立磁盘目录，副本仍共享该控制面。它能演示受控故障切换，不能证明任意网络分区下具备共识或生产级可用性。具体差异会在相关步骤说明。

1.3 准备 Java 21 与课程命令

项目使用 Gradle Wrapper `./gradlew`；Gradle 9.8.1 可运行于 Java 17+，Java 编译、测试和 demo 任务需要 Java 21+ JDK，并以 `--release21` 编译。不需要 Maven、Docker 或已启动的 Kafka。脚本使用 Bash 3.2+、`find`、`awk`、`curl` 与 `sha256sum` 或 `shasum`；准备 Tectonic 需要 `tar` 和 `install`，macOS 解压 Pandoc 还需要 `unzip`。EPUB 图形需要 Poppler 的 `pdftoppm`；`./gradlew :setupBook` 会检查它，但不安装系统软件。Linux/WSL 使用 `sudo apt-get install poppler-utils`，macOS 使用 `brew install poppler` 安装 Poppler。先检查 `java -version`、`javac -version` 与 `./gradlew --version`。可通过 `JAVA_HOME` 或 `PATH` 使用任意符合要求的 JDK；SDKMAN 可选，项目不会静默修改 SDKMAN、系统软件或 shell 配置。

`./gradlew :setup` 通过 Gradle 解析固定版本 JUnit 1.11.4 并验证 checksum，不再把 JUnit 放入 `.tools/`。`./gradlew :setupBook` 准备经过 checksum 校验的 Tectonic 0.17.0 与 Pandoc

3.12.1 到 `.tools/`, 并检查 Poppler; 不安装系统软件。`./gradlew :doctor` 检查 Gradle、Java、Gradle 管理的 JUnit 与可选教材工具。`./gradlew :compile` 编译学生骨架与测试; 编译成功不表示练习已经实现。课程清单与教材默认英文; 设置 `COURSE_LANG=zh` 后运行 `./gradlew :list` 可显示中文步骤标题。`./gradlew :book` 编译英文 PDF/EPUB 到 `build/book/en/simpleKafka.pdf` 与 `build/book/en/simpleKafka.epub`; 设置 `COURSE_LANG=zh` 后运行同一命令, 生成 `build/book/zh/simpleKafka-zh.pdf` 与 `build/book/zh/simpleKafka-zh.epub`。每次还会更新所选语言的 `docs/book/dist/` 分发文件; 读者可直接打开已分发成品, 无需 Java 或编译工具。book 任务需要启动 Wrapper JVM, 但不编译 Java 或解析 JUnit。首次构建需联网获取 TeX bundle; 两种语言都在线成功后, 设置 `BOOK_OFFLINE=1` 并用 `./gradlew :book --offline` 离线构建。`Gradle --offline` 控制 Gradle 依赖; `BOOK_OFFLINE=1` 为两次 Tectonic 调用启用 TeX bundle only-cached 模式。

1.4 读懂二进制与文件 I/O

文件只保存字节, 不会替程序记住字段边界。UTF-8 字符“值”占 3 个字节, 因此课程例子中的 value 长度是 3, 而不是字符个数 1; 普通记录的 $\text{length}=28+3=31$, 总占用 $=4+31=35$ 字节。编码格式必须同时规定字段顺序、整数宽度和字节序。本课程用大端序: 例如整数 1 的四个字节是 00000001, 最高有效字节在前。

`ByteBuffer` 是一段带容量、position 与 limit 的字节视图。position 指下一次读写的位置; limit 是当前可读/可写范围的末端。写入模式下调用 `flip()` 会把 position 归零、limit 设为刚写完的位置, 随后便可读取这些字节; 不能把 flip 当成清空数据。buffer 的 position 是其中的字节游标, 后续练习中的 record offset 是分区内的逻辑编号, 不能互换。解码器只应推进到已成功消费的边界。

`FileChannel` 可以从指定文件 byte position 读取或写入, 也能请求强制同步已写数据。一次 `channel.read` 可能只填入部分 buffer, 一次 `write` 也可能只写出部分内容; 课程提供的 `ChannelIO.readFully/writeFully` 会循环处理这种情况。步骤 2 会用锁 (lock) 让同一日志上的并发追加一个接一个完成, 避免批次交错; 锁只保护短暂的共享状态操作, 不应拿着它等待网络。每批追加后的 `force(false)` 是请求持久化文件数据的简化方式, 不是对掉电、电源缓存、介质损坏或文件系统语义的数学保证。CRC32C 检测记录字节意外改变, 但不修复错误, 也不能证明数据来源可信。

1.5 测试失败的阅读法

先读失败的 Step 测试名和断言, 再回到本章契约。测试报告中的第一处异常通常比最后一段堆栈更有用: 未实现方法给出步骤和方法名; 课程错误应保留 `ErrorCode`; 意外 `IOException` 可能被包装为 `STORAGE_ERROR` 并保留 cause。不要把错误映射成空列表或成功默认值来让测试变绿。一个单步定位命令只运行该步, 不代表累计依赖都满足; 累计命令会再次评估之前每步。课程 runner 按步骤分组输出测试名称、各步和总计数; 失败时显示断言、精确方法定位与重跑命令。报告始终为英文; legacy JUnit XML 保存到 `build/student/reports/` 或 `build/reference/reports/`。自定

义 JavaExec 课程任务输出纯文本。标准 Gradle `:exercises:test` 与 `:reference:test` 支持 IDE 调试及精确 `--tests` 选择；其 XML/HTML 报告位于 `build/<mode>/gradle-test-results/` 与 `build/<mode>/gradle-test-reports/`。runner 保留 `NO_COLOR` 与 `TERM=dumb` 颜色门禁。

命令	意义
<code>./gradlew :list</code>	显示 31 步、编辑方法与前置关系。
<code>./gradlew :stepTest -Pstep=12</code>	只定位第 12 步；失败时辅助找当前问题。
<code>./gradlew :test -Pstep=12</code>	累计运行 Step01Test 至 Step12Test。
<code>./gradlew :referenceTest -Pstep=31</code>	对隔离参考实现运行完整 31 步契约。
<code>./gradlew :referenceDemo</code>	运行完整参考 TCP/failover 场景。
<code>./gradlew :referenceConsistencyDemo</code>	运行第 8 章真实 TCP 与磁盘一致性场景。

步骤编号接受 1–31 的十进制数字，可有前导零（例如 `-Pstep=1` 或 `-Pstep=01`）。章节边界是第 4、8、12、16、20、24、28、31 步：只有该边界累计测试全绿，才算章节完成。`./gradlew :test -Pstep=8` 通过意味着第 1–8 步全部通过；`./gradlew :stepTest -Pstep=8` 仅定位第 8 步，不授予晋级。全课程测试不应把参考源放进学生 classpath，参考模式也不能代替自己的练习进度。

1.6 已学概念小结与后续导航

先记住三点：主题是记录分类，分区是各自有序的日志，而每条记录的 offset 只在自己的分区内有意义；发送、读取和业务处理是不同事件，读取本身不删除记录；日志保留决定记录何时删除，消费进度决定某个消费者从哪里继续。后续每遇到新的术语和不变量，都会先在对步骤解释，不需要预先掌握 Kafka 缩写。

可以带着具体问题进入后续步骤：[第 1 步（记录编解码）](#) 解释一条记录如何变成有明确边界的字节；[第 4 步（崩溃恢复）](#) 说明如何判断文件末尾是未写完还是完整数据损坏；[第 14 步（拉取消费者）](#) 展示消费者读取位置如何推进；[第 22 步（ISR 与高水位）](#) 解释多份日志进度不一致时，普通消费者能看到哪些记录；[第 25 步（干净选举与 epoch）](#) 讲一个 broker 接替写入角色后，如何拒绝旧角色发来的请求。

第二章 记录与单段持久化

本章从“进程退出后，怎样把一条消息从文件中原样找回来”开始。内存会随进程结束而消失，文件却只保存字节，不会自动记住哪些字节属于同一条记录。课程先定义每条记录的二进制边界，再把记录追加成只追加日志；后续用逻辑 offset（位点）标识记录，用文件 byte position（字节位置）找到它们。LEO（log end offset，日志末端位点）是下一条可写记录的 offset，不是最后一条记录的 offset，也不是文件字节数。这里的概念是课程格式，不是 Kafka 的磁盘格式。

length	CRC32C	offset	timestamp	key/value len	key value
4 B	4 B	8 B	8 B	8 B	变长

图 2.1: 一条磁盘记录。length 不包含最前面的 4 字节自身。

2.1 第 1 步：记录编解码

目标与前置知识

本步目标：定义稳定的磁盘边界，使写入方和读取方对每个字节有相同解释。

开始前：理解长度、定宽整数、UTF-8 字节和 CRC32C；不要求先会二进制或文件扫描。

背景与推理

文件中的数字和字段都只是字节；格式必须约定字段顺序、宽度和字节序，才能从字节还原字段。本课程用大端（高位字节先放）格式 `length:int32|crc32c:int32|offset:int64|timestamp:int64|keyLength:int32|valueLength:int32|key|value`。length 从 CRC 字段起计，不含最前面的 4 字节 length 自身。定宽部分除 payload 外占 32 字节总空间，其中 length 值的固定部分是 28 字节。CRC32C 检查 offset 到 value 末尾是否与写入时一致；它能发现常见的意外损坏，但不是身份验证或安全签名。

字段	字节宽度	含义
length	int32, 4	不含自己的前 4 字节; 合法值为 28–1,048,576。
crc32c	int32, 4	校验 offset 至 value 末尾, 不包含 length 与 CRC 自身。
offset	int64, 8	分区内的逻辑编号, 必须非负。
timestamp	int64, 8	随记录保存的非负时间值。
keyLength	int32, 4	-1 表示 null key, 0 表示空 key, 否则是 key 的字节数。
valueLength	int32, 4	value 的非负字节数; value 不能为 null。
key、value	变长	按各自 length 紧随长度字段; 总字节数按编码后的 bytes 计算。

起点与编辑位置。

已给: LogRecord 表示带逻辑 offset 的记录; RecordData 保存 key、value、timestamp; CourseException 与 ErrorCode、大端字段及 CRC32C 工具已提供。

请修改: 仓库根目录下的 exercises/src/main/java/io/simplekafka/storage/RecordCodec.java: 实现 public static ByteBuffer encode(LogRecord record) 与 public static LogRecord decode(ByteBuffer source)。不要改 provided 类型、参考实现或测试。

开始前先读

这是课程第一步; 只需能读 Java 类、方法和异常。本步引入 byte (字节, 8 bit) 作为文件长度单位: UTF-8 字符可以由多个字节组成, 所以长度必须数编码后的字节而不是字符。下一步会把编码结果写进文件。

输入、输出与状态

- `encode` 接受一个 `LogRecord(offset, data)`; 若 record 为 null, 报 `INVALID_REQUEST`。offset 与 timestamp 均为非负 long, value 必须存在 (可为零字节), key 可为 null、空数组或非空字节数组。成功返回 position=0、limit 恰好到该记录末尾的新 buffer, 不读取或改变文件状态。
- `decode` 从 source 当前 position 开始读; source 为 null 时抛出 `NullPointerException`, 没有消耗输入。它按大端解释字段, 不依赖 source 自身的 byte order; heap、direct、read-only、slice buffer 都是有效输入。成功只消费一条记录并将 position 推到该记录之后, limit 不变, 后续记录字节留给下一次调用。
- length 合法范围是 28 到 1,048,576 (含); 总记录空间是 $4 + \text{length} = 32 + \text{keyBytes} + \text{valueBytes}$ 。null key 写作 `keyLength=-1`, 空 key 写作 0; `valueLength` 必须非负。普通记录没有额外身份字段。

行为契约与不变量。输入不完整 (连 4 字节 length 前缀都不全, 或合法声明的 body 尚未全部到达) 报 `INVALID_REQUEST`。完整但 length 越界、字段结构不合法、offset/timestamp 为负或 CRC 不匹配报 `CORRUPT_RECORD`。encode 的非法记录或大于上限的记录报 `INVALID_REQUEST`。decode 无论成功或失败都不得改 limit; 失败不得推进 source position。

手算一个完整记录

先看测试的独立 golden: offset=7、timestamp=23、key=null、value 为空, 所有整数为大端。十六进制字段依次为 0000001cdf159f32000000000000000070000000000000017ffffffff00000000: length=28; CRC32C 为 df159f32; keyLength 的 ffffffff 是 -1; valueLength=0。前缀 4 字节不包含在 length 中, 所以此记录总长是 32 字节。

另一个尺寸例子使用 null key 和 UTF-8 value “值” (假定 UTF-8 字节为 e580bc, 共 3 字节): length=28+0+3=31, 总空间 4+31=35 字节。空 key 会编码为 keyLength=0, 和 null key 的 -1 不相同; 这不是上面的空 value golden。

可观测例子与测试点。Step01Test 用独立字节 golden 比较 encode/decode, 覆盖 null key 与空 key、零长 value、连续解码、多种 ByteBuffer 视图、每个不完整前缀、越界 length、CRC/结构损坏以及最大长度。关键方法: encodeAndDecodeMatchIndependentBigEndianGoldenRecords 检查 golden, decodesOneRecordFromHeapDirectReadOnlyAndSliceBuffers 检查 buffer 视图, rejectsEveryIncompletePrefixAndOutOfRangeRecordLength 与 rejectsCorruptionInEveryStoredField 检查错误边界, enforcesLengthLimitAndSupportsLargestNonnegativeLogicalValues 检查最大值。最大普通记录的 length=1,048,576、总长 =1,048,580; 再多一个 value 字节必须拒绝。

常见误区与真实 Kafka 的差异

不能把 length 前缀算进 length, 也不能把 CRC 字段本身放进 CRC 覆盖区。不要把 Unicode 字符数当作 valueBytes。CRC 匹配只说明这些被保护的字节与校验值一致, 不证明来源可信。

分级提示 (不是答案)

- 一级 先把 4 字节 length、4 字节 CRC、两个 8 字节整数、两个 4 字节长度和变长尾部逐项列出。
- 二级 把 length 的取值范围、它覆盖的字段和总分配字节数分开计算; UTF-8 先转成字节再数长度。
- 三级 逐个检查解码失败发生在“不完整”还是“完整但非法”, 再用成功后 position/limit 与后缀 sentinel 验证边界。

命令与晋级。 ./gradlew :stepTest -Pstep=1 用于定位本步; ./gradlew :test -Pstep=1 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. 为什么 null key 与零字节空 key 必须使用不同表示?
2. CRC 校验成功能说明什么, 不能说明什么?

自检参考解释 null key 表示没有 key, 空 key 表示存在一个长度为零的 key; 把两者合并会丢失调用者数据。CRC32C 可以发现覆盖字节的意外变化, 但不能证明发送者身份, 也不能代替结构

字段校验。

2.2 第 2 步：日志追加

目标与前置知识

本步目标：实现单段日志的连续 offset 分配，并在成功返回前完成课程规定的写入。

开始前：复用记录编码，理解共享状态的锁、完整写循环和文件字节位置。

背景与推理

追加 (append) 把新记录放到已有日志尾部，不覆盖旧记录。一个分区日志把“下一条 offset”和文件尾字节位置作为共享状态；锁 (lock) 是保护共享状态的一段互斥操作，同一时刻只允许一个追加批次读取/更新这些状态，否则两个线程可能分配到同一 offset 或把批次字节交错。一次 `FileChannel.write` 允许只写出部分 buffer，提供的 `ChannelIO.writeFully` 会继续写完。课程要求完整批次写完并调用 `force(false)` 后才发布结果。

起点与编辑位置。

已给：`SegmentLog.create(Path,long)` 建立新段，段对象、`FileChannel`、锁、`baseOffset`、`nextOffset`、`close()`、`RecordCodec.encodedSize(RecordData)` 和 `ChannelIO.writeFully(...)` 已提供。

请修改：`exercises/src/main/java/io/simplekafka/storage/SegmentLog.java`：实现 `public AppendResult append(List<RecordData> records)`。

开始前先读

先完成 [第 1 步（记录编解码）](#)，尤其要能算出一条记录完整编码后的字节数，并知道非法记录在写入前可拒绝。测试从非零 `baseOffset` 开始，因此不要假定每个日志都从 offset 0 起。

输入、输出与状态

- 输入是按调用顺序写入的非空 `List<RecordData>`；每条 data 要满足第 1 步的值、时间戳及最大长度规则。输出 `AppendResult(firstOffset,nextOffset)` 是左闭右开区间，含 `firstOffset`、不含 `nextOffset`。
- 空/null batch、null 元素、非法数据、单条超限，或 `nextOffset` 无法继续递增时以 `INVALID_REQUEST` 拒绝。已知非法输入必须在任何写盘前发现。
- 成功时文件在尾部增加整个 batch，LEO 移至 `nextOffset`；实际文件写入或 `force` 的 I/O 错误报 `STORAGE_ERROR`。I/O 失败可能已经写入部分或全部字节，课程不承诺回滚；因此不能声称失败后磁盘必定未改变。已关闭段也不可追加。

行为契约与不变量。在锁内以当前 LEO 分配连续 offset，保持一个 batch 的记录相邻且不与并发 batch 交错；全部记录先预检，再写入。只有完整 `writeFully` 与 `force` 成功后才返回区间并发布新的 LEO。无效输入不改变文件和 LEO。

手算一个完整追加

初始段以 `baseOffset=5` 创建，文件为空，`LEO=5`。追加三条普通记录，每条假定 `null key`、1 字节 UTF-8 value，因此每条占 33 字节：先调用 `append([a,b])`，写入 offsets 5、6，返回 `[5,7)`，文件长 66、`LEO=7`；再调用 `append([c])`，写入 offset 7，返回 `[7,8)`，文件长 99、`LEO=8`。之后若有一条非法记录，拒绝后文件和 `LEO` 仍为 99 与 8，下一条有效记录从 8 开始。

可观测例子与测试点。 `Step02Test` 的 `appendsBatchesFromNonzeroBaseAndPreservesIndependentDiskEncoding` 验证 `base=5` 时的 `[5,7)`、`[7,8)` 和磁盘字段；`rejectsInvalidBatchesAtomicallyWithoutCreatingOffsetGaps` 验证空/`null`/含 `null`/超长 batch 拒绝后文件、`LEO` 与后续 offset；`rejectsAppendAtMaximumOffsetWithoutChangingEmptyFile` 验证溢出；`concurrentBatchesReceiveDisjointRangesWithoutInterleaving` 验证并发区间不重叠、不交错。

常见误区与真实 Kafka 的差异

不能只在末尾发现第三条无效后才返回错误，否则前两条可能已经写进日志。也不能用原始 value 长度替代 `RecordCodec` 的完整编码大小。`force` 成功是课程向上层返回的条件，不是断电后物理介质绝不丢数据的证明。

分级提示（不是答案）

- 一级 先写出调用前的 `LEO`，再把它作为 batch 首 offset；`nextOffset` 是下一批的起点。
- 二级 把整批输入预检与持锁读取当前 `LEO` 分开考虑，并且用 `writeFully` 处理部分写。
- 三级 并发测试检查两个 `AppendResult` 区间和文件中每个 offset 的顺序，不要只断言最终 `LEO`。

命令与晋级。 `./gradlew :stepTest -Pstep=2` 用于定位本步；`./gradlew :test -Pstep=2` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么整批预检必须发生在第一次磁盘写入之前？
2. 一条写入若抛出 I/O 错误，为什么不能自动断言它完全没写入？

自检参考解释 整批预检能使可预见的参数错误不留下半个 batch，也不消耗 offset。I/O 错误发生在操作系统已经接受部分字节之后也可能抛出，故失败返回不等于磁盘回滚；恢复步骤必须辨认合法边界。

2.3 第 3 步：按位点读取

目标与前置知识

本步目标：从逻辑 offset 读取完整记录，并同时遵守记录数与编码字节预算。

开始前：区分逻辑位点、文件字节位置、hint（搜索提示）和追加游标。

背景与推理

调用者用 offset 指定想要的第一条逻辑记录；文件读取器需要从某个 byte position 开始扫描。offset 是分区日志的编号，byte position 是本段文件内从 0 起算的字节数，它们单位不同、不能互换。bytePositionHint 只帮助减少扫描：必须是记录边界且不能跳过目标；真正返回仍从请求 offset 起。记录不可拆分，maxBytes 按完整编码记录计（包含 4 字节 length 前缀）。显式文件位置读取也让读操作不必改变追加位置。

起点与编辑位置。

已给：段的创建、LEO、锁与记录解码已提供；read(long,int,int) 会使用段起点作为默认 hint。

请修改：exercises/src/main/java/io/simplekafka/storage/SegmentLog.java：实现 public List<LogRecord> readFrom(long offset, long bytePositionHint, int maxRecords, int maxBytes)。

开始前先读

完成 [第 2 步（日志追加）](#)，理解成功追加后 offset 连续、文件按完整编码字节增长。没有稀疏索引时可以从段首找目标；本步的 hint 只是物理起点，不会改变逻辑请求。

输入、输出与状态

- offset 必须在 [baseOffset, LEO]；正好等于 LEO 是合法空读。maxRecords 与 maxBytes 都必须大于 0。返回记录按 offset 连续，且不超过任一预算。
- bytePositionHint 是本段文件中的非负 byte position，必须位于实际记录边界且不晚于请求 offset 的物理位置；文件末尾 hint 仅当 offset=LEO 合法。负数、越过目标、落在记录内部或越过 EOF 的 hint 报 INVALID_REQUEST；offset 越过范围报 OFFSET_OUT_OF_RANGE；磁盘上完整记录损坏报 CORRUPT_RECORD；段已关闭或底层 I/O 失败报 STORAGE_ERROR。
- 若第一条完整记录本身大于 maxBytes，返回空列表，不返回半条。成功或失败读取均不追加、不更改 LEO，也不推进可供后续读取使用的持久消费进度。

行为契约与不变量。从 hint 验证后的记录边界向前扫描，忽略 hint 所在 offset 小于目标的完整记录，输出必须从请求 offset 开始。每条记录完整后才计入记录数和 byte budget；只读不移动追加游标。

手算完整记录预算与提示

假设段 base=0，依次追加 5 条记录；每条 null key、1 字节 value、普通格式，因此都占 $32 + 1 = 33$ 字节。它们的 offset 为 0-4，文件起始 position 分别为 0、33、66、99、132，EOF 为 165。调用 readFrom(2, 33, 2, 66) 时 hint=33 指向 offset 1 的合法边界，读取器跳过 offset 1

后仍从 offset 2 返回 offsets 2、3。预算 32 返回 0 条；33 返回 1 条；65 仍只能容纳 1 条；66 正好容纳 2 条。offset=5 且 hint=165 返回空。

可观测例子与测试点。 Step03Test 的 validatesRecordBoundaryHintsAndWholeRecordBudgets 检查 33-byte 位置、hint -1/1/67/166 和跳过目标的 165、预算及错误码；enforcesNonzeroBaseBoundsAndKeepsReadPagingSeparateFromAppendPosition 检查非零 base 和分页不改追加位点；rereadsCompleteRecordsAndRejectsDamageWrittenThroughAnotherHandle 检查外部损坏报 CORRUPT_RECORD。

常见误区与真实 Kafka 的差异

把 offset=2 当成文件第 2 字节会从记录中间开始；把 hint 当成返回首 offset 会漏掉目标之前之后的语义。字节预算不是 payload 大小，长度、CRC、字段头都占磁盘空间。

分级提示（不是答案）

一级 画出 offset→ 文件起始 position 的两行映射，明确每个量的单位。

二级 再验证 hint 不晚于目标、处于记录边界；输出从 offset 开始而不是从 hint 开始。

三级 对 32、33、65、66 分别累计完整记录大小，并单独测首记录超过预算时的空结果。

命令与晋级。 ./gradlew :stepTest -Pstep=3 用于定位本步；./gradlew :test -Pstep=3 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为何 hint 指到 offset 1 的边界时，读取 offset 2 不能把 offset 1 返回？
2. 当第一条记录放不进 maxBytes 时，为什么返回部分记录不是合理选择？

自检参考解释 hint 只帮助定位搜索起点，请求 offset 才定义返回范围。记录含长度、校验和字段，部分字节不是可解码的完整记录；返回空让调用者知道本次没有完整记录适配预算。

2.4 第 4 步：崩溃恢复

目标与前置知识

本步目标：重启时恢复一段连续日志的 LEO，并只截去能够证明不完整的文件尾。

开始前：区分未写完的记录与一条完整但损坏的记录。

背景与推理

进程可能在写 length、固定字段或 value 的中途退出。恢复从 baseOffset 开始，将每个完整记录的 offset 与“下一条期望 offset”逐条比较。日志边界由 length 和完整字段共同决定：EOF

不足以组成下一条记录时，可以安全回到该记录起点；但完整记录的 CRC、字段或连续 offset 不合法，属于损坏证据，不能静默截掉。

起点与编辑位置。

已给：SegmentLog.open(Path,long) 会打开现有文件并调用恢复；文件位置、baseOffset、LEO 字段与第 1-3 步 codec/读写路径已给定。

请修改：exercises/src/main/java/io/simplekafka/storage/SegmentLog.java：实现 public long recover()。

开始前先读

先完成 第 3 步（按位点读取），能够从长度前缀和固定字段辨认完整记录边界。此方法既被显式调用，也在 open 打开旧文件时调用；打开成功后再 recover 一次应得到同一结果。

输入、输出与状态

- 输入是打开的段及其 baseOffset；空段允许从任意非负 base 开始。输出为恢复后的 LEO，即下一条应追加的 offset；空段的 LEO=baseOffset。
- 剩余字节少于 4，length 前缀不完整：截到当前记录起点。完整读到 length 但值不在 [28,1,048,576]：CORRUPT_RECORD，即使 body 尚未出现也不能当残尾。length 合法但声明的 body 尚不完整：截到当前记录起点。
- 完整记录 CRC/字段非法或 offset 不等于下一期望值：CORRUPT_RECORD，拒绝并保持原文件字节不变。段已关闭或底层 I/O 失败报 STORAGE_ERROR，I/O cause 按异常契约保留。

行为契约与不变量。成功恢复后文件只保留连续、完整、有效的前缀，LEO 指向其末端；只允许修复尾部不完整记录，不跳过或改写任何完整损坏记录。再次 recover 或关闭后 reopen 必须幂等。

手算恢复并继续追加

假定 base=0，文件含两条完整普通记录：null key、每条 value 为 1 byte，所以 offsets 0、1 各占 33 bytes，完整前缀长 66，下一 offset 为 2。offset 2 的第三条记录同样声明总长 33；文件只再含该条前 17 bytes（其中已包括完整 length=29，但 body 不完整），总文件长 83。recover 截到 byte position 66，返回 LEO=2；随后追加 value 为 1 byte 的一条记录，获得 [2,3)。相反，若第三条只写了完整 length 前缀而声明 length=1,048,577，则长度本身非法，文件不截断并报 CORRUPT_RECORD。

可观测例子与测试点。Step04Test 的 truncatesEveryIncompleteTailPrefixAndKeepsACompleteThirdRecord 与 truncatesAnIncompleteFirstRecordAtZeroAndNonzeroBases 验证只截去不完整尾；recoversEmptyAndNonzeroBaseFilesIdempotentlyAndRejectsWrongBase 检查空段、非零 base、重复恢复；rejectsCompleteCorruptionAndOffsetDiscontinuitiesWithoutChangingFileBytes 验证 CRC、字段、非法长度和 offset 断裂不改文件。

常见误区与真实 Kafka 的差异

把“任何解析失败”都当残尾，会吞掉完整记录损坏；把任何 EOF 都当坏文件，又无法恢复正常写入中断。只有合法 length 表明“期望多少 body”之后，才可区分短 body 与非法长度。

分级提示（不是答案）

一级 扫描状态只需记录当前有效 byte boundary 与下一期望 offset。

二级 按“prefix 不足 4 / length 非法 / length 合法但 body 不足 / 完整有效或完整损坏”分类。

三级 用两条有效记录加半条尾记录 reopen；再单独用相同前缀加 CRC 损坏的完整记录，并比较错误码与文件字节。

命令与晋级。 `./gradlew :stepTest -Pstep=4` 用于定位本步；`./gradlew :test -Pstep=4` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么完整 length 前缀中的非法数值不能被当成“body 写了一半”？
2. 恢复后为什么返回 LEO=2，而不是最后一条记录的 offset=1？

自检参考解释 非法 length 已经给出无法接受的完整声明，不是“合法记录没写完”，因此是 corruption。LEO 是下一条可追加的位置；最后已有记录的 offset=1，后续必须写 2。

第三章 分段、索引、保留与截断

单个日志文件增长后，重启扫描会越来越长；若只想清理旧数据，也无法在文件中间删除而不重写其余内容。分区日志因此由多个 segment（段文件）首尾组成：每段存一段连续 offset，当前 active segment 接收追加，已封闭的段不再追加。段内 byte position 是文件字节数，不是分区 offset。sparse index（稀疏索引）只保存若干 offset→byte position 提示，可从主日志重建；它加快查找，却不保存记录真相。retention（保留）删除旧的完整段前缀；truncation（截断）删除某个 nextOffset 之后的尾部，两者方向和目的不同。

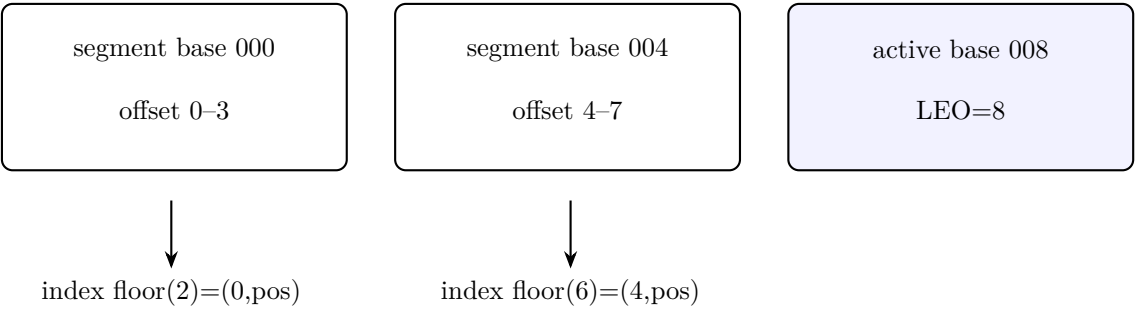


图 3.1: 分段日志的逻辑范围与段内稀疏索引。段文件名中的 base offset 是该段逻辑起点。

3.1 第 5 步：稀疏索引

目标与前置知识

本步目标：构建可持久化、可重建的 offset 到 byte-position 稀疏映射。

开始前：理解严格递增观察、floor 查找和主日志扫描。

背景与推理

一个索引项是两个大端 int64: offset, position，每项 16 字节。索引不是每条记录都存点，而是按段内逻辑 offset 与 base 的距离每隔 intervalRecords 条保存一个点；首条记录总要保存。floor(target) 选择不大于目标的最大已保存 offset，让后续扫描从已知记录边界开始。即使没有索引项，也能从 (baseOffset, 0) 安全扫描。索引缺失或损坏可从主日志重建，主日志损坏则不能靠索引修好。

起点与编辑位置。

已给: `SparseIndex` 的路径、`baseOffset`、`intervalRecords`、持久文件、加载和关闭已提供; `SegmentLog` 可供重扫主日志。

请修改: 实现 `exercises/src/main/java/io/simplekafka/storage/SparseIndex.java` 的 `public synchronized void add(long offset, long position)`、`public synchronized IndexEntry floor(long offset)` 和 `public synchronized void rebuild(Path logFile, long requestedBaseOffset)`。

开始前先读

完成 第 4 步 (崩溃恢复), 能从主日志验证完整记录、CRC 与连续 offset。索引项中的 position 单位是该段文件的字节, offset 是分区内记录编号; 两者不能互换。

输入、输出与状态

- `add` 接收一条已观察记录的逻辑 offset 与其段内 byte position。首次观察必须是 (`baseOffset`, 0); 后续每次观察的 offset 与 position 都必须严格递增, position 非负。即使某次观察不落盘, 它仍参与下一次顺序检查。
- 只有 $(\text{offset} - \text{baseOffset}) \bmod \text{intervalRecords} = 0$ 的观察持久化; 首次 (`baseOffset`, 0) 一定持久化。拒绝的观察报 `INVALID_REQUEST`, 不改变索引文件或顺序状态。索引关闭或底层 I/O 失败报 `STORAGE_ERROR`。 `floor` 若索引已关闭也报 `STORAGE_ERROR`。
- `floor` 返回不大于 target 的最大索引项; 若没有合格项 (包括 target 小于 base), 返回 (`baseOffset`, 0)。 `rebuild` 接收主日志路径与必须匹配的 base, 清除旧索引后扫描并验证主日志, 重建所有应保存的点; 日志内容本身不改。base 不匹配/路径非法报 `INVALID_REQUEST`, 日志损坏报 `CORRUPT_RECORD`, 已关闭索引或底层 I/O 失败报 `STORAGE_ERROR`。

行为契约与不变量。索引文件每项按大端顺序保存 8-byte offset 与 8-byte position。所有观察用于严格递增校验, 落盘频率按 offset 相对 base 的间隔计算, 而不是按 add 调用次数或固定 byte 间隔计算。丢失、截短、陈旧或非法索引必须可从当前日志重建; 重建不能使坏日志看似有效。

手算索引项与 floor

假定 `baseOffset=8`、`intervalRecords=2`, 段中有 offsets 8、9、10、11 四条记录, 每条 null key、1-byte value, 故每条 33 bytes, 起始 position 分别为 0、33、66、99。依次观察四项: offset 8 的第一项写入 (8,0), 9 的观察只校验顺序, 10 与 base 相差 2 因而写入 (10,66), 11 仍只校验顺序。索引文件恰含两个 16-byte 项; `floor(11)=(10,66)`, `floor(9)=(8,0)`, `floor(7)` 的无候选约定也是 (8,0)。删掉索引后从四条仍完好的日志扫描, 会重新得到同样两项。

可观测例子与测试点。 `Step05Test: writesBigEndianEntriesAtIntervalAndFloorsSafely` 检查 16-byte 项、间隔和 floor; `supportsNonzeroBasesAndReopensPersistedPoints` 检查非零 base 与重开; `rejectedObservationsLeaveTheIndexAndOrderingStateUnchanged` 检查拒绝后状态不变; `rebuildsMissingTruncatedAndStaleIndexesFromTheCurrentLog` 与 `rebuildRejectsMalformedLogsWithoutChangingTheirBytes` 分别检查索引重建和坏日志拒绝。

常见误区与真实 Kafka 的差异

floor 不是“最接近的任意项”：只能取不晚于目标的项。索引位置必须指向记录开头，否则读者会把 payload 当 header。索引可以丢弃重建，主日志不可以用猜测替代。

分级提示（不是答案）

一级 写出候选条件 $\text{offset} \leq \text{target}$ ，再定义没有候选时的结果：(base,0)。

二级 把每个 observation 的顺序状态与该位置是否已持久化分开；区间以 offset-base 为准。

三级 删除索引文件，再创建 SparseIndex，通过真实日志读取观察重建，而不只检查内存 floor。

命令与晋级。 ./gradlew :stepTest -Pstep=5 用于定位本步；./gradlew :test -Pstep=5 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么 floor 找到索引项后仍要逐条扫描主日志？
2. 索引坏了可以重建，为什么主日志坏了不能采用同一种恢复方式？

自检参考解释 索引只提供一个不越过目标的搜索起点，通常没有目标本身的精确项，所以仍要验证并扫描记录。索引是冗余提示，可由记录事实重算；坏日志中的记录内容没有另一个可信来源可推导。

3.2 第 6 步：分段日志

目标与前置知识

本步目标：把单段日志组合为按容量滚动的分区日志，并使读取跨段连续。

开始前：理解 active segment、完整记录容量、索引提示和全局 LEO。

背景与推理

PartitionLog 以 baseOffset 顺序持有多个 SegmentLog。一个 active segment（活动段）是当前可继续追加的最后一段；它装不下下一条完整记录时，先封闭旧段，再以当前分区 LEO 创建新段。容量以完整编码记录字节数计算，而不是 value 长度。若段为空，允许一条本身合法但超过 segmentBytes 的记录独占它；否则永远无法追加这种记录。跨段读取找到包含目标 offset 的段后用该段 index floor 作为 hint，再向后扫描；maxRecords/maxBytes 是整次读取的总预算，不在换段时重置。

起点与编辑位置。

已给：每个段的 create/open/recover、稀疏索引、文件发现和资源关闭路径已提供；PartitionLog 的 mutationVersion() 与私有 markMutation() 已给定。

请修改：实现 `exercises/src/main/java/io/simplekafka/storage/PartitionLog.java` 的 `public synchronized AppendResult append(List<RecordData> records)` 和 `public List<LogRecord> read(long offset, int maxRecords, int maxBytes)`。构造器 `PartitionLog(Path, long, int)` 已提供。

开始前先读

完成 [第 5 步（稀疏索引）](#)；需理解 `SegmentLog` 的追加/读取与 `index` 的 `floor` 提示。段文件名只认 20 位数字后缀 `.log` 的普通文件，配对 `.index` 属于索引；别的文件或目录不是可清理的 `segment`。

输入、输出与状态

- `append` 输入为顺序 batch，输出整批全局 `[firstOffset, nextOffset)`；`offset` 在段边界仍连续。空/非法 batch 或 `offset` 溢出以 `INVALID_REQUEST` 拒绝，且不得先创建新段。段关闭或底层 I/O 失败报 `STORAGE_ERROR`；有效追加开始后的失败不保证回滚文件。
- `read` 的 `offset` 范围为保留区间 `[logStartOffset, LEO]`，`LEO` 返回空；`maxRecords`/`maxBytes` 必须为正。返回 `offset` 有序且连续，跨所有段共享总记录数与完整编码字节预算。`offset` 越界为 `OFFSET_OUT_OF_RANGE`，发现段间缺口或坏记录为 `CORRUPT_RECORD`，I/O/关闭错误为 `STORAGE_ERROR`。首条超过 byte budget 时返回空。
- 每个通过完整预检的 `append batch` 在首次文件写入/滚段前令本打开对象的 `mutationVersion` 加 1，跨多个段也只加一次；无效输入不增加。它是内存变更代数，不写入磁盘；写入开始后的 I/O 失败也不能使旧 `proof` 继续可信。`LEO` 与段文件在成功追加后更新，读取不改它们。

行为契约与不变量。只有非空 `active` 段放不下下一条完整记录时才滚段；空段可独占大于目标容量的合法单条记录。整批预检先于首次创建/滚动，避免无效 batch 留下空段。`reopen` 扫描已登记的段恢复 `LEO`；未知文件、目录必须保留。

手算滚段和跨段预算

情形 A：`segmentBytes=64`，三条普通记录各为 `null key`、空 `value`、总长 32 bytes。`offset 0, 1` 正好占满 `base 0` 段 64 bytes；`offset 2` 前滚到 `base 2`，新段大小 32。一次 `append` 返回 `[0,3)`，`bases=[0,2]`、`sizes=[64,32]`。情形 B：三条依次占 32、132、32 bytes，`segmentBytes` 仍为 64；132-byte 记录不适合已有非空段，于 `base 1` 单独占段，随后 32-byte 记录在 `base 2` 开新段，三个段大小 `[32,132,32]`。段能超过目标值，但只有因为单条合法记录本身超限且段为空。

对情形 A，从 `offset 0` 调用 `read(0,3,64)` 只能返回 0、1；预算 96 可返回 0、1、2。切换到第二段时剩余预算为 32，不可重置为 64。

可观测例子与测试点。`Step06Test: rollsAtExactByteBoundariesAndWritesIndependentRecords` 检查滚段容量；`appliesReadBudgetsAcrossSegmentsAndValidatesBoundsAndLimits` 检查跨段读取预算；`invalidBatchDoesNotMutateAnExistingMultiSegmentLog` 检查非法 batch 不改目录；`acceptsAnOversizedRecordBetweenRecordsInOneBatchAndReopensEv`

`everySegment` 检查超大单条独占段；`recoversOnlyAnIncompleteActiveTailAndRebuildsIndexesOnReopen` 与 `refusesCompleteCrcCorruptionAndSegmentBaseGapsWithoutChangingLogs` 检查恢复边界；`concurrentBatchesStayContiguousAndSurviveReopen` 检查并发重开连续性。

常见误区与真实 Kafka 的差异

如果只计算 `payload` 会漏掉长度、CRC 与固定字段；如果每个 `segment` 都重新得到完整 `maxBytes`，跨段读取会超预算。单段 `append` 不负责滚段，滚段责任属于 `PartitionLog`。

分级提示（不是答案）

一级 先用 `RecordCodec.encodedSize` 得到每条完整记录空间，再模拟当前 `active` 的剩余空间。

二级 沿 `batch` 逐条维护全局 `nextOffset` 和当前段字节数；在新段启动后仍保留同一读取预算。

三级 分别核对目录中的 `base`、每个段的精确大小、`mutationVersion` 与 `reopen` 后逐条读取结果。

命令与晋级。 `./gradlew :stepTest -Pstep=6` 用于定位本步；`./gradlew :test -Pstep=6` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么非法 `batch` 必须在创建下一段之前拒绝？
2. 读取从一个段进入下一个段时，字节预算要带着什么状态继续？

自检参考解释 先校验可避免无效请求产生空段或改变目录；`byte budget` 是一次调用的累计消耗，换物理文件不会把已经用掉的额度返还。

3.3 第 7 步：日志保留

目标与前置知识

本步目标：删除已过期的封闭段前缀，同时保留活动段、日志末端和真实新起点。

开始前：区分物理日志保留与消费者进度。

背景与推理

`PartitionLog` 当前保留区间是 `[logStartOffset, LEO)`。`deleteBefore` 是调用者显式触发的同步操作，不会因 `consumer` 已经提交位点而自动运行。段不可按单条记录切开；只能删除封闭段，而且最后一个段必须保留为 `active` 以便追加。新起点是删除后第一段的 `base`，可能小于传入阈值，因为段边界不必刚好落在阈值上。

起点与编辑位置。

已给: PartitionLog 已提供多段发现、当前起点/末端、关闭路径, 以及 `mutationVersion()` 和私有 `markMutation()`。

请修改: 实现 `public synchronized long deleteBefore(long offset)`, 文件位于 `exercises/src/main/java/io/simplekafka/storage/PartitionLog.java`。

开始前先读

完成 **第 6 步 (分段日志)**, 理解段 base 与下一段 base 界定的 offset 范围, 以及 active 段不能删除。consumer commit 是另一种状态, 不是本方法的输入。

输入、输出与状态

- offset 是非负 long 的保留下界; 负值报 `INVALID_REQUEST` 且不改状态。方法返回实际 `logStartOffset`, 不是原样返回请求阈值。
- 只删除 `endOffset` (下一段 base) 不大于阈值的封闭段及同 base 的 `.index`; active 段必须保留, 至少保留一段。未知文件/目录不删除。删除不会移动 LEO、改变保留段内容或自动调整消费者位点。
- 实际删掉至少一段时, 首次删除前对 `mutationVersion` 加 1 (本次调用只加一次)。没有符合条件的段时返回当前起点且版本不变。删除后的 read 若 offset 小于新起点报 `OFFSET_OUT_OF_RANGE`, 不得静默 seek 到起点。已关闭日志或底层 I/O 失败报 `STORAGE_ERROR`; 若文件删除中途失败, 不承诺将已经发生的删除回滚。

行为契约与不变量。 仅以完整封闭段作为删除单位, 成对删除日志与索引, 不能删除 active 或最后剩余段。重开后仍须恢复相同 `logStartOffset` 与 LEO; no-op 不增加 `mutationVersion`。

手算保留边界

假定 9 条记录 offsets 0-8, 每条 null key、空 value, 占 32 bytes; `segmentBytes=96`, 所以三段为 base 0、3、6, 每段 3 条、96 bytes, LEO=9, base 6 是 active。deleteBefore(4) 只删除 end=3 的 base 0 段, 实际起点 =3; deleteBefore(9) 再删除 end=6 的 base 3 段, 起点 =6; deleteBefore(1000) 仍保留 active base 6, 起点仍 =6。三次调用中 LEO 始终为 9; 第三次没有删除, 不增加 `mutationVersion`。阈值 4 不对应新起点 3, 正因为删除单位是段。

可观测例子与测试点。 `Step07Test: deletesOnlyEligibleClosedSegmentsAndKeepsTheActiveSegment` 检查 `end<=threshold`、配对索引、start/LEO 与未知文件; `retentionKeepsTheOnlyActiveSegmentWhetherEmptyOrNonempty` 检查唯一一段不删除; `deletesClosedSegmentAtItsExactEndOffset` 检查等于阈值的边界可删。

常见误区与真实 Kafka 的差异

若只删.log 会留下悬空索引; 若把阈值当成“删除所有 `offset< 阈值`”, 会错误地切开段或删除 active。提交位点和日志文件删除互相独立。

分级提示（不是答案）

一级 逐段写出 base、exclusive end 和 active 状态，再判断 $\text{end} \leq \text{阈值}$ 。

二级 单独演算只有一个空段、只有一个非空段的情况，确保仍至少留下 active。

三级 删除后列出 .log/.index 文件名并重开，分别验证新起点前、边界处和 LEO 的读取。

命令与晋级。 ./gradlew :stepTest -Pstep=7 用于定位本步；./gradlew :test -Pstep=7 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么 deleteBefore(4) 可能返回 logStartOffset=3 而不是 4？
2. 即使某个 consumer 已提交到 9，为什么这本身不能触发删除？

自检参考解释 retention 以完整 segment 为单位，base 3 才是可保留前缀的下一边界。提交 offset 只记录处理进度；它不会自动删除文件，也不证明其他 consumer 不再需要旧数据。

3.4 第 8 步：尾部截断

目标与前置知识

本步目标：按 nextOffset 删除一段日志的分叉尾部，并使该日志可从目标位点继续追加。

开始前：理解 half-open 范围、分段边界和索引同步。

背景与推理

truncateTo(nextOffset) 的 nextOffset 是“下一条要写的位置”：保留所有 $\text{offset} < \text{nextOffset}$ 的记录，移除其余尾部。它与 retention 相反——retention 删除旧前缀，truncation 删除新后缀。若目标位于段中，要截断该段到目标记录的 byte boundary 并重建索引；位于段 base 时要留下以该 base 为 active 的空段。这个本地日志操作本身不证明某个副本切换安全，也不改 consumer 位点。

起点与编辑位置。

已给：前几步已建立段关闭、文件截断、索引重建和全局段表；PartitionLog 的 mutationVersion() 与私有 markMutation() 已给定。

请修改：实现 public synchronized void truncateTo(long nextOffset)，文件位于 exercises/src/main/java/io/simplekafka/storage/PartitionLog.java。

开始前先读

完成 **第 7 步（日志保留）**，知道实际 logStartOffset 可能大于 0，也知道 LEO 是 exclusive end。截断只能发生在保留区间内，不可恢复已被 retention 删除的前缀。

输入、输出与状态

- `nextOffset` 合法范围为闭区间 $[\text{logStartOffset}, \text{LEO}]$ ；超出范围报 `OFFSET_OUT_OF_RANGE`，且文件、索引、段表、LEO 与 `mutationVersion` 都不变。方法没有返回值。
- 成功保留所有 $\text{offset} < \text{nextOffset}$ 的记录，删除之后的记录/段及对应索引，LEO 变成 `nextOffset`。`nextOffset=LEO` 是 no-op；有效改变在首次写或删文件前把 `mutationVersion` 增加一次，无效与 no-op 不增加。未知文件保留。已关闭日志或底层 I/O 失败报 `STORAGE_ERROR`；开始有效修改后的 I/O 失败不承诺回滚，已增加的 `mutationVersion` 不回退。
- 成功后可立刻 `append`，第一条新记录的 `offset` 必须正好等于 `nextOffset`。重开时索引与活动段状态须与截断后的日志一致。

行为契约与不变量。截断边界永远表示下一条记录的 `offset`；没有“保留记录 5 并写 5”的含义。目标在段边界时必须保留能接收后续追加的 active 段；目标在段内时同步裁剪日志尾和索引。

手算四个截断结果

沿用 Step 7 的 fixture：假定记录 0–8 各为 32 bytes, `segmentBytes=96`, 三个完整段 `bases=[0,3,6]`, `sizes=[96,96,96]`, `LEO=9`, `start=0`。四种独立初始状态相同、分别执行一次：

1. `truncateTo(5)`：保留 offsets 0–4；结果 `bases=[0,3]`、`sizes=[96,64]`、`LEO=5`。
2. `truncateTo(3)`：保留 0–2；结果 `bases=[0,3]`、`sizes=[96,0]`，base 3 的空段接收后续 `offset 3`。
3. `truncateTo(0)`：保留空日志；结果只有 base 0、size 0，下一次追加从 0 开始。
4. `truncateTo(9)`：目标等于 LEO，无变化；`bases/sizes` 仍为 $[0,3,6]/[96,96,96]$ ，`mutationVersion` 不增加。

可观测例子与测试点。Step08Test: `truncatesAtBoundariesWithinSegmentsAtLeoAndAtLogStart` 检查段内、段边界、LEO 与起点；`truncateRebuildsMixedSizeSegmentStateAndIndexPoints` 检查文件/索引同步；`invalidAndRepeatedTruncationsPreserveTheRetainedFiles` 检查无效与 no-op；`truncatesThenRetainsAcrossRestartAndContinuesAtTheReopenedEnd` 检查重开、保留和继续追加；`fixedSeedOperationsMatchAnIndependentSegmentAndOffsetModel` 检查多次状态转换。

常见误区与真实 Kafka 的差异

`truncateTo(5)` 保留 `offset 4`，不是保留到 `offset 5`；5 是下一条的位点。只改 log 文件而不裁剪/重建 index，会让读者继续跳到已删分叉记录。

分级提示（不是答案）

- 一级 先将 `nextOffset` 翻译成“保留 `offset` 小于它”，不要把它当作最后保留的 `offset`。
- 二级 分别画出目标位于段内、段 base、LEO 和 `logStart` 时应保留的 active 文件。
- 三级 检查磁盘文件与索引、关闭重开，再 `append` 一条并验证 `offset` 恰好等于目标。

命令与晋级。 `./gradlew :stepTest -Pstep=8` 用于定位本步；`./gradlew :test -Pstep=8` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. `truncateTo(5)` 后为什么下一条应写入 5 而不是 4？
2. 为什么截断记录文件尾后还需要处理稀疏索引？

自检参考解释 half-open 范围保留所有 `offset < 5` 的记录，因此最后保留 4、下一条为 5。索引是记录边界的物理提示，若留下旧尾索引，读取可能跳入已删除数据而不是当前日志。

第四章 分区与真实网络 broker

前两章已经把记录写成可恢复的单段/分段日志；本章让一个 topic（主题，消息的命名分类）拥有多个 partition（分区，每个分区是一条各自有序的日志），再让客户端通过 loopback TCP 与 broker（保存分区数据并响应请求的进程）通信。同一 topic 的每个 partition 都有自己的 offset 序列，例如 orders-0 的 offset 0 和 orders-1 的 offset 0 是两条不同记录，不能比较出 topic 的全局顺序。metadata（元数据）告诉客户端有哪些分区、leader broker 和连接地址。TCP 只交付有序字节流，不保留 Java write 的边界，故应用协议需要 frame 长度。课程协议仅用于教学，不兼容 Kafka wire protocol。

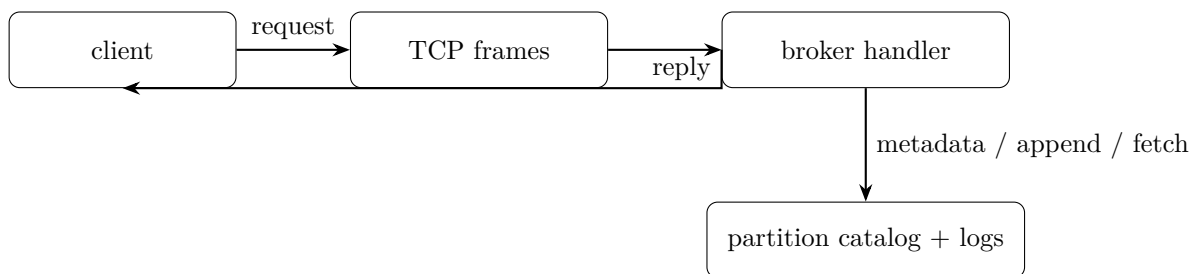


图 4.1: 一个连接上的请求/响应路径。message body 编解码由 provided 代码承担, 学生实现 frame 与业务分派。

4.1 第 9 步：分区路由

目标与前置知识

本步目标：让相同 key 稳定进入同一分区，并为 null key 提供可观察的轮转分配。

开始前：理解 key 字节、CRC32C unsigned 值与正分区数取模。

背景与推理

producer 在追加之前选择一个 partition，才能确定记录进入哪条有序日志。课程规则是：非 null key 取其字节的 CRC32C，把 32-bit 结果按 unsigned 数值解释，再对分区数取模；同一字节串与同一分区数会稳定选中同一分区。null key 没有可 hash 的字节，由同一个 Partitioner 实例在可用分区中轮转。空 byte array 是长度为零但确实存在的 key，不等同 null。

起点与编辑位置。

已给：无参 `Partitioner()` 构造器已提供，但轮转游标状态不是 `provided` 答案；学生可在本类添加实现所需实例状态。

请修改：实现 `exercises/src/main/java/io/simplekafka/client/Partitioner.java` 的 `public int choose(byte[] key, int partitions)`。

开始前先读

本步依赖 [第 8 步（尾部截断）](#)；上一章建立了分区日志可承载的连续 `offset`。本章开头已解释 `topic`、`partition` 与 `broker`：本步只决定写入哪个已有 `partition`，不创建主题或日志。

输入、输出与状态

- `key` 是 `null` 或任意非 `null` `byte array`（含空数组）；`partitions` 是可用分区数，必须 >0 。返回 `int` 分区编号，范围 $[0, partitions)$ 。
- 非 `null` `key` 使用 `unsigned(CRC32C(key)) mod partitions`，不读写 `null-key` 游标；空数组 `CRC32C=0`。`null` `key` 使用该实例游标：先归一化 `cursor mod partitions` 得到当前所选分区，再把游标设为 $(selectedPartition+1) mod partitions$ ；分区数变化时也按新的正数归一化。
- `partitions ≤ 0` 报 `INVALID_REQUEST` 且不推进游标。不同 `Partitioner` 实例互不共享游标；这是内存状态，没有磁盘或网络副作用。

行为契约与不变量。分区数必须先验证为正。Java `signed int` `CRC` 不可直接得到负分区余数，须按无符号 32-bit 数解释。只有 `null` `key` 调用轮转游标；`keyed` 与空数组选择都不改变该游标。分区数变动后每次 `null` 调用仍须返回有效编号。

手算 `key` 路由与游标

假定 `key` 字符串“123456789”使用 `ASCII` 字节，`CRC32C` 为 `0xe3069283`（`unsigned` 数值 3,808,858,755）；对 7 个分区取模为 2，所以返回 `partition 2`。空数组 `CRC` 为 0，7 分区时返回 0，但不改变 `null` 游标。全新实例的 `null` 调用若 `partitions` 依次为 3、3、1、3、3、2、2，结果依次为 0、1、0、0、1、0、1：分区数为 1 时归一化后的游标回到 0，后续使用新的分区数。

可观测例子与测试点。`Step09Test: routesByKnownUnsignedCrc32cVectorsAndKeepsEmptyKeyAsAKey` 检查 `CRC32C` 与空 `key`；`keyedAndEmptyKeysDoNotAdvanceTheNullKeyCursor` 检查游标不变；`nullKeyCursorsAreIndependentAndSinglePartitionAlwaysSelectsZero` 与 `nullKeyCursorStaysWithinPartitionsWhenTopicSizesChange` 检查实例隔离和分区数变化；`requiresAPositivePartitionCount` 检查非法 `count`。

常见误区与真实 *Kafka* 的差异

不要使用 Java 对象的 `hashCode()` 替代 `key bytes` 的 `CRC32C`，也不要对可能为负的 `signed int` 直接取余。真实 *Kafka* 的 `key hash` 与 `null-key sticky batching` 规则不同；本课程的实例级轮转不代表 *Kafka* 默认行为。

分级提示（不是答案）

一级 先画出 null、空数组、非空数组三种输入与各自是否影响游标。

二级 将 CRC 视为 0 到 $2^{32} - 1$ 的 unsigned 数，再对正分区数取模。

三级 手工逐次维护 null 游标；在 count 变化时先归一化，并插入一次 keyed 调用确认它不改变游标。

命令与晋级。 `./gradlew :stepTest -Pstep=9` 用于定位本步；`./gradlew :test -Pstep=9` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么分区 hash 应基于 key bytes，而不是 RecordData 对象的 Java hashCode？
2. null key 和零长度 key 在路由与状态更新上有什么不同？

自检参考解释 hash 作用于稳定的 key bytes，Java 对象 hashCode 不是课程规定且不保证同样的跨重启语义。null key 轮转并更新实例游标；空数组有实际 CRC32C 值 0，确定路由且不更新游标。

4.2 第 10 步：主题元数据

目标与前置知识

本步目标：在本地 catalog 登记固定分区集合，并返回可供客户端路由的确定性 metadata。

开始前：区分磁盘目录与内存中的 topic 注册。

背景与推理

topic 是读者使用的逻辑名称；其 partition 目录里保存日志文件。metadata 是 broker 对外给客户端的结构化描述，不是从任意磁盘目录推测出的目录清单。创建时固定分区数量后，单机 broker 对每个分区公布 TopicPartition、当前 broker leader、epoch、replicas、ISR 与 endpoint；本课程单机 epoch=0，replicas 和 ISR 都只有这个 broker。

起点与编辑位置。

已给：PartitionCatalog 构造器固定 root、broker id 和 endpoint；TopicPartition、PartitionMetadata、日志注册与目录根检查已提供。

请修改：实现 `exercises/src/main/java/io/simplekafka/broker/PartitionCatalog.java` 的 `public synchronized void createTopic(String topic, int partitionCount)` 与 `public synchronized List<PartitionMetadata> metadata(String topic)`。

开始前先读

完成 [第 9 步 \(分区路由\)](#)，知道 partition id 是 topic 内编号；并使用 [第 6 步 \(分段日志\)](#) 的 PartitionLog 作为每个分区的存储。metadata 只登记/描述已有分区，不负责 producer 选 key。

输入、输出与状态

- topic 必须是 1-255 个 ASCII 字节，字符限于 [A-Za-z0-9._-]，且不能是 . 或 ...。partitionCount 必须为正；这些检查要在创建 topic/partition 路径前完成。无效名称或数量报 INVALID_REQUEST，不得改目录。
- createTopic 成功无返回值，注册 partitions 0 到 count-1 并打开各自日志。同名同 count 再调用是幂等操作，不清空现有数据；同名不同 count 报 INVALID_REQUEST，原 metadata、文件与日志保持不变。底层创建/打开 I/O 错误为 STORAGE_ERROR。catalog 已关闭后调用则抛 IllegalStateException。
- metadata 按 partition id 升序返回列表；每项单机 leaderId= 当前 brokerId、epoch=0、replicas=[brokerId]、isr=[brokerId]，endpoint 为构造器地址。未登记的 topic 报 UNKNOWN_TOPIC_OR_PARTITION。新 catalog 实例的注册表是空的，必须显式 createTopic 才重新登记并打开目录；意外残留数字目录不会自动加入 metadata。catalog 已关闭后调用抛 IllegalStateException。

行为契约与不变量。创建前先验证标识符与正 partitionCount；同定义重复创建不改磁盘，不同数量拒绝且不变更旧目录。元数据排序稳定且不暴露 root 外路径。topic 名最大 255 字节是模型界限；具体文件系统若拒绝 255 字节单个目录名，可以导致目录创建失败，但 TopicPartition 本身仍接受该标识符。

手算创建、重开与注册

假定 brokerId=7、endpoint 已配置、root 为空。createTopic(orders,2) 后 metadata 顺序为 orders-0、orders-1，两项都 leader=7、epoch=0、replicas=isr=[7]。把 records 0、1 追加到 orders-0 后关闭并新建 Catalog：此时 metadata(orders) 是 UNKNOWN_TOPIC_OR_PARTITION，因为 topic 注册不是持久 metadata。再显式 createTopic(orders,2) 会打开同一目录，orders-0 仍可读 0、1；追加一条返回 [2,3)。即使磁盘上额外有 orders/9 目录，它也不会增加已登记的两项。

可观测例子与测试点。Step10Test: createsIndependentFixedTopicsAndReopensOnlyWithExplicitPartitionCounts 检查 metadata、磁盘日志与显式重登记；rejectsInvalidCountsAndTopicNamesWithoutMutatingTheFilesystemOrEscapingTheRoot 检查参数校验与目录不变；acceptsA255ByteTopicIdentifierWhereTheFilesystemAllowsIt 检查标识符长度边界。

常见误区与真实 Kafka 的差异

不可把用户 topic 未验证就拼接路径；也不可枚举现有目录来猜分区数，把手工残留误注册。真实 Kafka 的 topic 控制面与扩分区流程不由本课程 catalog 模拟。

分级提示（不是答案）

一级 先区分“内存注册表中的 topic”与“磁盘上存在的日志目录”。

二级 把同名同 count、同名不同 count、重启未注册三种调用分别写出状态结果。

三级 记录目录快照，在错误创建与重启前后对比，再显式注册后检查 offset 连续性。

命令与晋级。 `./gradlew :stepTest -Pstep=10` 用于定位本步；`./gradlew :test -Pstep=10` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么同名但分区数不同不能当成幂等成功？
2. 重启后已有的数字目录能证明哪些事实，又不能证明什么？

自检参考解释 分区数决定可路由的固定集合；悄悄接受另一数量会使同一 topic 的 metadata/日志不一致。目录证明某些文件存在，但不能证明它们属于当前登记的 topic 集合或决定未出现分区的数量。

4.3 第 11 步：TCP 帧协议

目标与前置知识

本步目标：把 TCP 字节流切分为有界帧，并在连接中逐帧往返请求或响应。

开始前：理解短读、长度前缀、EOF 位置和 correlation id。

背景与推理

TCP 提供有序字节流，却不保存应用每次 write 的边界：一次写入可能分多次 read 到达，多次写入也可能合并到一次 read。frame（帧）用自身 length 告诉读取方后面这一条协议单元有多少字节；固定 header 提供版本、API、请求关联号和错误码，payload 才是业务 body。correlationId 是请求与回复配对的编号。

起点与编辑位置。

已给：Frame 值类型、API id、ErrorCode wire id、Messages body 类型与 MessageCodec body codec 均已提供；本步不重写业务 body。

请修改：实现 `exercises/src/main/java/io/simplekafka/protocol/FrameCodec.java` 的 `public static void write(OutputStream output, Frame frame) throws IOException` 与 `public static Frame read(InputStream input) throws IOException`。

开始前先读

完成 **第 10 步（主题元数据）**，认识 API 通过协议请求 broker；并理解 **第 1 步（记录编解码）** 的大端定宽整数。这里的 frame 是课程 TCP 封装，不是磁盘 record，也不是 Kafka 官方请

求头。

输入、输出与状态

- `wire` 顺序固定为大端 `length:int32|version:int16(1)|api:int16|correlationId:int32|error:int16|payload`。length 不含自身的 4 字节，但包含其余 10-byte header 与 payload；合法范围 [10,8,388,608]，payload 最大 8,388,598 bytes，最大物理帧共 8,388,612 bytes。
- `write` 写一个 Frame；null 参数、未知 API 或总 length 越界时报 `INVALID_REQUEST`，并须在开始写流前拒绝。成功输出恰好一帧。底层流错误按同一 `IOException` 传播；发生实际 I/O 错误时不承诺外部流回滚。
- `read` 每次最多返回一帧。input 为 null 时报 `INVALID_REQUEST`。若第一个 length 字节前已 EOF，返回 null；若 length 前缀或 frame body 只收到一部分，报 `INVALID_REQUEST`。length 越界、version 非 1、未知 API 或未知 error wire id 也报 `INVALID_REQUEST`；检查长度后才按声明大小读取/分配。输入流 I/O 异常按原 `IOException` 传播。

行为契约与不变量。读循环必须填满所需字节，不能假定一次 read 得到 header/body。正常边界 EOF 才返回 null；半帧不是空结果。连续多帧按各自 length 消费且不吞下一帧。超长/非法前缀只能消耗已读的 4-byte length，不能继续消费其后缀或按恶意长度分配内存。frame 中已知 error id 只表示该帧的错误字段有效；provided server 还会在分派前以 `INVALID_REQUEST` 拒绝带非零 error 的请求帧。

逐字段解读 golden 帧

Step11Test 的请求/响应 golden 为 0000000d00010001000000290008010203。首 4 bytes 的 13 是 length，即之后 10-byte header 加 3-byte payload；version=0001=1，API=0001=METADATA，correlationId=00000029=41，error=0008=REQUEST_TIMEOUT，payload 为 010203。因此物理帧总长是 17 bytes。若没有 payload，length 仍为 10，完整物理帧长 14 bytes。

可观测例子与测试点。Step11Test: `writesAndReadsTheIndependentGoldenFrameAndHandlesEmptyPayload` 检查 golden；`acceptsMaximumFrameAndRejectsOneByteOverBeforeWritingAnything` 检查长度边界和零字节拒绝；`rejectsEveryIncompletePrefixOfTheGoldenFrame` 检查 EOF；`readsBackToBackFramesFromStreamsDeliveringOneTwoOrThreeBytesAtATime` 检查短读和连续帧；`rejectsInvalidLengthsWithoutConsumingBytesAfterTheHeader` 与 `rejectsUnknownVersionApiAndErrorIds` 检查格式边界；`propagatesTheSameUnderlyingIOExceptionFromReadAndWrite` 检查异常原样传播。

常见误区与真实 Kafka 的差异

一次 read 不保证填满 length 或 header；先按未经验证的 length 分配可能让无效输入放大内存请求。课程 version=1/API/错误号只是教学 wire contract，不能连接官方 Kafka client。

分级提示（不是答案）

一级 把固定 header 的字段宽度相加得到 length 最小值 10，再判断 length 是否包括自身。

二级 把 EOF 分为“还没开始新帧”与“帧已经开始却缺字节”，并让 readFully 循环处理短读。

三级 逐字节输入 golden，确认每次只消费一帧；给超长 length 后追加 sentinel，确认 sentinel 未被读取。

命令与晋级。 ./gradlew :stepTest -Pstep=11 用于定位本步；./gradlew :test -Pstep=11 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么回复要带回对应请求的 correlationId？
2. 干净 EOF 与半帧 EOF 分开报告有什么价值？

自检参考解释 关联号让客户端把并发或连续请求的回复配对到正确调用。边界 EOF 说明连接正常结束；半帧意味着协议数据被截断，不能把它伪装成“没有更多请求”。

4.4 第 12 步：单机 broker

目标与前置知识

本步目标：把 metadata、produce、fetch 业务请求接到本地 catalog，并用真实 loopback TCP 验证完整路径。

开始前：串起 frame、已提供 body codec 与日志的输入/输出边界。

背景与推理

broker 是请求语义边界：它把收到的 Request 分派到 metadata、produce 或 fetch，检查分区、参数和 epoch 后调用对应 backend，再返回 success body 或带错误码的 ErrorBody。produce 把 records 追加到一个分区，fetch 按 offset/预算主动读取记录；读取不会删除日志。单机模式没有 follower，故本步 high watermark (HW，可见提交边界) 与 LEO 相等。测试真实 TCP 同时经过客户端 framing、provided body codec、server handler 与返回 framing。

起点与编辑位置。

已给：BrokerHarness.single、TCP server/client、MessageCodec 的请求/响应体、请求类型/回复类型、日志注册与服务生命周期已提供；SingleBrokerMain 用 loopback 与 OS 分配端口。

请修改：实现 exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java 的 public Messages.Reply handle(Messages.Request request)。本步只实现 METADATA、PRODUCE、FETCH 分派，不重写 transport/body codec。

开始前先读

完成第 11 步 (TCP 帧协议)，并具备第 9 步 (分区路由)、第 10 步 (主题元数据) 所用的 topic/partition 概念。本步 fetch 使用单机 HW=LEO；多副本时 HW 的意义会在复制章重新讲解，不把单机规则外推。

输入、输出与状态

- metadata 输入 `MetadataRequest(topic)`，成功输出 `MetadataBody(List<PartitionMetadata>)`；null 或格式非法的 topic 返回 `INVALID_REQUEST`，未知的合法 topic 返回 `UNKNOWN_TOPIC_OR_PARTITION`。它只读 catalog，不改日志。
- produce 输入 `ProduceRequest(tp, epoch, acks, timeoutMillis, records)`；单机只接受 epoch=0, acks 必须是已提供的 `Acks.LEADER(wire=1)` 或 `Acks.ALL(wire=-1)`, timeoutMillis 非负, records 非空。单机 backend 忽略 acks 模式，非负 timeout 不触发等待；副本确认含义见第 23 步 (生产确认)。检查顺序是解析 partition、检查 produce 参数、取得 backend、检查 epoch/leader，最后调用 backend produce。未知 partition 返回 `UNKNOWN_TOPIC_OR_PARTITION`；参数错误为 `INVALID_REQUEST`；不匹配 epoch 为 `FENCED_EPOCH`。成功追加并返回 `ProduceBody(AppendResult)`。前置校验拒绝时不追加；底层 I/O 在写入开始后失败时，不保证没有物理副作用。
- fetch 输入 `FetchRequest(tp, epoch, offset, maxRecords, maxBytes, token)`；本步 token=null，不实现之后的消费组围栏。检查顺序是解析 partition、检查 maxRecords/maxBytes 为正、取得 backend、检查 epoch/leader、再读取 backend。单机接受 epoch=0；越出已保留区间 `[logStartOffset, LEO]` 的 offset 返回 `OFFSET_OUT_OF_RANGE`，已知请求校验失败不改日志。成功 `FetchBody(records, logStartOffset, logEndOffset, highWatermark, epoch)`，records 不超过任一预算且不拆记录；本步 HW=LEO，offset=LEO 是空成功读取。fetch 不修改日志。
- handler 返回 `Reply`：成功时 error=NONE 并携带对应 success body；失败时带原课程错误码及安全 `ErrorBody`。null 或不支持的 request 报 `INVALID_REQUEST`。`CourseException` 保留错误码；参数类 `IllegalArgumentException/NullPointerException` 映射 `INVALID_REQUEST`，意外 runtime 映射不泄露原因的 `STORAGE_ERROR`。`ExerciseNotImplementedException` 透传供课程 runner 定位未完成方法。transport 层 (已提供) 处理非法 frame、request frame 非零 error、body 格式/尾随字节与回复 frame 编码；handler 不重复实现这些解析。

行为契约与不变量。unknown partition、stale epoch 与位点越界必须保留各自错误码；已知在访问 backend 前拒绝的请求不能追加。单机 metadata epoch=0；成功 fetch 不超过 LEO=HW。错误只影响当前请求，不能把 Java 堆栈或私有异常消息送上线，后续合法 TCP 请求仍可处理。provided `MessageCodec` 会把 fetch maxBytes 夹到单个 response frame 可容纳的上限 8,388,566 磁盘字节；一条超大完整记录不拆分，剩余记录留给下一次从后续 offset 拉取。

手算 produce、fetch 与大响应分页

假定 orders-0 已创建、epoch=0、LEO=0，并用 `acks=Acks.LEADER`、`timeoutMillis=0` 调用 produce，按序写三条：key *k* (1 byte) /value *v*0 (2 bytes)、key *k*/value *v*1、null key/value *v*2。普通记录总长分别是 35、35、34 bytes，成功返回 [0,3)，新 LEO=3。fetch offset=0、maxRecords=2、maxBytes=4096 返回 offsets 0、1，FetchBody 中 start=0、LEO=3、HW=3、epoch=0；fetch offset=3 返回空。若改用 maxBytes=69（比两条 35-byte 记录总和少 1），只返回 offset 0。

再假定 9 条记录各自使用 null key 与 1,048,548-byte value；每条普通磁盘记录大小 1,048,580。请求 maxBytes=10,000,000 会被 provided codec 限到 8,388,566：7 条共 7,340,060 bytes，可返回 offsets 0-6；8 条共 8,388,640 超限，因此第一次响应不能包含 offset 7，下一次以 offset 7 请求返回 7、8。记录不得拆开以凑预算。

可观测例子与测试点。 `Step12Test: servesPreciseMetadataRecordsAndFailuresWithoutMutatingTheLog` 检查 metadata、produce/fetch 和错误不变更日志；`boundsLargeFetchRepliesAndPreservesRecordContinuation` 与 `returnsAnErrorReplyWhenAHandlerProducesAnOversizedFetchBody` 检查大响应预算；`sanitizesHandlerRuntimeExceptionsAndKeepsBothConnectionsUsable` 检查错误消毒与服务续用；`isolatesExistingTopicPartitionsAndServesFetchesFromNonzeroLogStart` 检查分区/起点隔离；`rawTcpRequestsAndMalformedBodiesStayIsolatedToTheirConnections` 检查真实 TCP 失败隔离。

常见误区与真实 Kafka 的差异

直接调用 handler 看不到 TCP 边界、`correlationId` 或 reply codec 错误；把异常吞成空成功会让调用者误以为追加/读取已成功。一次业务错误不得让 broker 停止；但成功 produce 与客户端收到回复不是同一个网络动作，回复可能在网络失败时不可达。

分级提示（不是答案）

- 一级 先把 METADATA、PRODUCE、FETCH 各自的输入类型、前置检查与成功 body 列成三条路径。
- 二级 错误发生在 frame/MessageCodec、handler 校验还是 backend，要保留对应层责任与错误码。
- 三级 用真实 loopback TCP 先 metadata、produce、fetch，再发一个已知失败请求并继续 fetch，比较日志与回复。

命令与晋级。 `./gradlew :stepTest -Pstep=12` 用于定位本步；`./gradlew :test -Pstep=12` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么 error Reply 不能把 Java 堆栈或内部异常消息发给客户端？
2. 单机模式中 HW=LEO，为何不能直接把这条规则当作副本模式的语义？

自检参考解释 内部异常可能泄露实现细节且不是稳定协议契约；对外只返回课程错误码和简短安全 message。单机没有复制进度差异；复制时 LEO 只表示本地尾端，HW 另行决定可读范围。

第五章 生产消费、位点与重复投递

本章先区分四种容易混淆的进度。日志末端 LEO 是某分区下一条可追加记录的 offset；复制时，HW（高水位）是下一条 offset 边界，只有小于 HW 的记录处于复制可见范围。在目前的单 broker 路径中 HW 与 LEO 相同，复制章节才会出现两者不同。consumer 的 position 是客户端内存中“下次从哪里拉”的 offset；committed next offset 是按 group 与分区保存、重启后可恢复的进度。poll 只推进内存 position，commit 才写入进度日志；二者都不删除消息。producer receipt 是 broker 对某次追加返回的分区及半开 offset 区间 [firstOffset,nextOffset)，不等于消费提交。本章的直接 RpcClient 由客户端接管并在 close 时关闭；传入的共享 MetadataRouter 则由创建者负责关闭。

5.1 第 13 步：批量生产者

目标与前置知识

本步目标：实现按分区有序的批量缓冲、达到阈值时同步发送，并在显式 flush 时交还完整 receipt。

开始前：理解 producer、分区路由、batch、确认与请求结果未知分别是什么。

背景与推理

producer 是把记录写入 broker 的客户端；每条记录先按 topic（记录分类）和 partition（各自有序的日志）定位。batch 是暂存在客户端、准备一次发送的一组记录；按分区分别缓冲，所以一个分区达到阈值时不会顺便发送其他分区的记录。receipt 是 broker 回复的实际 offset 区间；ACK 是 producer 请求的确认模式。本步的单机 backend 忽略 ACK；非负的 ProduceRequest.timeoutMillis 也不触发等待。producer 同步等待 broker 回复，实际 TCP 等待上限由单独配置的 RpcClient 控制；这不表示多副本确认，副本 ACK 含义见 [第 23 步（生产确认）](#)。收到错误或 timeout 并不能证明没有追加：请求可能已到 broker 并落盘，只是回复丢失。课程的普通 SimpleProducer 不自动重试，也不使用后台定时器。

开始前先读

先理解 [第 3 步（分区与分区内 offset）](#)，再复习 [第 9 步（key 如何选择分区）](#)、[第 10 步（metadata 如何列出分区）](#)及 [第 12 步（broker 如何处理 PRODUCE）](#)。特别注意 Partitioner.choose 是第 9

步学生已完成的代码,并非本步提供的答案;本步当前使用直接 `RpcClient` 路径, `MetadataRouter` 客户端路由留到第 27 步。

起点与编辑位置。

已给: `RpcClient`、消息/记录值类型和 broker metadata/produce 协议已提供;复用第 9 步实现的 `Partitioner`。

请修改: `exercises/src/main/java/io/simplekafka/client/SimpleProducer.java`:
`void send(String topic, byte[] key, byte[] value, long timestamp)` 与 `List<ProduceReceipt> flush()`。只实现这两个练习方法,不改 `Partitioner`、协议或测试。

参数、结果与状态

- 构造参数 `batchRecords` 必须大于 0, 否则 `INVALID_REQUEST`; 传入的 `client/partitioner` 为 `null` 抛 `NullPointerException`。默认 `ACK` 为 `LEADER`, 请求字段 `timeoutMillis=5,000 ms`。已有 `setAcks(Acks, long)` 修改后续请求的 `ACK/timeout` 字段, `timeout` 必须非负, 且只能在没有待发 `batch` 时更改; `null acks` 抛 `NullPointerException`。
- `send` 的 `topic` 必须是课程有效标识; `key` 可为 `null`; `value` 必须非 `null`; `timestamp` 是非负的 `long`, 课程不要求把它解释成真实墙钟。成功调用不返回 `receipt`, 而是把记录加入对应 `TopicPartition` 的 `FIFO batch`; 达到阈值即同步发送该分区。
- `flush()` 发送所有非空分区 `batch`, 返回自上次 `flush` 以来积累的不可变 `receipt` 列表, 并清空这份 `receipt` 累积; 没有待发记录时不发空请求, 返回空列表。
- 发送前的 metadata/参数错误没有 broker 追加。进入 `produce RPC/回复路径` 后的错误可能已有追加; `producer` 保存首次错误并进入结果未知状态, 后续 `send`、`flush`、`setAcks` 均报 `REQUEST_TIMEOUT`, 只能关闭后换新实例。

行为契约与不变量。相同分区按 `send` 顺序追加; 不同分区各自成 `batch`。自动发送的 `receipt` 也必须留到下一次显式 `flush`, `receipt` 使用左闭右开区间。`null key` 由分区器的实例级轮转处理, 非 `null key` 走第 9 步路由。

必须满足的边界

- `null value`、负 `timestamp`、无效 `topic` 在进入 `produce` 之前报 `INVALID_REQUEST`; 未知 `topic` 报 `UNKNOWN_TOPIC_OR_PARTITION`。这些检查不会追加, 也不会让 `producer` 进入结果未知状态。
- 超长普通记录 (总长超过 1,048,580 bytes; `null key` 时 `value` 上限为 1,048,548 bytes) 报 `INVALID_REQUEST`, 且 broker 未收到 `produce frame`、没有追加。若 `batch` 尚未达到阈值, `send` 可能先正常返回; 本地编码器要到自动发送或 `flush()` 才发现超长。此时 `producer` 会把这次 `produce` 路径异常记为结果未知, 后续操作报 `REQUEST_TIMEOUT`。这是课程客户端的保守状态行为, 不代表 broker 可能已经追加, 也不得重发该 `batch`。
- 首次发送 `batch` 的同步路径发生任何异常时, 当前调用传播原错误码并锁存 `producer` 的 `unknown-outcome` 状态; 之后 `send`、`flush`、`setAcks` 均报 `REQUEST_TIMEOUT`, 不得重发。实际追加是否发生取决于异常位置: 网络/回复丢失时可能已经追加; 本地编码错误能确定未发送, 但本课程实现仍会锁存该状态。

- 正在缓冲记录时调用 `setAcks` 报 `INVALID_REQUEST`；负 `timeout` 也报 `INVALID_REQUEST`。关闭后调用 `producer` 操作报 `IllegalStateException`。

可观测例子与测试点。手算：broker 的 `orders-0` 初始 `LEO=0`，构造 `batchRecords=2` 的 `producer`。`send` 的两条同 `key` 记录依次缓冲，第二条触发发送，broker 追加 `offset 0、1`，区间为 `[0,2)`；第三条缓冲后尚不可读。下一次 `flush()` 发送第三条并返回两张 `receipt`：`[0,2)` 与 `[2,3)`；再 `flush` 返回空。前提是 `topic` 只有该目标分区或 `key` 已知路由到它。

`Step13Test.flushesAtTheBatchThresholdAndReturnsAutomaticAndExplicitRanges` 检查阈值与只交还一次的 `receipt`；`automaticFlushForOnePartitionLeavesAnotherPartitionsPendingBatchInvisible` 检查分区 `batch` 独立；`rejectsInvalidSendsWithoutPersistingAnyRecords` 和 `doesNotRetryAProduceWhoseAppendSucceededBeforeItsResponseTimedOut` 检查参数拒绝及追加后丢回复不重试。

分级提示（不是答案）

- 一级** 先分清请求发出、broker 追加、客户端收到确认和调用者取得 `receipt` 的时刻；它们可能不是同一时刻。
- 二级** 用 `TopicPartition` 作 `batch` 边界，分别手算同分区达到阈值和不同分区尚未达到阈值。
- 三级** 用测试中的“追加成功后延迟/丢弃回复”情形检查：客户端是否保留首次错误，并且是否把旧 `batch` 再发一次。

自检问题

- 为什么 `timeout` 不能证明记录没有写入？
- 若自动发送已成功，为什么 `receipt` 还要由下一次 `flush` 返回？

自检解释。 请求回复可能在追加之后丢失，所以客户端无法判断结果；`receipt` 是调用者获知具体 `offset` 区间的依据，丢掉它会使已追加记录变得不可追踪。

常见误区与真实 Kafka 的差异

把“收到错误”理解成“肯定未写入”并重试会产生重复。真实 Kafka 还有异步发送与更多确认选项；本课程的 `SimpleProducer` 是同步、显式 `flush`、无自动重试的简化客户端。

命令与晋级。 `./gradlew :stepTest -Pstep=13` 用于定位本步；`./gradlew :test -Pstep=13` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

5.2 第 14 步：拉取消费者

目标与前置知识

本步目标：构建单线程、单轮 `pull consumer`，显式管理分配分区和内存 `position`。

开始前：理解“下一条 `offset`”、`seek` 重放、空 `fetch` 与跨分区共享预算。

背景与推理

consumer 是主动向 broker 拉取记录的客户端；读取不会从日志删除数据。position 是某分区下次 fetch 的 offset，不是已经持久化的消费进度。手动 assign 建立客户端负责读取的分区集合；seek 只改变本地读取位置。poll 发起一轮有上限的 fetch，而不是持续等待。

开始前先读

复习 第 3 步（分区和局部 offset）、第 12 步（broker 的 FETCH 请求）。第 13 步只是课程顺序上的前一步，本步不调用 producer，也不读取 OffsetStore；本步先从 offset 0 建立纯内存 position，第 15 步才加入恢复与提交。

起点与编辑位置。

已给：broker 的 FETCH 路由、RpcClient、LogRecord 与 TopicPartition 类型已提供。

请修改：exercises/src/main/java/io/simplekafka/client/SimpleConsumer.java: void assign(Collection<TopicPartition> partitions)、void seek(TopicPartition tp, long offset)、Map<TopicPartition, List<LogRecord>> poll(int maxRecords, int maxBytes)。
只实现三者；只读 position(TopicPartition) 已提供。

参数、结果与状态

- assign 用新集合替换旧 assignment：去重并按 topic/partition 排序；保留分区沿用 position，撤销分区移除，新分区从 0 开始。null 集合抛 NullPointerException，含 null 元素报 INVALID_REQUEST，拒绝时旧 assignment 不变。
- seek(tp, offset) 只接受已分配 tp 与非负 next offset；null tp 抛 NullPointerException，未分配报 NOT_ASSIGNED，负数报 OFFSET_OUT_OF_RANGE，拒绝时 position 不变。seek 不提交进度。
- poll 的 maxRecords 与 maxBytes 都必须大于 0，否则 INVALID_REQUEST 且不推进 position。二者是所有已分配分区共享的总预算；扫描顺序为 topic、partition 升序。空 assignment 不发 fetch，空分区不消耗预算也不出现在返回 map 中；成功返回的 map 不可变。consumer 关闭后调用这些操作报 IllegalStateException，未分配 partition 的 position(tp) 报 NOT_ASSIGNED。
- 本步普通记录计费为 32+keyBytes+valueBytes；不能拆分记录。每分区成功回复须从请求 position 起连续且不超剩余记录/字节预算，否则该分区报 INVALID_REQUEST 且不前移。若 seek 到已删除的起点之前或当前 LEO 之后，broker 报 OFFSET_OUT_OF_RANGE，失败分区不推进。服务端其他错误原码传播；先前已成功分区的 position 可保留推进，失败分区保持原值。

行为契约与不变量。成功返回记录的分区 position 推进到最后一条 offset 加一；未返回的分区、空 fetch 与失败分区不推进。poll 不自动 commit，也不删除日志。过大的正 maxBytes 会由 broker 限制到保守单帧预算；下一轮仍从首个未返回 offset 继续。

可观测例子与测试点。假设 orders-0、orders-1 各有两条普通记录，每条 key=null、value 为空、因此计费 32 bytes；二者 position 初始均为 0。assign 两分区后 poll(4,64) 按排序先读

orders-0 的 offset 0、1，返回只含该分区的 map，position 变为 (2,0)，预算用尽。把两者 seek 回 0 后 poll(4,63)，第一分区只能返回 offset 0，剩余 31 bytes 放不下下一条，最终 position 为 (1,0)。若之后的脚本化测试 broker 对第二分区返回 UNKNOWN_TOPIC_OR_PARTITION（模拟该请求失败），第一次 position 可为 1、失败分区仍为 0。

Step14Test.pollsInPartitionOrderWithinRoundBudgetsAndPreservesManualPositions 检查排序、预算、seek 与 assign；pollContinuesPastEmptyPartitionsWithoutSpendingTheSharedBudget 检查空分区；leavesFailedPartitionPositionUnchangedForFetchErrorsAndMalformedSuccessReplies 检查失败、断续 offset 和超预算响应。

分级提示（不是答案）

- 一级** 分别记录 assignment、每分区内存 position 和持久 committed offset；本步只改前两者。
- 二级** 每返回一条记录就从同一个 poll 剩余预算扣除完整编码字节与一条记录，不要给每个分区重置预算。
- 三级** 先让一个较早排序的分区成功、再让后一个分区失败，逐项检查两个 position，而不要假设整轮原子回滚。

自检问题

1. poll 读到 offset 0、1 并把 position 设为 2 后，重建 consumer 为什么仍可能从 0 读？
2. 为什么后一个分区 fetch 失败时不能把它的 position 也推进？

自检解释。 position 只存在旧客户端内存中，重建后若没有 resume/commit 就从新 assignment 的默认 0 开始；失败分区没有被成功读取，推进它会跳过数据，而先前成功分区的读取进度可单独保留。

常见误区与真实 Kafka 的差异

读取成功不等于持久提交；seek 到日志末端得到空结果，不是让 position 自动增加的理由。真实 Kafka 有持续 poll、心跳及更复杂的分配生命周期，本步只实现单线程、有限轮次。

命令与晋级。 ./gradlew :stepTest -Pstep=14 用于定位本步；./gradlew :test -Pstep=14 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

5.3 第 15 步：消费位点持久化

目标与前置知识

本步目标：把各 group/分区的下一条 offset 写入本地持久日志，并由新 consumer 恢复。

开始前：区分消费位点、OffsetStore 自己的物理日志 offset，以及 group key。

背景与推理

consumer position 是客户端内存的读取游标；commit 把“下一条要读取的 offset”保存为消费位点。OffsetStore 自己也用追加日志，但那条日志的物理 offset 只编号存储记录，payload 中的 nextOffset 才是消费进度。键为 (group, TopicPartition)，不同 group 或分区互不覆盖；读取和提交都不删除业务记录，也不改变 HW。

开始前先读

需要理解 第 1 步（日志记录与恢复）、第 12 步（broker 请求路由）和第 14 步（position、assign、poll）。手动 consumer 用 null group token；本步还没有第 20 步的成员围栏。

起点与编辑位置。

已给：Offset entry 的 MessageCodec 编解码、底层 PartitionLog、记录恢复和 COMMIT_OFFSET/FETCH_OFFSET 请求类型已提供。

请修改：exercises/src/main/java/io/simplekafka/group/OffsetStore.java: void commit(OffsetKey key, long nextOffset)、OptionalLong fetch(OffsetKey key); exercises/src/main/java/io/simplekafka/client/SimpleConsumer.java: void resume(Collection<TopicPartition> partitions)、void commitSync(); 并在 exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java 的 handle(Messages.Request) 增补 COMMIT_OFFSET/FETCH_OFFSET 分支。

参数、结果与状态

- OffsetKey 标识 group 与一个 topic-partition；commit 的 nextOffset 必须非负，允许显式回退。commit 的 null key、负数，以及 fetch 的 null key 均报 INVALID_REQUEST；已关闭 store 或存储故障为 STORAGE_ERROR。fetch 返回最新 OptionalLong；没有提交是 empty，不是 offset 0 的已提交记录。
- 每次 commit 以 null record key、timestamp=0、编码后的 offset entry 追加并 force；重开时扫描同一 key 的最后一条。物理日志 offset 与 payload 的 nextOffset 没有相等要求。完整损坏记录报 CORRUPT_RECORD 且不改文件；合法但不完整的尾记录可按底层恢复规则截断。
- resume 对分区去重、排序并替换 assignment；每个 position 设成已提交值，缺失时为 0，不把已提交值夹到当前 LEO。位点查询失败会传播，但 assignment 已替换且不会回滚：新分区仍为 0，保留分区保留原 position。commitSync 为每个已分配分区各发一次提交，包括 position=0；空 assignment 不发送提交请求。poll 本身不提交，多个分区提交不是一个原子事务。
- 协议成功响应：CommitOffsetRequest(OffsetKey, nextOffset, null) 得 EmptyBody；FetchOffsetRequest(OffsetKey) 得带 OptionalLong 的 OffsetBody。负提交值经 broker 也报 INVALID_REQUEST。

行为契约与不变量。最后一次追加的同 key 值为当前提交，允许回退；group 和 topic-partition 任一不同都隔离。resume 只恢复持久消费进度，不修复越界提交，也不会自动把 consumer position 写回 store。

可观测例子与测试点。初始 store 无 workers/orders-0 值。commit(3) 后关闭并重开，fetch 得 3；再 commit(1) 并重开，fetch 得 1，展示 latest-wins 且可回退。另一 group 的相同分区仍为空。新 consumer.resume 后 position=1；若提交值设成 99，即使分区 LEO=3，resume 仍为 99，随后 poll 才报 OFFSET_OUT_OF_RANGE 且不改该值。

Step15Test.offsetStorePersistsNextOffsetsPerGroupAndPartitionAndAllowsRewind 检查隔离、回退和重开；rawOffsetRequestsReturnExactBodiesAndIsolateGroupTopicAndPartitionKeys 检查协议体；resumeCommitsAllPositionsAndDoesNotClampPastLogEnd 检查 resume；offsetStoreTruncatesAnIncompleteTailAndRejectsCompleteBadCrcWith outMutation 检查恢复和完整损坏。

分级提示（不是答案）

- 一级 先把消费位点看成“下一条业务记录的编号”，不要用 OffsetStore 日志的 offset 代替它。
- 二级 同一 OffsetKey 可以有多条历史提交；以追加顺序最后一条为准，不按 map 顺序猜测。
- 三级 分开验证两件事：store 关闭重开后值是否仍在，以及全新 consumer.resume 后 position 是否从该值开始。

自检问题

1. 处理完 offset 2 后为什么提交 nextOffset=3，而不是 2？
2. 如果 nextOffset=99 而 LEO=3，resume 是否应自动改成 3？

自检解释。 nextOffset 表示下次要处理的位置，所以 3 才跳过已处理的 0-2；store 只忠实恢复提交值，越界由后续 fetch 明确报错，自动夹取会悄悄改写调用者的进度。

常见误区与真实 Kafka 的差异

提交 last processed offset 会让它在恢复后重读；把 committed offset 与复制可见边界混为一谈也不对。真实 Kafka 把 offset 放在复制/压缩的内部主题；本课程是本地单 coordinator 追加日志，不提供高可用。

命令与晋级。 ./gradlew :stepTest -Pstep=15 用于定位本步；./gradlew :test -Pstep=15 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

5.4 第 16 步：至少一次处理

目标与前置知识

本步目标：安全安排 poll、业务副作用和 offset commit 的先后，并能手算故障后的重放。

开始前：理解至少一次、内存 position 与持久提交的区别，以及跨请求非原子边界。

背景与推理

业务副作用是处理记录时对外部状态的改变，例如写本地账本或发送通知。课程的 `ProcessingLoop` 一次调用先 `poll`，再按 `TopicPartition` 升序、同分区 `offset` 升序调用 `processor`；所有回调成功后才调用一次 `commitSync`。副作用与 `offset` 日志不是同一事务：若处理已成功而提交失败，恢复后必须重读，可能再次产生副作用。至少一次的含义是宁可重复已成功工作，也不把未成功处理的记录静默跳过。

开始前先读

需要 第 14 步 (`poll` 与 `position`)、第 15 步 (持久 `next offset` 与 `commitSync`)。本步提供回调接口，不要求外部副作用与 `offset store` 共享事务。

起点与编辑位置。

已给：`RecordProcessor.process(TopicPartition, LogRecord)` 回调接口与单轮 `poll/commit` 生命周期已提供。

请修改：`exercises/src/main/java/io/simplekafka/client/ProcessingLoop.java`：`intrunOnce(intmaxRecords, intmaxBytes) throws Exception`。只实现单轮控制；不改 `consumer`、`store` 或回调接口。

参数、结果与状态

- `maxRecords`、`maxBytes` 是本轮 `poll` 跨分区的正总预算；非正值由 `poll` 报 `INVALID_REQUEST`。成功返回本轮回调并提交的记录数，空 `poll` 仍调用 `commitSync` 并返回 0。
- `callback` 输入为分区和一条 `LogRecord`，不返回值，可抛任意 `Exception`。回调异常原样传播，本轮不提交任何分区；`poll` 可能已经推进 `consumer` 的内存 `position`。
- `poll`、`callback` 或 `commit` 出现异常后，此 `ProcessingLoop` 变为 `failed`；再次 `runOnce` 报 `IllegalStateException`。关闭旧 `consumer`，创建新 `consumer` 和 `loop`，并 `resume` 已持久提交位置。副作用已经写出但 `offset` 尚未提交时，副作用可能重做。
- 所有回调成功后 `commitSync` 对每个已分配分区分别提交；不是跨分区事务，某个 `commit` 失败时较早提交可能已成功。失败原异常/协议错误向调用者传播，方法不返回成功计数。

行为契约与不变量。回调只按稳定分区、`offset` 顺序执行；只有所有被本轮 `poll` 返回的记录都处理成功，才开始提交。一个 `callback` 失败时不提交任一分区。任何本轮失败后不得复用 `loop` 或把已推进的内存 `position` 当作 durable progress。

可观测例子与测试点。初始 `orders-0`、`orders-1` 各有 `offset 0、1`，`group workers` 没有提交，`poll(4,8192)` 返回四条。回调先产生 effects `p0:0, p0:1`，再在 `p1:0` 抛错；`p0/p1` 的 `committed offset` 都仍缺失，尽管旧 `consumer` 内存 `position` 已到 2。重建并 `resume` 后从 0/0 重读四条，之前 `p0` 两个副作用会再次发生。另一种故障是所有回调完成后 `commit` 请求失败：副作用已发生而提交未完整确认，恢复仍可能重放；若有多个分区提交，部分 `key` 可能已写入。

`Step16Test.laterPartitionFailureLeavesBothOffsetsUncommittedAndReplaysEarlierSideEffects` 覆盖跨分区回调失败；`commitFailureAfterSideEffectsReplaysTheSameRecordsAtLeastOnce` 覆盖提交故障；`boundedRoundsProcessInTopicPartitionOffsetOrderA`

`ndCommitFinalPositions` 覆盖排序、预算、空轮；`failedPollProcessesNothingAndResumeRetriesThePreviouslyFetchedRecord` 覆盖 poll 失败。

分级提示（不是答案）

- 一级 把 poll、每次副作用、每次 offset 请求、崩溃/重建按时间顺序写出。
- 二级 先按排序规则列出本轮回调，再判断失败落在 callback 还是 commit；这两者对副作用的影响不同。
- 三级 观察 Step16Test 中每个失败后 store 的值、consumer 的 position 和 loop 是否还能调用；分别记录它们，不能用一个“进度”概括。

自检问题

1. 如果 p0 的回调成功、p1 的回调失败，为什么 p0 的成功副作用也会重放？
2. commitSync 对三个分区是不是全成或全败？

自检解释。 契约等所有回调完成才开始提交，因此 callback 失败时没有任何分区被提交，恢复从旧的 durable offsets 重新读取；commitSync 是多个请求，不是跨分区事务，故较早分区可能已提交。

常见误区与真实 Kafka 的差异

吞掉 callback 异常后继续 commit 会丢失失败记录；继续使用失败 loop 则可能跳过未提交数据。至少一次不等于 exactly-once；应用需让副作用可去重，第 31 步才实现本地持久业务去重。

命令与晋级。 `./gradlew :stepTest -Pstep=16` 用于定位本步；`./gradlew :test -Pstep=16` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

第六章 消费组、重平衡与提交围栏

消费组是一群协作读取同一 topic 的 consumer：成员是参与者标识，assignment 是当前每个成员拥有的分区列表，同组一个分区同时只分给一个成员。负责保存组状态并答复加入/查询的 coordinator 在成员变化时重算整个 assignment，这次变化叫 rebalance。generation 是每次拓扑变化递增的组版本，不是时钟或消费 offset；稍后的 GroupToken 会把组、成员和 generation 一起带入请求，使服务端能拒绝旧成员。心跳用于表示成员仍活跃，过期成员会被移除。本课程是同一 topic、即时重分配、单进程可信 coordinator 的教学模型，不是完整 Kafka 组协议或高可用共识。

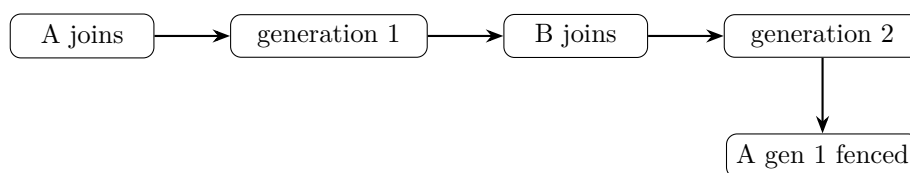


图 6.1: 成员拓扑变化递增 generation；旧 token 不能覆盖当前提交。

6.1 第 17 步：分区分配

目标与前置知识

本步目标：实现确定、平衡且每个分区只有一个 owner 的 Round Robin 分配。

开始前：理解去重、稳定排序、不可变结果和商余数分配。

背景与推理

分区可以在 consumer 间并行处理，但若同一分区被两名成员同时处理，进度会冲突。Round Robin 在排序后的成员间循环派发排序后的分区；结果可重现，每个成员分到的分区数最多相差一。这里的 assignor 是纯计算函数，尚不负责判断成员是否真的加入了 group。

开始前先读

先复习 [第 3 步 \(TopicPartition 与分区排序\)](#)。本步的分配器会被 [第 18 步 \(成员 coordinator\)](#) 调用；它只根据传入的 partition/member 列表分配，不查询目录、不改日志。

起点与编辑位置。

已给: `TopicPartition` 值类型、`group` 返回值类型已提供; `RoundRobinAssignor` 构造器无状态, 不提供分配算法答案。

请修改: `exercises/src/main/java/io/simplekafka/group/RoundRobinAssignor.java`:
`Map<String,List<TopicPartition>>assign(List<TopicPartition>partitions,List<String>members)`。只实现 `assign`。

参数、结果与状态

- `partitions` 是待分配的 `TopicPartition` 列表, `members` 是成员标识列表; 两列表及元素均不可为 `null`。相同 `partition/member` 只保留一份。
- `partition` 按 `topic` 后 `partition number` 升序, `member` 按 `Java String` 自然顺序排序; 不是输入顺序, 也不是数字字符串的数值顺序。
- 返回 `map` 的每个不同 `member` 都有列表, 包括空列表; `map` 和内部列表均不可变。assignor 不改变输入列表, 也没有跨调用状态。
- `null` 输入/元素, 或 `member` 标识 UTF-8 编码长度不在 1-255 bytes, 报 `INVALID_REQUEST`。无 `member` 时返回空 `map`; 有 `member` 但无 `partition` 时保留所有成员并给空列表。

行为契约与不变量。至少有一个不同 `member` 时, 每个不同 `partition` 恰好出现一次且 `owner` 属于 `members`; 所有不同 `member` 都出现在结果中。成员分区数量最大值与最小值之差不超过 1。无 `member` 时遵循上项, 返回空 `map`, 不产生 `owner`。

可观测例子与测试点。初始 `partitions` 明确为 `[orders-0,orders-1,orders-2,orders-3,orders-4]`, `members` 输入顺序为 `[b,a,a]`。排序去重后 `members=[a,b]`, 分区 0、2、4 给 `a`, 1、3 给 `b`; 改乱输入顺序结果不变。若有 5 个分区和按 `Java` 字符串顺序排列的 `[a,b,c,d,e,f,g]`, 前五人各得一个、`f/g` 保留空列表。

`Step17Test.sortsAndDeduplicatesPartitionsAndMembersBeforeRoundRobinAssignment` 检查去重与确定顺序; `assignsFivePartitionsToTheFirstFiveOfSevenSortedMembersAndKeepsEmptyMembers` 检查多成员空列表; `sortsCrossTopicPartitionsAndMemberIdsLexicallyAfterDeduplication` 检查跨 `topic` 与 "10" 排在 "2" 前的词典顺序。

分级提示 (不是答案)

- 一级 把“唯一 `owner`”拆成完整覆盖和没有重复两项分别检查。
- 二级 先排好成员与分区, 再按索引取模分配; 输入 `List` 的顺序不是本课程规则。
- 三级 分别手算 5 分区/2 成员与 5 分区/7 成员, 并把重复输入加入测试, 确认它不会让一个分区出现两次。

自检问题

1. 为什么稳定排序是可观察契约, 而不只是输出美观?
2. 成员多于分区时, 为什么仍要返回空列表成员?

自检解释。 稳定排序让相同集合每次产生相同 `owner`, 便于协调与验证; 空列表明确表达成员仍在组中但当前无分区。

常见误区与真实 Kafka 的差异

只返回有分区成员会丢失空 assignment；依赖偶然 map 遍历顺序会破坏可重现性。真实 Kafka 有多种 assignor 和协作式重平衡，本课程只做单 topic 的简单 Round Robin，不声称它是 Kafka 默认算法。

命令与晋级。 `./gradlew :stepTest -Pstep=17` 用于定位本步；`./gradlew :test -Pstep=17` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

6.2 第 18 步：成员与重平衡

目标与前置知识

本步目标：管理 group 成员、generation 与全组分配，在成员拓扑变化时正确重平衡。

开始前：理解 join、leave、generation 与 committed offset 各自代表什么。

背景与推理

成员是以 member id 标识的 consumer 实例；一个 group 在本课程中订阅一个 topic。coordinator 保存成员集合及它们共享的 generation，并用第 17 步 assignor 计算完整分配。新成员加入或成员离开改变拓扑，generation 加一并立即重算所有成员的分区。重复同一 member/topic 的 join 只刷新心跳，不改变拓扑；generation 不是时间戳，也不保存消费进度。

开始前先读

先掌握 [第 10 步（topic metadata 与分区目录）](#)，再看 [第 17 步（确定分配）](#)。需要知道一个 topic 对应哪些 TopicPartition，并区分 coordinator 的内存成员状态与第 15 步的持久消费位点。

起点与编辑位置。

已给：PartitionCatalog、组锁、成员状态容器、TimeSource 与第 17 步 assignor 已提供；消息请求值类型已编码。

请修改：exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java：GroupAssignmentJoin(Stringgroup,Stringmember,Stringtopic)、GroupAssignmentLeave(Stringgroup,Stringmember)、GroupAssignmentAssignment(Stringgroup,Stringmember)；并在 exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java 的 handle(Messages.Request) 增补 JOIN_GROUP/LEAVE_GROUP/GROUP_ASSIGNMENT。

参数、结果与状态

- group/member 必须是非空且不超过 255 UTF-8 bytes 的标识；topic 必须是有效课程 topic。首次 join 只在 topic 已存在后创建组，初始 generation=0，成功加入后为 1。

- 新成员加入或已知成员成功离开, generation 恰加一, 返回包含全体当前成员的 `GroupAssignment`。同 member、同 topic 重复 join 仅更新 lastHeartbeat 并返回现有 assignment; 不得重复分配或增加 generation。
- 已存在 group 的 topic 冲突优先报 `INVALID_REQUEST`; 未知 topic 报 `UNKNOWN_TOPIC_OR_PARTITION`, 且不创建 group/消耗 generation。标识无效报 `INVALID_REQUEST`。未知 group/member 的 leave 或 assignment 报 `UNKNOWN_MEMBER`; coordinator 关闭后操作报 `IllegalStateException`。
- 组变空时保留 coordinator 内的 generation 历史和已提交 offsets; offset store 与成员生命周期分离。成员集合本身是进程内状态, 不因 offsets 持久而变成持久组协议。

行为契约与不变量。每次真实拓扑变化只增加一次 generation, 并为全部剩余成员重新计算分配; 重复 join 不改变 generation。assignment 查询只对当前已知成员成功, 且返回同一份全组映射。

可观测例子与测试点。初始 workers 组不存在, orders 有分区 0、1、2。a join 得 generation 1、拥有全部三分区; a 同 topic 再 join 仍为 generation 1。b join 后 generation 2, a 得 0、2, b 得 1; c join 后 generation 3, 三人各得一个。c leave 后 generation 4, 重算为 a 得 0、2、b 得 1。若随后组变空, generation 仍保留, 原有 group offsets 仍可 fetch。

`Step18Test.membershipChangesRebalanceEverySurvivorAndRetainEmptyGroupGenerationOverTcp` 通过真实 TCP 检查加入、重复加入、全组重算、topic/unknown-member 错误、空组进度保留及再次加入。

分级提示 (不是答案)

- 一级** 先判断请求是否真的改变成员集合; 重复 join、被拒绝的 join 与新成员加入不是同一状态变化。
- 二级** 每次拓扑变化后检查所有成员都看到相同 generation 和完整 assignment, 不能只更新新成员。
- 三级** 从零列出 a 加入、a 重复、b 加入、c 加入、c 离开的 generation 和每个 owner。

自检问题

1. 为什么 b 加入时 a 也必须获得新 generation 的 assignment?
2. 最后一个成员离开后, 为什么 group offset 不该删除?

自检解释。 分区可能已从 a 转交给 b, 若 a 保留旧 assignment 就无法对齐服务端当前所有权; consumer 进度属于 group/topic-partition, 不属于仍在线的 member, 所以空组也可保留进度。

常见误区与真实 Kafka 的差异

给每个成员独立递增版本会使 token 不一致; 组空不等于消费进度失效。真实 Kafka 有加入、同步 assignment 与协作撤销等多阶段协议; 本课程即时重分配, 不模拟完整 classic barrier 或 KIP-848。

命令与晋级。 `./gradlew :stepTest -Pstep=18` 用于定位本步；`./gradlew :test -Pstep=18` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

6.3 第 19 步：心跳与过期

目标与前置知识

本步目标：用可控时钟识别失联成员，并把同组到期成员合并为一次拓扑变化。

开始前：理解 `lastHeartbeat`、`session timeout` 和包含等号的边界。

背景与推理

成员进程可能崩溃而来不及 `leave`；`coordinator` 不能只凭沉默立刻移除它。成功 `heartbeat` 记录注入式时钟当前毫秒值，表示该成员仍活跃。若经过的时间达到 `session timeout`，成员过期。确定性时钟让边界测试不需依赖 `sleep`；提供的周期 `worker` 是运行时触发机制，和测试直接调用过期 `hook` 是两回事。

开始前先读

需要 [第 18 步（成员、generation 和 assignment）](#)。要能解释同一 `group` 的成员共享 `generation`，并知道被移除成员的旧 `token` 不再是有效心跳。

起点与编辑位置。

已给：`TimeSource`、组锁、正 `session timeout` 和周期调度生命周期已提供。

请修改：`exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java`：
`void heartbeat(GroupToken token)、Set<String> expire()`；并在 `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java` 的 `handle(Messages.Request)` 增补 `HEARTBEAT`。

参数、结果与状态

- `coordinator` 构造时 `sessionTimeoutMillis` 必须大于 0，否则 `INVALID_REQUEST`。`heartbeat` `token` 为 `null` 报 `INVALID_REQUEST`；先判断 `member` 是否存在，不存在报 `UNKNOWN_MEMBER`；之后才检查 `generation`，旧值或 `future` 值报 `ILLEGAL_GENERATION`。失败不刷新时间；有效 `token` 才把该 `member` 的 `lastHeartbeat` 更新到当前 `clock`。`coordinator` 关闭后操作报 `IllegalStateException`。
- `expire()` 不接收参数，使用注入的当前时钟，返回已移除成员的排序集合，元素格式为 `group/member`。当 `now-lastHeartbeat>=sessionTimeoutMillis` 时移除；未达到阈值不变，时间戳位于未来的成员也不会提前过期。

- 一次 expire 同一 group 中可同时移除多个到期成员，但该 group generation 只增加一次、只重算一次；其余未到期成员保留。返回空集合表示本轮没有移除成员；重复调用无新变化。过期不会删 committed offsets。
- broker 的周期 worker 会自动触发检查；确定性测试应移动 TimeSource 再调用提供的 expireGroups() 或 coordinator expire()，而不是用真实 sleep 猜时序。

行为契约与不变量。过期边界采用 $\geq$ ；无效/旧 heartbeat 不续期。过期结果按 group 再 member 升序排列。每个因到期而变化的 group 每轮 generation 仅增加一次；组空后仍保留 generation。

可观测例子与测试点。设 clock=1000、timeout=100，a 与 b 均在 1000 加入 workers，此时 generation=2，a 的 generation-1 heartbeat 已过期。clock=1099 时，a 用旧 token 或 future generation 99 心跳均报 ILLEGAL_GENERATION，不存在的成员报 UNKNOWN_MEMBER；这些失败不刷新。b 用 generation 2 成功 heartbeat，更新为 1099。1099 调 expire 返回空；clock=1100 再调，只移除 workers/a（恰好经过 100 ms），generation 变 3，b 获得全部三分区。

Step19Test.rejectsStaleAndFutureHeartbeatsAndExpiresOnlyTheUnrefreshedMemberAtTheBoundary 检查 1099/1100 与错误 heartbeat；expirationOrdersAllIdentifiersAndAdvancesEachChangedGroupOnce 检查排序及同组一次递增；idempotentJoinRefreshesHeartbeatButRejectedJoinDoesNot 检查续期；replicatedClusterExpiresIdleMembersWithoutIncomingRequests 与 expireGroupsProvidesDeterministicClusterExpiryAndChecksLifecycle 分别检查周期 worker 和确定性 hook。

分级提示（不是答案）

- 一级 在同一状态观察中使用当前时钟、判断 elapsed 时间并移除成员；失败 heartbeat 不应写 lastHeartbeat。
- 二级 返回值要能指出具体的 group/member，而不仅是过期数量。
- 三级 timeout=100、lastHeartbeat=1000 时，比较 1099 与 1100；再在 1099 刷新 B，确认本轮只移除 A。

自检问题

1. 为什么必须在契约中明确 timeout 的等号边界？
2. 同组 A、C 在一次 expire 中都到期，为什么 generation 不能加两次？

自检解释。若不规定等号，恰好到期时就会产生不同的成员所有权结果；一次 expire 是一次观察到的拓扑变化，所以同组一起移除只触发一次全组重分配。

常见误区与真实 Kafka 的差异

用系统时间和 sleep 测精确阈值会受调度影响；边界用注入 clock，worker 另做集成检查。先写 lastHeartbeat 再验证 token 会让过期成员非法续命。真实 Kafka 心跳还受配置、网络和协调协议影响。

命令与晋级。 `./gradlew :stepTest -Pstep=19` 用于定位本步；`./gradlew :test -Pstep=19` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

6.4 第 20 步：消费组围栏

目标与前置知识

本步目标：将 assignment 与 fetch/offset commit 结合，用 generation token 拒绝僵尸成员的过期操作。

开始前：理解 token 三元组、撤销分区、刷新 assignment 与外部副作用边界。

背景与推理

rebalance 后，旧 consumer 线程可能仍在运行。GroupToken=(group,member,generation) 是请求携带的组成员版本；broker 必须确认成员仍存在、generation 当前、该成员拥有目标分区，才允许带 token 的 FETCH 或 COMMIT。token 围栏能拒绝旧请求，但不能撤回已经送到旧客户端的记录，也不能撤销已发生的业务副作用。手动模式不带 token，仍由此前的普通 consumer 路径使用。

开始前先读

需要 第 14 步（fetch 的 position）、第 15 步（持久 offset key）、第 17 步（分配算法）、第 18 步（generation/assignment）及第 19 步（heartbeat/expiry）。本步把它们组合起来；不要把成员 generation 当成消费 offset。

起点与编辑位置。

已给：group 请求/响应类型、RpcClient、SimpleConsumer 的 pull/offset 路径及成员生命周期骨架已提供。

请修改：exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java：void validate(GroupToken token, TopicPartition tp)；exercises/src/main/java/io/simplekafka/client/GroupConsumer.java：GroupConsumer(RpcClient client, String group, String member)（接管 client 并在 close 时关闭）、GroupConsumer(Metadata Router router, String group, String member)（借用 router，不关闭它）；两个构造器的 group/member 均须为 1-255 UTF-8 bytes。只实现 subscribe(String topic)、Map<TopicPartition, List<LogRecord>> poll(int maxRecords, int maxBytes)、commitSync()；并在 exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java 增补带 token 的 FETCH/COMMIT 围栏。

参数、结果与状态

- GroupCoordinator.validate(token, tp) 要求当前存在的 member、当前 generation 和该 member 对目标分区的所有权。未知 member 报 UNKNOWN_MEMBER，generation 不符（包括

future token) 报 `ILLEGAL_GENERATION`, 未分配给此 member 报 `NOT_ASSIGNED`; null tp 报 `INVALID_REQUEST`, 拒绝不会写 `OffsetStore`。

- `subscribe(topic)` 显式 join 并拿到 assignment/token; 新分区从 committed offset 或 0 恢复, 仍保留的分区保留原 position, 撤销分区丢弃。未订阅就 poll/commit 报 `INVALID_REQUEST`; 未知 topic 等错误向调用者传播。consumer 关闭后操作报 `IllegalStateException`。
- `poll(maxRecords,maxBytes)` 要求两个总预算均为正, 否则报 `INVALID_REQUEST`, 并在 fetch 请求前 heartbeat; 返回的 map 不可变。若 heartbeat 请求返回 `ILLEGAL_GENERATION`, 只查询当前 assignment、更新本地 token/position, 再用新 token fetch; 撤销位置删除, 新分区从 committed/0 起。遇 `UNKNOWN_MEMBER` 不自动 join, 必须由调用者显式 subscribe; 其他错误传播。成功 fetch 推进相应的内存 position, 不直接提交。
- `commitSync()` 用当前 token 对每个已分配分区分别提交 position; 它不 heartbeat, 也不自行刷新 assignment。旧 token/非 owner 被拒时 offset store 不变; 逐分区请求不是原子事务, 较早提交可能在后续请求失败前已成功。broker 的 COMMIT 分支要求非负 nextOffset, 且 OffsetKey 的 group 与 token、分区与请求一致; 否则 `INVALID_REQUEST` 且不追加。服务端在 coordinator 锁内完成 validate 与 store append, 锁顺序 coordinator 再 store; 它们不是分区日志事务。普通手动请求的 null token 仍合法。
- 协议成功时 token FETCH 返回 `FetchBody`, token COMMIT 返回 `EmptyBody`; assignment 查询返回 `GroupBody`。已发送记录和客户端副作用不因后续拒绝而回滚。coordinator 关闭后操作报 `IllegalStateException`。

行为契约与不变量。FETCH 与 COMMIT 都围栏 member、generation 和分区 owner。COMMIT 的 OffsetKey 必须与 token 的 group 及请求分区匹配; 任何拒绝不得追加该次 offset。generation 变化后, 旧成员不能提交已撤销分区的缓存 position。

可观测例子与测试点。初始 orders-0 只有 offset 0; orders-1 有 offset 0-8, 且 workers/orders-1 已提交 7。a subscribe 得 generation 1、分到两分区; poll(1,4096) 先读 p0/offset0, position 为 p0=1、p1=7。b subscribe 后 generation 2, a 只持有 p0、b 持有 p1。a 用旧 token commitSync 报 `ILLEGAL_GENERATION`, p0 仍未提交、p1 仍为 7。此时 p0 追加 offset1; a poll 检测旧 generation, 获取新 assignment, 保留 p0=1、丢弃已撤销 p1, 再读 p0/offset1; commit 后 p0=2, p1 仍为 7。b 首次 poll 从已提交的 p1=7 开始。若 broker fixture 重启, 成员内存状态消失、日志和 offsets 保留 (p0=2、p1=9); 创建新的 GroupConsumer a 并显式 subscribe, generation 从 1 开始, 新 position 从已提交的 2、9 恢复。

Step20Test.coordinatorFencesCommitsByGenerationOwnershipAndMatchingOffsetKey 检查 coordinator 与错误 key; rawTcpFetchAndCommitTokensFenceInvalidOwnersWithoutChangingOffsets 检查真实 TCP 的旧/future/未知/非 owner token; groupConsumerPreservesRetainedPositionDropsRevokedPartitionsAndRecoversAfterExpiryAndRestart 检查刷新、接管与重启恢复; groupConsumerClosesItsOwnedRpcClientAfterPollingARecord 检查资源归属。

分级提示 (不是答案)

一级 请求校验必须同时核对 member、generation 和 partition owner; 只比较 group 名称不足以围栏。

- 二级** 重平衡后对每个分区分类：保留的本地 position、刚取得所有权需从 committed/0 开始的位置、已撤销必须丢弃的位置。
- 三级** 沿用示例让 a 用旧 generation 提交较大 offset；检查错误码及所有相关 OffsetKey 是否保持原值。

自检问题

1. generation 围栏可以拒绝什么，为什么不能保证外部副作用 exactly-once？
2. 若 validate 解锁后再 commit，两个操作之间可能发生什么？

自检解释。 服务端能拒绝旧 token 的 fetch/commit，却无法撤回已交付的记录或外部系统已执行的动作；若 validate 与追加不在同一 coordinator 临界区，rebalance 可插入两者之间，让旧 member 越权写进度。

常见误区与真实 Kafka 的差异

先校验、释放锁、再提交会留下 rebalance 竞态。把 ILLEGAL_GENERATION 和 UNKNOWN_MEMBER 都转成自动重新 join 会掩盖不同状态；真实 Kafka 组协议和事务提交更复杂，本课程单进程 coordinator 与本地 offset store 不等于高可用 `__consumer_offsets`。

命令与晋级。 `./gradlew :stepTest -Pstep=20` 用于定位本步；`./gradlew :test -Pstep=20` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

第七章 拉取副本、ISR、高水位与确认

本章从单机日志进入复制。**课程的分布式模型边界必须先读清**：三个独立 broker 各有 TCP server、线程与磁盘目录，follower 必须经 TCP 拉记录，不能共享内存复制；但它们运行在同一个 JVM，受一个可信 ClusterAuthority 管理。authority 线性化 metadata、epoch 与 HW 状态，保证教学切换可控。它不是 KRaft，不具备 controller 共识或 authority HA；整个 JVM/authority 丢失后的集群恢复不作保证。

复制只有在追加、报告和可见性边界彼此一致时才有意义。LEO 是每份日志自己的下一条 offset。ISR 是当前被允许参与确认的副本集合。HW 是普通消费者能读取的提交边界，消费者只能读取 $\text{offset} < \text{HW}$ 。副本通过真实网络拉取；authority 只提供一致状态和角色决策，不代替数据移动。

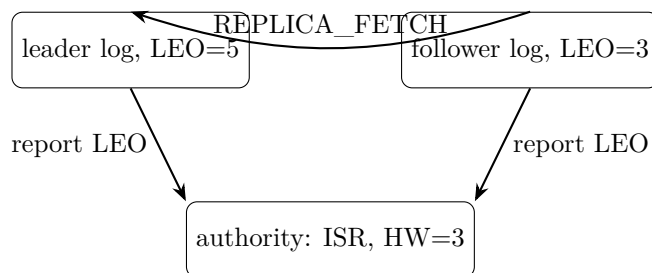


图 7.1: 副本日志由 TCP 拉取，进度由可信 authority 串行化。示例 $\text{HW} = \min(5, 3) = 3$ 。

7.1 第 21 步：拉取复制

目标与前置知识

本步目标：把“另一份日志”做成可验证的磁盘副本：follower 通过网络拉取 leader 记录并追加到自己的日志。

开始前：理解副本为何有独立日志、复制读取为何能超过普通消费者的可见边界，以及一次 poll 不等于同步完成。

背景与推理

先修入口：课程步骤表按顺序要求完成第 20 步（消费组围栏）；实现时还要能读第 6 步（分段日志）的 PartitionLog 与第 12 步（broker 请求）的 request/reply。leader 是当前接受分区写入的

broker; follower 是保存同一分区另一份日志、主动向 leader 拉记录的 broker。每台 broker 的日志独立落盘，复制不是共享对象或复制内存。LEO 是本地日志下一条记录的 offset；HW 是普通消费者可读的排他边界。follower 复制需要拿到 leader 的未提交尾部来追赶，所以 REPLICATION_FETCH 可读 leader LEO，普通 consumer FETCH 则只读 HW 以下。一次 poll 只完成“取回并本地追加”；第 22 步才把成功落盘的 LEO 报给 authority。

起点与编辑位置。

已给：三 broker TCP harness、Messages.ReplicaFetchRequest / ReplicaFetchBody、ReplicaState、PartitionLog 生命周期及 framed RPC 已提供。fixture 的初始 authority reportedLEO 为 0；不能把直接写入 leader 磁盘误当作已报告同步。

请修改：exercises/src/main/java/io/simplekafka/replication/FollowerReplicator.
java: pollOnce(intmaxRecords, intmaxBytes) 返回本次追加条数；并在 exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java 的 handle(Messages.Request) 中接通 REPLICATION_FETCH 分支。

输入、输出与状态。 maxRecords 是正的记录条数上限，maxBytes 是正的编码字节预算，记录不能拆分；不足以容纳下一整条时可以暂时没有进展。构造时 broker id 非负并须与 ReplicaState 的 id 相同；topic-partition、日志、借用的 RpcClient 与状态对象须非空。poll 返回实际追加数；空回复返回 0。RPC 客户端归调用者所有，poll 不关闭它。成功改变 follower 磁盘和本地 LEO；它不改 authority 的 reportedLEO、ISR 或 HW。

行为契约与不变量。 先在状态 monitor 下确认本地 broker 在线且为 follower，并捕获 epoch；在 monitor 外发请求；验证响应后再于同一 monitor 内复核 epoch/角色并追加。请求必须带当前分区、本地 LEO、broker id、捕获 epoch、两个正预算及 recoveryRead=false。

- 响应必须是副本读取 body，body epoch 与捕获 epoch 相同，leader LEO 不得低于本地 LEO，返回记录须从本地 LEO 开始、offset 连续且均小于 body 的 leader LEO。收到字节不代表已成功：只有 PartitionLog.append 成功后才增加返回条数。
- 非正预算或意外 body 为 INVALID_REQUEST；offline/已经成为 leader 为 NOT_LEADER；RPC 前后本地 epoch 改变或响应 epoch 错误为 FENCED_EPOCH；leader LEO 后退、缺 offset、错首 offset 或追加范围不对齐为 CORRUPT_RECORD。leader 端保留起点之外的远端错误（例如 OFFSET_OUT_OF_RANGE）及其 message 原样传播；本地存储错误保持原错误。
- 任何上述验证失败均不得改 follower 日志；若网络请求期间角色变化，不得追加。leader 在 LEO 处返回空列表合法；反复空 poll 不改文件。broker 将过大的正字节预算压到单帧 reply 上限，后续 poll 继续复制剩余连续后缀。若本地写入发生 STORAGE_ERROR，不计入返回条数、也不报告进度；写入可能已有部分磁盘副作用，不能假定自动回滚，应遵循日志 failed/reopen 恢复路径。此步骤不调用 report，也不推进 HW。

分级提示（不是答案）

一级 把“网络已返回”与“follower 磁盘追加成功”分开记录。

二级 逐项比对响应 epoch、首 offset、连续 offset 和 leader LEO；不要拿 HW 替代 LEO。

三级 对同一 follower 再 poll 一次，并分别造错 epoch、错首 offset、超出 leader LEO 的回复；检查 LEO 与文件字节是否保持不变。

可观测例子与测试点。手算前提：分区起点为 0，epoch=0，broker 1 是 leader、broker 2 是 follower；leader 磁盘有三条普通记录，key 均为 null、value 分别是 1 个 ASCII 字节 a,b,c、timestamp 为 11、12、13，故每条占 33 bytes；leader LEO=3、HW=0，authority 初始对三台 broker 的 reportedLEO 仍为 0。broker 2 调用 pollOnce(10,16384) 收到 offset 0-2，追加三条后返回 3、本地 LEO=3；authority 不变。再次 poll 得空回复，返回 0 且文件字节不变。HW=0 时普通消费者仍看不到这些记录，但副本请求能读取它们。

测试与完成标准。 真实 Step21Test 覆盖 replicaFetchIncludesUncommittedLeaderTailAndReturnsEmptyAtLeo、followerCopySurvivesEmptyPollsAndCatalogReopen、boundsLargerReplicaFetchesAndCopiesTheCompleteSuffix、rejectedReplicaFetchRequestPreserveDiskAndAuthority、followerAheadOfLeaderIsRejectedWithoutTruncatingItsLog 与 malformedRealTcpReplicaFetchRepliesAreRejectedBeforeAppend：检查真实 TCP、磁盘字节、空 poll、单帧分页、follower 领先时不截尾、epoch/offset 错误无副作用及重开内容。单步命令检查 Step21；累计命令从 Step1 跑到 21。未实现骨架失败是预期，单步绿不表示第 6 章完成。

常见误区与真实 Kafka 的差异

直接共享 leader 的 PartitionLog 绕过网络与 follower 落盘；把 HW 当成 leader LEO 会使 follower 无法复制尚未提交的尾部。真实 Kafka 的副本协议更复杂；这里的 REPLICA_FETCH 是课程协议，不据此推断真实 Kafka 的内部帧或控制面。

命令与晋级。 ./gradlew :stepTest -Pstep=21 用于定位本步；./gradlew :test -Pstep=21 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么副本读取能取得 HW 以上、普通 consumer 暂时不可见的记录？
2. 为什么 follower 本地 append 成功之前不能把新 LEO 报给 authority？

自检答案。 复制必须把 leader 未提交的尾部带到 follower，才能追赶当前 LEO；普通 consumer 的 HW 边界承担另一项工作。先持久化再报告可避免 authority 把尚未写入 follower 磁盘的进度计入确认。

7.2 第 22 步：ISR 与高水位

目标与前置知识

本步目标：只用经过校验的进度报告计算消费者可见边界，并在受控时钟下移出长期未追上的 follower。

开始前：理解物理 LEO、authority 报告、ISR、minISR 与 HW 不是同一个状态。

背景与推理

先修第 21 步 (拉取复制)：follower 已在自己的日志写入记录，但该动作尚未报告。authority 的每分区状态保存各 broker 最近一次成功报告的 LEO；它与某份日志刚读出的物理 LEO 可以暂时不同。ISR 是当前参与确认的副本集合，minISR 是允许 ALL 请求继续的最少 ISR 数；HW 是 ISR 已共同确认前缀的排他末端。只有成功 report 才能改变 authority 进度。

起点与编辑位置。

已给：每分区锁和 Condition、已分配 replica 集合、在线状态、authority 进度容器与可注入毫秒时钟 TimeSource 已提供。

请修改：exercises/src/main/java/io/simplekafka/replication/ReplicationTracker.
java: report(intbrokerId,intepoch,longleo) 返回 void; expireLagging() 返回被移出 ISR 的 broker id 集合; advanceHighWatermark() 返回当前 HW。

输入、输出与状态。 report 的 broker 必须非负、已分配且在线，epoch/LEO 非负且 epoch 必须等于当前 leader epoch；无效输入不更改快照。leader 报告不得倒退；follower 报告不得超过 leader 已报告 LEO、也不得在同 epoch 倒退。成功报告更新该 replica 的 reportedLEO；follower 只有当 LEO 追到本次观察的 leader LEO 时才更新 last-caught-up 时间。报告本身不会把已移出 ISR 的成员重新加入。

行为契约与不变量。若 ISR 含 leader、成员进度都存在且 ISR 大小至少 minISR，则候选 HW 是 ISR 中最新成功报告 LEO 的最小值，并应用 $HW = \max(oldHW, candidate)$ ；否则 HW 冻结。HW 单调不减。

- 负 broker id、epoch 或 LEO，未知/未分配/离线 broker，leader LEO 倒退，或 follower 超前/倒退均为 INVALID_REQUEST；错误 epoch 为 FENCED_EPOCH。失败 report 不改变 reportedLEO、ISR 或 HW。
- expireLagging() 检查 ISR 中的非 leader；没有 catch-up 时间或 $now - lastCaughtUp \geq lagTimeoutMillis$ 时移出 ISR，返回不可变的已移出集合。timeout=0 表示立即到期；now 早于 catch-up 时间时不按超时删除。过期不删除副本分配和日志，也不删除 leader。
- 移出成员后重新考虑 HW；若余下 ISR 低于 minISR 则冻结。有效 report、实际过期或 HW 推进时通知等待者；advanceHighWatermark() 按当前 ISR 重算并返回 HW，若 HW 实际推进会 signal 等待者；它本身不报告新的副本进度，也不执行 lag 过期。

分级提示 (不是答案)

- 一级** 先验证 follower 报告的“不超过 leader”与“同 epoch 不倒退”，再计算 HW。
- 二级** minISR 不足时不要用已有副本的最小 LEO 推进边界；caught-up 时间只在追平时刷新。
- 三级** 在固定 clock 上分别检查超时前一毫秒、恰好到期时刻及一个非法 report；比较快照而不靠 sleep。

可观测例子与测试点。手算进度例：replicas/ISR=1,2,3、minISR=2、旧 HW=0，leader 1 与 follower 2、3 的已报告 LEO 分别为 5、3、4，则候选最小值为 3，HW=3；follower 2 成功报告 5 后候选为 4，HW=4；follower 3 再报告 5 后 HW=5。单独看超时边界：follower 3 在 clock=1000ms 追平，lag timeout=1000ms，则 now=1999ms 尚保留，now=2000ms 恰好到期并从 ISR 移除；不因它的 LEO 而改写日志。

测试与完成标准。 Step22Test.rejectsInvalidProgressAndExpiresOnlyReplicasThatMissTheExactCatchUpDeadline 使用 clock=1000 起始，验证负进度、future epoch、未分配/离线、follower 超前及倒退均不改 snapshot；clock=1999 不过期、2000 移除 broker 3；也验证 ISR 缩小时 HW 冻结、重新报告推进到 4，而只剩 leader 时即使其报告到 5，HW 仍停在 4。单步命令只定位 Step 22；累计命令从 Step 1 到 22，单步通过不等于第 6 章完成。

常见误区与真实 Kafka 的差异

在线不等于 ISR，物理日志 LEO 不等于 authority 已知 LEO；用前者直接算 HW 会把未确认数据暴露给普通读取。课程用可信单 JVM authority 串行化状态；真实 Kafka 的跨 epoch HW 规则不保证本课程这里的全局单调性。

命令与晋级。 ./gradlew :stepTest -Pstep=22 用于定位本步；./gradlew :test -Pstep=22 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. minISR 不足时为什么已有副本的 LEO 也不能推进 HW？
2. 某 follower 只是在线却没有追到 leader LEO，为什么不能刷新 catch-up 时间？

自检答案。 HW 表示可读的共同前缀，不只是一个副本的写入位置；ISR 数不足时无法按本课程契约满足确认要求。只有实际追平的报告能证明 follower 已到当前 leader 尾部，在线连接本身不证明它持有该数据。

7.3 第 23 步：生产确认

目标与前置知识

本步目标：解释写入成功、复制提交与调用方收到确认的区别，并实现两种等待规则。

开始前：理解 half-open offset 范围、Condition、单调时钟 deadline，以及超时不撤销 append。

背景与推理

先修第 22 步 (ISR 与 HW 计算)。ACK 是 producer 要求 broker 等到哪一层再回复：LEADER 表示 leader 本地追加完成即可返回；ALL 表示需要 minISR 足够，且 HW 到达本批次的 nextOffset。

若返回范围为 $[a,b)$ ，需要 $HW \geq b$ 才说明这批完整进入提交前缀；读者只能取 $offset < b$ 。deadline 是 `System.nanoTime()` 时间域的绝对时刻，`timeoutMillis` 则是请求时长，不能互换。

起点与编辑位置。

已给：ReplicationTracker 的 partition lock/Condition、AppendResult、Acks、RecordData 与 TimeSource 已提供；deadline helper `deadlineAfterMillis(long)` 也已提供。

请修改：`exercises/src/main/java/io/simplekafka/replication/AckPolicy.java`：
`validateBeforeAppend(Acksacks)` 返回 void；`await(AppendResultresult,Acksacks, in tepoch,longdeadlineNanos)` 返回 void。

输入、输出与副作用。 `validateBeforeAppend` 接受非空的 LEADER/ALL；LEADER 不求 ISR 检查，ALL 在追加前检查 ISR 大小。`await` 接受非空结果与 ACK 模式、该 append 捕获的 epoch、单调时钟绝对 deadline；LEADER 立即返回，ALL 等待相应 `nextOffset`。调用端的 `ReplicatedPartition.produce` 负责拒绝负 timeout 或 epoch。null 参数按接口抛 `NullPointerException`。任何追加之后的等待错误都不会撤回磁盘记录。

行为契约与不变量。 ALL 追加前 $ISR < \min ISR$ 报 `NOT_ENOUGH_REPLICAS`，因此还没有追加；追加后按 `epoch`→`ISR`→`HW`→`deadline` 重查：`epoch` 改变为 `FENCED_EPOCH`，ISR 低于 `minISR` 为 `NOT_ENOUGH_REPLICAS`，HW 已到结果 `nextOffset` 则成功，否则 deadline 到期为 `REQUEST_TIMEOUT`。HW 已满足时优先于过期 deadline 返回成功。Condition 等待释放 partition lock，醒来后必须重查；中断恢复线程 interrupt 标志并报 `REQUEST_TIMEOUT`，保留中断为 cause。LEADER 的 `await` 立即完成。

分级提示（不是答案）

- 一级 先分开“追加前 `minISR` 检查”和“追加后 HW 等待”，它们的副作用和错误不同。
- 二级 Condition 被 signal 只表示状态可能变化；重新取得锁后按优先顺序检查全部条件。
- 三级 用 latch 控制 report 顺序，验证 ALL 等待被报告唤醒；再用零 timeout、epoch 改变、ISR 下降和 interrupt 检查失败后的 LEO/HW。

可观测例子与测试点。假设 leader epoch=0、ISR=1,2,3、`minISR`=2，追加一条普通记录返回 [0,1)，其 key 为 null、value 为空、总长 32 bytes，leader LEO=1、follower LEO=0、HW=0。ALL timeout=0 在追加后报 `REQUEST_TIMEOUT`：LEO 保持 1、HW 保持 0、调用方没有 receipt，consumer 看不到 offset 0。稍后两个 follower 都复制并成功 report 到 1，HW 可推进到 1，记录变得可见；原请求不会追溯性地再得到成功 receipt。若在 append 前 ISR 已不足，ALL 则直接报 `NOT_ENOUGH_REPLICAS` 且 LEO 仍为 0。

测试与完成标准。 `Step23Test` 包含 `insufficientIsrRejectsAllBeforeAppendWhileLeaderAckStillAppends`、`exactHighWatermarkBoundarySucceedsButOneOffsetBelowWaitsForProgress`、`zeroTimeoutRetainsTheAppendWithoutAdvancingTheHighWatermark`、`epochChangeWhileWaitingFencesTheAcknowledgement` 与 `isrReductionWhileWaitingFailsAndInterruptionPreservesTheInterruptFlag`，分别验证预检无副作用、HW 边界、超时保留数据、epoch/ISR 唤醒及中断标志。单步命令定位 Step 23；累计命令运行 Step 1-23，只有 Step 24 累计绿才完成本章。

常见误区与真实 Kafka 的差异

不能持 partition lock 等复制网络，否则复制报告永远无法取得锁；wall clock 会受校时跳变影响。timeout 只说明调用方没有按期限取得确认，并不说明 append 没发生；本课程仅提供两种 ACK 且不替调用方重试。

命令与晋级。 `./gradlew :stepTest -Pstep=23` 用于定位本步；`./gradlew :test -Pstep=23` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. Condition 释放锁之后，为什么必须重查 epoch、ISR、HW 与 deadline?
2. ALL 超时，LEO、HW 与调用方知道写入结果之间是什么关系?

自检答案。 等待期间任何状态都可能改变，signal 不是满足条件的证明。超时后 LEO 可已增加而 HW 未增加；因此数据可能稍后 visible，但调用方仍没有可安全使用的成功 receipt。

7.4 第 24 步：并发副本 broker

目标与前置知识

本步目标：把 leader 校验、写入、复制确认与普通读取组装成可并发服务的复制 broker。

开始前：理解 authority lock 与本地日志锁顺序、HW 读取裁剪和 socket/Condition 生命周期。

背景与推理

先修第 21 步（拉取复制）、第 22 步（ISR 与 HW）及第 23 步（生产确认）。普通 consumer 只能读完整提交前缀 $\text{offset} < \text{HW}$ ；offset 在保留范围内但 $\text{offset} \geq \text{HW}$ 时返回空列表。位于 LEO 之外的 offset 才越界。复制 fetch 是另一种请求，可为 follower 取未提交尾部。produce 的权限检查、append 与 leader progress 必须在 authority per-partition lock 保护下线性化；锁序是 authority lock → 本地 PartitionLog monitor。网络调用不能持这两把锁；ALL 等待必须在锁外进行，让 follower report 能继续。

起点与编辑位置。

已给：独立 TCP endpoints、broker worker、ReplicaState、ReplicationTracker、AckPolicy、PartitionLog 与 broker 路由已提供。

请修改：`exercises/src/main/java/io/simplekafka/replication/ReplicatedPartition.java`：`produce(List<RecordData>records, Acksacks, int epoch, long timeoutMillis)` 返回 `AppendResult`；`fetch(long offset, int maxRecords, int maxBytes, int epoch)` 返回 `List<LogRecord>`。

参数、返回与状态。 `produce` 要求 `records/acks` 非空、`epoch` 和 `timeoutMillis` 非负、`broker` 当前在线且是请求 `epoch` 的 `leader`；成功返回 `half-open append` 范围，并可能追加日志、更新复制进度。`fetch` 要求 `offset/epoch` 非负、`maxRecords/maxBytes` 正；它只读取日志和 `authority` 状态，返回受记录数、编码字节预算与 `HW` 三重约束的完整记录，不拆单条记录。网络 `worker` 路由由提供的 `handler` 承担。

行为契约与不变量。 `produce` 先检查参数/`null`，再对 `ALL` 执行首次 `append` 前 `minISR` 检查；该检查发生在 `online/角色/epoch` 检查之前。随后在 `authority lock` 下同步并校验 `online/leader/epoch`，再次检查 `ALL minISR`（防止首次检查后状态变化），然后本地 `append`、`leader report`/可能推进 `HW`，最后解锁并等待 `ACK`。首次或第二次 `minISR` 检查失败、角色/`epoch` 失败均不得追加；首次检查可能使 `minISR` 错误先于 `NOT_LEADER/FENCED_EPOCH` 返回。负参数或空 `produce batch` 为 `INVALID_REQUEST`，非 `leader` 为 `NOT_LEADER`，旧 `epoch` 为 `FENCED_EPOCH`，存储错误为 `STORAGE_ERROR`。追加后 `ACK` 的 `NOT_ENOUGH_REPLICAS`、`REQUEST_TIMEOUT` 或 `FENCED_EPOCH` 不回滚 `append`。

- `fetch` 对负 `offset/epoch` 或非正预算报 `INVALID_REQUEST`，对 `stale epoch` 报 `FENCED_EPOCH`，请求者不是在线 `leader` 报 `NOT_LEADER`；`offset<logStart` 或 `offset>LEO` 报 `OFFSET_OUT_OF_RANGE`。
- `fetch` 的合法 `offset` 域是闭区间 `[logStart,LEO]`；域内且 `offset ≥ HW` 返回空，`offset<HW` 时仍只返回 `offset<HW` 的整条记录。`LEO` 是下一 `offset`，故 `offset=LEO` 可以合法但为空；超过 `LEO` 不合法。
- 锁序必须保持 `authority→local log`，不得倒序或持锁跨 `broker RPC`。关闭 `broker` 时必须让 `socket/等待中的 ACK worker` 结束；`pending append` 仍可能留在日志中。

分级提示（不是答案）

- 一级 画出 `produce`、`replica poll` 与 `consumer fetch` 的锁范围，并把每次 `TCP` 调用放在图外。
- 二级 检查 `HW` 截断发生在普通 `fetch` 回复之前，而不是依赖 `consumer` 自己忽略数据。
- 三级 让 `ALL producer` 追加后暂停复制，同时从另一个连接 `fetch`，再并发写入 `LEADER` 记录；恢复复制后检查三者都能完成。

可观测例子与测试点。并发例前提：`leader=1`、`follower=2/3`、`epoch=0`、初始空日志；先 `LEADER` 追加并复制 `offset 0`，三份 `LEO=1`、`HW=1`。再让 `ALL` 追加 `offset 1`，暂时 `LEO leader=2`、`HW=1`；另一个 `fetch(0)` 只能看到 `offset 0`，`fetch(1)` 为空。`ALL` 等待期间 `LEADER` 仍可追加 `offset 2`（范围 `[2,3)`），但 `fetch` 仍只见 `offset 0`。两个 `follower` 随后复制 `offset 1-2` 并报告 `LEO=3`，`HW` 推进到 `3`，`ALL` 返回 `[1,2)`，普通 `fetch` 才能看见 `0、1、2`。若每条普通记录 `key=null`、`value` 为空，则每条磁盘编码为 `32 bytes`：4 条中 `HW=2` 时，从 `offset 0` 且 `maxBytes=31` 返回空，预算 `32` 或 `63` 只容纳 `offset0`，预算 `64` 可容纳 `offset0、1`；`maxRecords=1` 即使预算更大也只返回一条。

测试与完成标准。 `Step24Test` 的 `pendingAllAckLeavesFetchAtOldHwAllowsLeaderProduceAndCompletesAfterTcpReplication` 检查等待时旧 `HW` 可读与其他 `produce` 进展；`fetchHonorsHwLeoAndWholeRecordByteBudgetsAndRejectsInvalidRequests` 检查 `31/32/63/64-byte` 边界、`HW/LEO/越界`；`allPrecheckDoesNotAppendAndZeroTimeoutRetainsAnInvisibleAppe`

ndUntilReplication 与 stoppingLeaderCompletesAnOutstandingAllAckRequestWithoutLeakingTheProducer 检查错误副作用和关闭唤醒。单步命令定位 Step 24；累计命令从 Step 1 到 24，是第 6 章完成边界。

常见误区与真实 Kafka 的差异

跨 socket 持锁会让唯一能推进复制的线程无法工作；HW 不是 group committed offset。此处三份独立日志经真实 TCP 复制，但 authority 仍是共享 JVM 内单点，因此不代表生产 HA。

命令与晋级。 ./gradlew :stepTest -Pstep=24 用于定位本步；./gradlew :test -Pstep=24 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. ALL 仍等待时，为什么另一个 fetch 可安全读取旧 HW 前缀？
2. 关闭 broker 时只关闭 socket，为什么未必足以结束所有等待线程？

自检答案。 fetch 读的是已经提交且不可变的旧前缀，未提交 offset 不会从回复泄漏。Condition waiter 还需被状态变化/关闭唤醒并检查退出条件；只关连接不能替代对等待线程和 worker 生命周期的处理。

第八章 epoch 围栏、干净选举、分叉修复与重新入组

本章将副本状态变化串成受控故障切换。这里必须牢记模型限制：本课程的 authority 是单 JVM 内可信的 metadata/HW 控制面，leader 切换由课程显式触发，不是 KRaft 控制器共识，也不覆盖网络分区下的双主仲裁。follower 通过真实 TCP 读取 leader 数据，每个 broker 仍写自己的独立日志。课程选择干净候选，不以数据丢失换取可用性；没有候选时服务必须保持不可用。

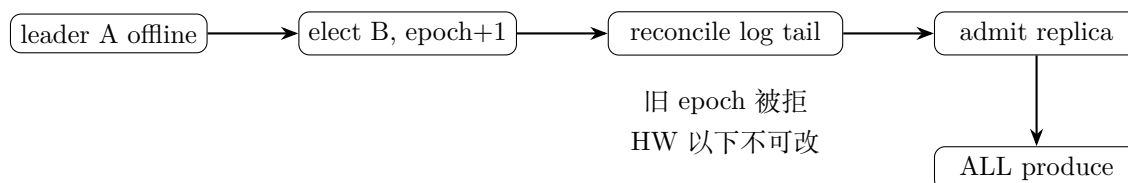


图 8.1: 角色切换必须先围栏旧角色，再修复并证明副本完整，最后才可重新进入 ISR。

8.1 第 25 步：干净选举与 epoch

目标与前置知识

本步目标：旧 leader 离线时只选择保有提交前缀的在线副本，并使旧角色请求失效。

开始前：理解选举候选资格、reported LEO、HW 下界与 epoch 围栏各自解决的问题。

背景与推理

先修第 24 步（并发复制 broker），并回看第 22 步（ISR 与 HW）。leader 是当前接受写入的 broker，follower 保存自己的分区日志。leader 故障后，选举决定由哪份已知日志继续服务；不是随便挑一个在线节点。leader epoch 是角色任期编号，角色改变时递增；请求携带 epoch，broker 用它识别旧任期。epoch 能拒绝旧 broker 的后续请求，但不能回滚它先前产生的磁盘或业务副作用。本课程 authority 是可信单 JVM 控制面，手动触发干净选举；无安全候选时宁可不可用，不做 unclean election。

起点与编辑位置。

已给：provided 的 ClusterAuthority (provided/src/main/java/io/simplekafka/cluster/ClusterAuthority.java) 提供 public boolean isOnline(int brokerId)、public Partit

ionMetadata(metadata(TopicPartitiontp) 与 publicReplicationSnapshot(snapshot(TopicPartitiontp)); ReplicationSnapshot 位于 provided/src/main/java/io/simplekafka/cluster/ReplicationSnapshot.java, 用于核对 ISR/HW/reportedLEO。三个 broker 的网络与本地日志已提供。

请修改: exercises/src/main/java/io/simplekafka/replication/LeaderElection.java: publicPartitionMetadataelect(TopicPartitiontp); publicvoidcheckLeader(intbrokerId, TopicPartitiontp, intepoch)。

输入、结果与状态。 elect 读取一个已知 topic-partition 的 leader、epoch、旧 ISR、在线状态、reportedLEO 与 HW；无单独的传输参数。leader 仍在线则原样返回 metadata，不做状态变更。成功切换只变 authority 角色/进度，不直接改任何 broker 的磁盘。checkLeader 验证 broker id、分区及请求 epoch。

行为契约与不变量。 候选必须同时属于选举开始时的旧 ISR、当前在线，且 authority 已报告的 LEO $\geq$ HW；物理日志看起来更长不能替代报告。符合资格者中选 broker id 最小者，不按 LEO 最大值选。成功时 epoch 恰加一、HW 不回退、ISR 重置为仅新 leader，其他 replica 的 authority reportedLEO 重置为 0 并进入恢复；它们必须修复再 admission 才可参与确认。旧 epoch 请求报 FENCED_EPOCH；epoch 正确但 broker 非在线 leader 报 NOT_LEADER。旧 leader 的 ACK waiter 也须被唤醒。

- 已知 leader 在线：返回现 metadata，epoch/ISR/HW/进度不变。没有候选：NO_ELIGIBLE_LEADER，metadata、epoch、HW、ISR 不变；在线但不属于旧 ISR 的 broker 不能补位。
- 负 broker id/epoch 为 INVALID_REQUEST；不存在分区为 UNKNOWN_TOPIC_OR_PARTITION。选举与检查均不发 TCP RPC。

分级提示（不是答案）

- 一级** 列候选时依次应用旧 ISR、online 与 reportedLEO $\geq$ HW，不要先按“看起来最新”排序。
- 二级** 把旧 epoch 与错误角色区分：前者是 FENCED_EPOCH，后者是 NOT_LEADER。
- 三级** 分别试 leader 仍在线、HW 以下 ISR 副本、在线但非 ISR 副本，以及完全无候选；比较调用前后的 authority snapshot。

可观测例子与测试点。 手算前提：leader 1 离线，epoch=0，旧 ISR=1,2,3，HW=3；日志从 offset 0 起无 retention，三条共同前缀记录 key=null、每个 value 为一个 ASCII 字节、timestamp 固定，故每条 33 bytes。副本 2 的 reportedLEO=3，副本 3 的 reportedLEO=5，二者在线且都属于 ISR。两者都满足 HW 下界，故选 id 较小的 broker 2，即使 broker 3 更长；结果 epoch=1、leader=2、ISR=2、HW 仍为 3。其他 broker 的 authority LEO 被重置为 0，但选举不删除其物理日志。随后 broker 3 带 epoch 1 请求会得 NOT_LEADER；带 epoch 0 的旧请求得 FENCED_EPOCH。若候选都低于 HW 或离线，即使 broker 4 在线也必须报告无候选且不改状态。

测试与完成标准。 Step25Test.electsTheLowestEligibleIsrAndFencesOldEpochWithoutUsingCommittedPrefix 检查在 leader 在线时重复选举无变化、停机切换、HW 保持及旧 epoch 围栏；electionRequiresTheCommittedPrefixBeforeChoosingTheLowestIsr 检查 reportedLEO

低于 HW 的较小 id 被跳过;electionFencesAnOutstandingAllAckWithoutRollingBackAppend 检查待确认 append 不被回滚。单步命令定位 Step25; 累计命令从 Step1 到 25, 单步绿不代表第 7 章完成。

常见误区与真实 Kafka 的差异

只挑在线且 LEO 最大的副本可能引入不在旧 ISR 的分叉; 无候选时偷偷选落后副本会丢掉已知提交前缀。真实 Kafka 的 controller、epoch 历史与共识超出课程模型; 本课程不保证 authority 丢失或网络分区时的生产级 HA。

命令与晋级。 ./gradlew :stepTest -Pstep=25 用于定位本步; ./gradlew :test -Pstep=25 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. 候选 $\text{reportedLEO} \geq HW$ 能保证什么最小安全条件?
2. 为何 epoch 正确但不是 leader 的 broker 仍不能接受 produce?

自检答案。 该条件保证候选至少拥有 authority 已标记为提交的前缀; 它不证明未提交尾部也相同。epoch 识别任期, leader 身份决定当前写入权限, 两项都必须成立。

8.2 第 26 步: 分叉日志修复

目标与前置知识

本步目标: 比较旧副本与新 leader 的相同 offset 内容, 保护共同提交前缀, 只修复允许变化的尾部。

开始前: 理解全前缀扫描、恢复读取、HW 严格边界与 proof 的有效范围。

背景与推理

先修第 25 步 (干净选举与 epoch), 并使用第 8 步 (尾部截断) 与第 21 步 (拉取复制) 中的日志/恢复读取概念。LEO 相同只表示条数相同, 不能证明内容相同; 修复需按 offset 比较每一条 RecordData。课程用易验证但较慢的全前缀扫描: 从 0 读到捕获的 leader LEO, 恢复读取可查看高于 HW 的 leader 尾部, 但不报告复制进度。HW 为排他边界, 因此冲突 $\text{offset} < \text{captured HW}$ 属于已提交内容, 拒绝修复; offset 恰好等于 HW 属于未提交尾部, 可以修。若存在多个可能的截断位置, 截断点不得小于 captured 或最终复核时的当前 HW。

起点与编辑位置。

已给: provided 的 `RpcClient.call(Messages.Request):Messages.Reply (provided/src/main/java/io/simplekafka/transport/RpcClient.java)`、`Messages.ReplicaFetchRequest/ReplicaFetchBody (provided/src/main/java/io/simplekafka/protocol/Message`

s.java)、RecoveryProof (provided/src/main/java/io/simplekafka/cluster/RecoveryProof.java) 及 ClusterAuthority 的 epoch/HW 快照与分区锁已提供; 恢复不自动加入 ISR。

请修改: exercises/src/main/java/io/simplekafka/replication/ReplicaReconciler.java: public long reconcile(int expectedEpoch), 已提供的 public Optional<RecoveryProof> recoveryProof(); 网络请求为 REPLICATION_FETCH(recoveryRead=true), 每批最多 4 条、4,200,000 bytes。

输入、输出与状态。 expectedEpoch 必须与当前 epoch 相符; 对象的 broker id、topic-partition、本地 log、借用 RPC client 与 authority 在构造时确定。reconcile 从 offset0 捕获 leader reportedLEO/HW 与本地 mutationVersion, RPC 全在锁外。成功返回本地 end offset, 并登记 proof; 恢复读取本身不更改 ISR/reportedLEO/HW。进入时清除旧 proof; 验证失败不发布新 proof。

行为契约与不变量。 扫描 leader 捕获前缀与本地完整前缀, 找到第一个实际内容差异或尾部结束点。

- 预期 epoch 过期为 FENCED_EPOCH; 分区不存在为 UNKNOWN_TOPIC_OR_PARTITION; 本地 broker 是 leader 或未分配为 INVALID_REQUEST; 本地 retained start 不为 0、leader 前缀不可读/不连续、已提交冲突或截断低于最终 HW 为 CORRUPT_RECORD。
- 本地 offset < captured HW 且与 leader 内容不同, 报 CORRUPT_RECORD, 保持文件原样; 缺少本地记录 (包括缺少提交前缀) 可由 leader 补齐。恰好 offset=captured HW 的首个分歧允许改写。仅比 leader 多出的未提交后缀可从 leader LEO 截掉。
- RPC 超时或本地日志 API 在扫描期间改变 mutationVersion (即使 truncate 后追加使 LEO 回到原值) 时报 REQUEST_TIMEOUT, 不修复且不发布 proof。epoch/leader 在扫描期改变报 FENCED_EPOCH。错误 RPC code 传播; unexpected body 为 INVALID_REQUEST; 本地磁盘错误保留 STORAGE_ERROR。截断/追加中的 I/O 不保证跨文件原子回滚。
- 最终在 authority lock → 本地 log monitor 顺序下复核 epoch、leader、HW、start、LEO、mutationVersion, 持有日志 monitor 到修复和 proof 发布。proof 绑定捕获 epoch、leader LEO、HW、本地 log 对象身份及最终 mutationVersion; 它不是持久校验, 也不能检测绕过 API 的外部磁盘篡改。

分级提示 (不是答案)

一级 比较“本地缺记录”与“同 offset 有不同内容”; 二者不意味着同一种损坏。

二级 把 HW 想成不含该 offset 的排他边界: 小于它受保护, 等于它可以成为首个可修复位置。

三级 分别构造共同提交前缀 + 未提交分歧尾部、短副本/空副本、同 LEO 并发替换; 检查磁盘与 proof 是否改变。

可观测例子与测试点。 假设 offset 0-2 是共同提交记录, key=null、每条 value 是一个 ASCII 字节、每条 33 bytes; HW=3, 捕获的 leader LEO=4。旧副本 offset3 为 X, 新 leader offset3 为 Y, 二者 timestamp 相同。offset0-2 相同, 首差异恰在 3=HW, 修复后本地 offset3 为 Y、LEO=4 并产生 proof; authority ISR/HW 不变。另一个独立状态若 offset2 已不同 (2 < HW), 报 CORRUPT_RECORD, 本地文件字节和 authority 不变且旧 proof 已清除。若本地为空则补齐 0-3, 不属于已提交冲突。

测试与完成标准。 `Step26Test.repairsOnlyDivergentUncommittedTailAtHighWatermarkAndReturnsLeaderPrefixProof` 检查 `offset=HW` 可修及 `proof`; `refusesMismatchAtHighWatermarkMinusOneWithoutChangingTheLogOrAuthorityAndClearsOldProof` 检查 `offset=HW-1` 的拒绝; `fillsShortAndEmptyReplicasWithTheCommittedPrefix`、`trimsOnlyAnUncommittedTailWhenTheLocalReplicaHasTheCompleteLeaderPrefix` 与 `repairsTenRecordsInFourRecordFetchRoundsWithoutReportingRecoveryProgress` 检查缺失/多余尾部和 4 条分页; 另有 RPC 超时、畸形回复、same-LEO mutation、epoch/HW 变化竞态。单步命令定位 Step26; 累计命令运行 Step1-26。

常见误区与真实 Kafka 的差异

本地 `HW=0` 不表示可以删掉本地已有的有效前缀; 缺失记录不是内容冲突。真实 Kafka 用 `leader-epoch` 历史更有效地找分歧; 本课程的 $O(n)$ 全扫描用于教学, 不应视为生产修复算法。

命令与晋级。 `./gradlew :stepTest -Pstep=26` 用于定位本步; `./gradlew :test -Pstep=26` 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. 为何已提交 `offset` 内容冲突与本地缺少已提交记录必须区别处理?
2. `leader` 捕获快照后又追加, 为什么扫描得到的旧 `proof` 不能直接用于 `admission`?

自检答案。 冲突意味着两边对被保护的已提交值不一致, 不能静默覆盖; 缺失表示副本落后, 若 `leader` 前缀可用则可补齐。`leader LEO` 改变会使 `proof` 没有证明最新尾部, 所以之后 `admission` 必须拒绝并重新扫描。

8.3 第 27 步: 元数据路由刷新

目标与前置知识

本步目标: 让客户端通过 `metadata` 把分区请求送到 `leader`, 同时避免因一次错误自动重放业务操作。

开始前: 从零理解 `bootstrap`、缓存路由、“结果未知”及为什么刷新控制面不等于重试数据面请求。

背景与推理

先修第 26 步 (分叉日志修复), 并回看第 13 步 (批量生产者) 与第 23 步 (生产确认)。客户端通常只知道 `bootstrap endpoint`——用于发现集群的入口; `metadata` 回答 `topic-partition` 当前 `leader endpoint` 与 `epoch`。缓存的 `leader` 是数据请求的目标, `authority/metadata` 请求是控制面, `produce/fetch` 是数据面。结果未知表示 `broker` 可能已完成 `append`, 而回复在网络中丢失或

ACK 等待失败，客户端无法从错误单独判断是否写入。若自动重送普通 produce，可能产生第二条记录。

起点与编辑位置。

已给：provided 的 `RpcClientFactory.connect(Endpoint):RpcClient` (provided/src/main/java/io/simplekafka/transport/RpcClientFactory.java)、`RpcClient.call(Messages.Request):Messages.Reply` (provided/src/main/java/io/simplekafka/transport/RpcClient.java)、`Endpoint` (provided/src/main/java/io/simplekafka/model/Endpoint.java) 与 `MessageCodec.encodeRequest/decodeRequest` (provided/src/main/java/io/simplekafka/protocol/MessageCodec.java) 负责连接/frame；broker metadata 由课程服务端提供。现有 `RpcClient` 单 broker 构造路径仍可用。

请修改：exercises/src/main/java/io/simplekafka/client/MetadataRouter.java：构造器 `MetadataRouter(List<Endpoint>bootstrap,RpcClientFactoryfactory)` 要求 bootstrap 非空且元素非 null、factory 非 null；`publicvoidrefresh(Stringtopic)`、`publicMessages.Replycall(TopicPartitiontp,Messages.Requestrequest)`。Router 缓存并拥有 factory 创建的 `RpcClient`，`close()` 关闭这些 client。路由构造器路径也在 exercises client 中：`SimpleProducer(MetadataRouter,Partitioner,int)`、`SimpleConsumer(MetadataRouter,String)`、`GroupConsumer(MetadataRouter,String,String)`；不要移除单 broker `RpcClient` overload。

输入、输出与状态。 `refresh` 按构造时 bootstrap 列表顺序请求一个 topic 的 metadata；成功缓存该 topic 的 partition metadata 并设置 active bootstrap。连接创建/连接请求失败或 `REQUEST_TIMEOUT` 可继续下一个入口；若所有入口失败，传播最后失败，不造假路由。topic 无效或回复含其他 topic 为 `INVALID_REQUEST`。非 timeout 的 broker error reply/body 错误立即传播，不尝试下一个入口；任何刷新失败都保留上次成功的缓存。Step27 中 `call` 接受 partition 相符的 `ProduceRequest / FetchRequest`；后续第 30 步再加入 `IDEMPOTENT_PRODUCE`。

行为契约与不变量。 `call` 按缓存的 leader endpoint 发出一次业务请求，并携带缓存 epoch。请求类型不支持或请求分区与 `tp` 不匹配为 `INVALID_REQUEST`；没有 topic/partition metadata 为 `UNKNOWN_TOPIC_OR_PARTITION`。

- 收到响应 `NOT_LEADER` 或 `FENCED_EPOCH` 后可 best-effort 刷新缓存，使下一次独立调用用新路由；必须返回本次原始 Reply。即使刷新失败也返回原始 Reply。
- 其他业务错误、RPC timeout、I/O/storage 异常不自动刷新/重放 produce 或 fetch。特别是调用超时可能发生在 broker append 后，未知结果不能解释为“未发生”。
- 刷新只是读 metadata；Router 不为当前 call 重试业务操作。后续调用由上层显式决定。关闭后的 router 操作抛 `IllegalStateException`。

分级提示（不是答案）

一级 给一次 `call` 只画一个业务 request；刷新 metadata 是另一次控制面请求，不是第二次 produce。

二级 检查 bootstrap 失败顺序与缓存：哪些连接错误可继续，哪些响应错误应原样终止刷新？

三级 用旧路由触发 FENCED_EPOCH 或 NOT_LEADER，核对返回的原错误、刷新后的缓存和下一次显式请求。

可观测例子与测试点。手算时假设 bootstrap=[broker1,broker2,broker3],缓存指向 broker1/epoch0,分区空;每条例子记录为 null key、一个 ASCII value 字节、固定非负 timestamp (33 bytes)。broker1 离线并已选举 broker2/epoch1。旧路由请求收到的原始 NOT_LEADER 或 FENCED_EPOCH 回复仍返回给调用方,刷新只把 cache 更新至 broker2/epoch1;该调用不在 broker2 再发 produce,因此不会在这里产生 receipt。调用方随后显式发一个新请求,若其得到 [0,1), broker2 只有一条新记录。若请求曾在 broker 追加后因回复超时,Router 也不重放;Step29 会观察其不确定结果。

测试与完成标准。 Step27Test.refreshSkipsUnavailableAndFactoryThrowingBootstraps BeforeCachingMetadata 检查 bootstrap 顺序; failedRefreshTimesOutWithoutReplacingCacheAndLaterSuccessfulRefreshReplacesIt 检查缓存保留; returnsLeaderErrorsUnchanged AndRefreshesOnlyForTheNextRequest 检查原回复与下一请求路由; routerDoesNotReplayAProduceAfterItsAppendResponseTimesOut、makesRoutedProducerOutcomeUnknownAfterEveryAppendError 与 routerDoesNotReplayStorageOrTransportFailures 检查不重放;另验证借用 router 生命周期和不同 topic 独立路由。单步命令定位 Step27; 累计命令运行 Step1-27。

常见误区与真实 Kafka 的差异

把 NOT_LEADER 当成任何情况下都“绝对没写入”会导致重复;刷新缓存后伪造成功 receipt 也不正确。普通 SimpleProducer 不自动重试。真实 Kafka 客户端有更复杂的重试与请求语义;本课程 Router 明确只刷新路由、不重放当前业务请求。

命令与晋级。 ./gradlew :stepTest -Pstep=27 用于定位本步; ./gradlew :test -Pstep=27 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿,才算完成本章并进入下一章。

自检问题

1. 为什么刷新 metadata 与重放业务请求必须是两个动作?
2. 客户端收到错误时,为什么不能只凭错误码断定服务端没有副作用?

自检答案。 metadata 描述之后请求应去哪里;它不能给先前请求补发结果。超时可能发生在 append 已完成之后,而 FENCED 也可能来自 ACK 等待阶段;若系统没有针对该请求的成功 receipt,调用方就应把结果视为未知而不是凭空判定未写入。

8.4 第 28 步：副本重新入 ISR

目标与前置知识

本步目标：只有新鲜的前缀 proof 才可把已修复副本重新纳入 ISR，并将修复、readmission 与 ALL 可用性连接起来。

开始前：理解 assignment、online/follower 角色、epoch、日志身份和 proof 版本。

背景与推理

先修第 26 步（分叉日志修复）与第 25 步（干净选举）。修复把日志变成当前 leader 前缀的候选副本；ISR readmission 是单独的控制面决定，成功后副本才参与 ALL/minISR 和 HW。LEO 相等只说明记录数量相同，不证明记录内容相同。reconciliation 的 proof 把一次前缀比较绑定到 broker、分区、epoch、leader LEO、本地日志对象与 mutationVersion；leader 后续追加、目标日志变更、截断或 retention 都会使证据过期。

起点与编辑位置。

已给：provided 的 ClusterAuthority (provided/src/main/java/io/simplekafka/cluster/ClusterAuthority.java) 管理角色/ISR/进度，RecoveryProof (provided/src/main/java/io/simplekafka/cluster/RecoveryProof.java) 记录 brokerId, tp, epoch, leader LEO, localLEO, highWatermark, localLog, localMutationVersion；副本重开与 authority 状态容器已提供。

请修改：exercises/src/main/java/io/simplekafka/replication/ReplicaAdmission.java：
`public boolean tryAdd(int brokerId, TopicPartition tp, int epoch)。`

输入、输出与状态。 tryAdd 不做 RPC；输入是目标 broker、分区和预期当前 epoch，具体本地 PartitionLog 在构造时绑定。true 表示已经在 ISR 或刚成功 admission；false 表示当前状态/proof 不允许加入。成功会把目标加进 ISR、以 leader LEO 初始化其 reportedLEO 和 caught-up clock、重算 HW、signal waiters，并消费/清除 proof。

行为契约与不变量。新 admission 必须确认目标已分配、在线、是当前 epoch follower；proof 的 broker/tp/epoch 正确且 leaderLEO 等于当前 authority reported leader LEO；本地日志必须是 proof 绑定的同一对象、mutationVersion 相同、logStart=0、localLEO=leaderLEO，且 proof HW 不高于 leaderLEO。authority lock→ 本地日志 monitor 下验证到 ISR 发布之间不得插入本地日志写。

- 负 broker id/epoch 或未分配 broker 为 INVALID_REQUEST；分区未知为 UNKNOWN_TOPIC_OR_PARTITION；读取本地日志失败为 STORAGE_ERROR。
- epoch 过期、目标 offline、目标当前为 leader，或没有/不匹配的新鲜 proof 时返回 false，不加入 ISR。proof 内容或日志对象/版本/start/LEO 不匹配时还清除 proof；必须再次 reconcile。任何目标日志 append/delete/truncate/close 或 leader LEO 改变都使旧 proof 不可用，即使 truncate+ 重写恢复同 LEO。

- 只有目标在线、是 follower 且使用当前 epoch 时，已在 ISR 的重复调用才返回 true，不改 HW；epoch 过期、目标 offline 或目标是 leader 时按上一项返回 false。首次 admission 后，reportedLEO 设为 leader LEO、caught-up 时间设为当前 clock；满足 minISR 时才可能推进 HW，否则仍可成功 readmission 但 ALL 预检仍失败。

分级提示（不是答案）

- 一级 把 proof 看成针对特定日志对象和版本的短期证据，而不是可永久复用的布尔值。
- 二级 列出 admission 重新核对的当前 leader 状态，以及目标本地日志的身份/版本/start/LEO。
- 三级 先验证无状态变化时 true，再让 leader 追加或目标日志 truncate 后尝试旧 proof，最后重新 reconcile 并 admit。

可观测例子与测试点。手算前提：分区 offset 0 的第一条普通记录 key=null、value 为一个 ASCII 字节、timestamp=30，占 33 bytes；leader1 与副本 2 复制并 report 到 LEO=1，HW=1。leader1 停机后 broker2 成为 epoch1 leader、ISR=2；重启的 broker1 仍有物理 LEO=1，但尚不在 ISR。reconcile 后 proof 绑定 broker1 日志和 leader LEO1。若 leader2 先在 offset1 追加一条 timestamp31 的同尺寸记录，当前 leader LEO=2；broker1 用旧 proof 调用 tryAdd(1,tp,1) 返回 false，ISR/HW 不变，broker1 仍 LEO1。再次 reconcile 把 offset1 补上并产生新 proof，再 admit 返回 true，ISR=1,2 且 reportedLEO=2、HW 可推进到 2。

测试与完成标准。 Step28Test.proofBecomesStaleWhenLeaderAppendsAfterReconciliation 验证 leader 追加使 proof 过期；admissionRequiresCurrentProofAndOnlineAssignedFollowerAndIsIdempotent 验证无 proof/旧 epoch/offline/unassigned 与重复调用；recoveryProofCannotBeUsedWithAnotherLogObjectWithTheSameRecordsAndVersion 与 admissionRejectsProofAfterTruncateAndAfterDifferentDataRestoresSameLeo 验证对象身份及同 LEO 重写；recoveredProofRestoresISRForAllAcksAndReplicaLogsConverge 与 minISR 等集成测试验证 HW 唤醒、ALL 和日志收敛。单步命令定位 Step28；累计命令运行 Step1-28，是第 7 章完成边界。

常见误区与真实 Kafka 的差异

相同 LEO 不证明前缀相同；只核对 proof 创建时的 epoch 而不核对当前 leader LEO 会漏掉新尾部。真实 Kafka 的 ISR 准入与复制协议更复杂；课程的 authority 和 proof 保证是教学模型，不是生产 controller 设计。

命令与晋级。 ./gradlew :stepTest -Pstep=28 用于定位本步；./gradlew :test -Pstep=28 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 两份日志的 LEO 相同为什么不能单独证明副本安全？
2. 为什么成功 admission 后必须更新进度、重算 HW 并唤醒 ALL waiter？

自检答案。 同样长度可以包含不同记录；proof 比较内容并绑定具体日志版本。副本加入 ISR 可能使满足 minISR 的共同最小 LEO 增加，所以需要重算 HW 并通知等待线程重新检查确认条件。

第九章 生产者、broker 与消费者的一致性

本章沿一条消息查看生产确认、broker 复制和 consumer 处理三个彼此独立的边界。结果未知表示服务端可能已完成动作但回复没到；重试是再次提交同一意图，可能产生重复。幂等表示相同稳定身份与内容的重复请求不会再增加一次持久效果；它不等于事务。至少一次处理愿意重放不确定工作，因此副作用可能重复。副作用是回调对日志以外状态的更改，例如更新本地账本。事务应把多个参与者的更改作为一个不可分割的提交；本课程 producer、复制日志、consumer offset 和账本之间没有共同事务。exactly-once 若指端到端每个业务事件恰有一次效果，需要协调这些边界；单独的 ACK、HW、commit 或 producer 幂等都不提供该保证。轨迹使用真实 TCP 和各 broker 独立日志，分开记录物理 LEO、authority 报告、HW、consumer position、持久 committed offset、可见记录与回调效果。

9.1 第 29 步：三层一致性故障实验

目标与前置知识

本步目标：通过可复现的 producer、broker 与 consumer 故障，记录各层真实发生的状态。

开始前：从基础区分成功追加、回复丢失、HW 以下可见性、内存 position、group committed offset 与外部副作用。

背景与推理

先修第 27 步（元数据路由刷新）与第 28 步（副本重新入 ISR）。LEO 是下一条可写的分区 offset；physical LEO 是该 broker 磁盘末端，reported LEO 是 authority 收到的复制进度。HW 是复制提交边界，consumer 只能看到 $\text{offset} < \text{HW}$ 。poll 可能使内存 position 前进，commit 才保存下一次 resume 的位点；两者都不删除日志。callback 可在 commit 前改变外部状态。receipt 是 producer 对追加请求的返回，不是 consumer 可见证明。

起点与编辑位置。

已给：provided 层的真实状态工具位于 `provided/src/main/java/io/simplekafka/lab/`：FaultProxy.java、EffectRecorder.java、ConsistencyObservation.java、ConsistencyTrace.java、LabSupport.java；ClusterHarness 位于 `provided/src/main/java/io/simplekafka/cluster/ClusterHarness.java`。

请修改：`exercises/src/main/java/io/simplekafka/lab/ConsistencyExperiments.java`：五个 public static 方法都返回 ConsistencyTrace：producerOutcomeUnknown(ClusterHarne

ss, TopicPartition, RecordData, FaultProxy)、leaderAckLoss(ClusterHarness, TopicPartition, RecordData)、allAckTimeout(ClusterHarness, TopicPartition, RecordData)、consumerReplay(ClusterHarness, TopicPartition, String, RecordData, EffectRecorder, FaultProxy) (throwsException) 与 staleGroupSideEffect(ClusterHarness, TopicPartition, String, RecordData, EffectRecorder)。

先理解观察工具。 ConsistencyObservation 的完整类型为 recordConsistencyObservation(String phase, ErrorCode error, ReplicationSnapshot replication, Map<Integer, Long> physicalLeo, OptionalLong position, OptionalLong committedOffset, List<LogRecord> visible, List<LogRecord> effects, Optional<AppendResult> receipt); ConsistencyTrace 为 recordConsistencyTrace(TopicPartition tp, List<ConsistencyObservation> observations)。effects 保留每次调用, 不去重。提供的 FaultProxy(endpoint) 暴露 upstream():Endpoint、start():Endpoint、endpoint():Endpoint、armed():boolean、dropNextReply(short)、rejectNextRequest(short)、forwarded/dropped/rejected(short):int 和 close(); 同一时刻只准 arm 一个 API。drop 会把请求转给真实 broker 并读到回复后丢弃回复; reject 在转发前返回 REQUEST_TIMEOUT 与 ErrorBody("injected pre-forward failure")。不匹配的 API 原样转发。LabSupport.replicateAndReport(ClusterHarness, TopicPartition):void 实际通过 TCP 复制并只报告持久后的 LEO; observe(String, ErrorCode, ClusterHarness, TopicPartition, OptionalLong, String, EffectRecorder, Optional<AppendResult>):ConsistencyObservation 独立读取可见记录与 committed offset; awaitLeaderLeo(ClusterHarness, TopicPartition, long):void 等待 authority 收到指定进度。provided EffectRecorder.process(TopicPartition, LogRecord):void 和 snapshot():List<LogRecord> 按调用顺序记录回调。

行为契约与不变量。五个方法都要求调用方提供已启动的 cluster、唯一 partition tp、非空且未 stamped 的输入记录。初态必须是 epoch0、leader1、replicas/ISR=1,2,3、三者在线且日志 start=end=0、HW=0、minISR=2。consumer 场景还要求 group 无 committed offset 且 recorder 初始为空; group 与 recorder 必须同时提供或同时不提供。两个 proxy 场景要求调用方持有已启动、尚未 armed 且 upstream 是 broker1 的 proxy。初态不符、参数空值或输入带 stamp 为 INVALID_REQUEST, 且不可追加/改 offset/改 recorder/proxy 故障计数。只关闭方法自有 client; 只捕获本实验预期错误, 其余异常传播。

实验一: producer 回复丢失后显式重发。 测试记录取 key="customer-A" (10 ASCII bytes)、value="request-unknown-73" (18 bytes)、timestamp=1723; 普通记录占 32 + 10 + 18 = 60 bytes。reader 从 position0 开始, producer 设置 batchRecords=1。时间线中 physical LEO 顺序为 broker 1/2/3; replication 值为 leader/epoch/ISR/reported LEO/HW。

- UNKNOWN: broker 已追加 offset0; LEADER 回复丢失, 因此 producer 报 REQUEST_TIMEOUT。状态: replication=(1/0/1,2,3/1,0,0/0), physical LEO=(1,0,0); position=0, committed=none, visible=[], effects=[], receipt=none。客户端没有收到回复。
- POISONED: 同一普通 producer 本地拒绝后续 send, error=REQUEST_TIMEOUT; replication/LEO 仍为上行状态, position=0, committed=none, visible=[], effects=[], receipt=none; 此时 forwarded(PRODUCE)=1、dropped(PRODUCE)=1, 第二次调用没有经过网络。

- EXPLICIT_RESEND: 新 producer 显式发送相同数据, 得到 receipt=[1,2)。三份日志完成复制并报告后:

replication=(1/0/1,2,3/2,2,2/2), physical LEO=(2,2,2);

reader position=2, committed=none, visible=[offset0,offset1] (内容相同), effects=[], error=NONE。

最终 forwarded(PRODUCE)=2、dropped(PRODUCE)=1。

实验二: LEADER receipt 不保护未提交尾部。 测试输入 key="leader-tail" (11 bytes)、value="acked-but-uncommitted" (21 bytes)、timestamp=2419, 记录 64 bytes。启动无 committed offset 的 reader, position=0。

- ACKED_NOT_VISIBLE: broker1 追加 offset0 后返回 LEADER receipt=[0,1), error=NONE; replication=(1/0/1,2,3/1,0,0/0), physical=(1,0,0), position=0、committed=none、visible=[], effects=[]。
- ELECTED_WITHOUT_TAIL: 停 broker1 后选举 broker2/epoch1; replication=(2/1/2/0,0,0/0), physical=(1,0,0), reader position=0、committed=none、visible=[], effects=[], receipt=none、error=NONE。旧日志还在 broker1 磁盘, 但不是新 leader 可见的 HW 前缀。
- OLD_TAIL_REMOVED: 重启 broker1, 先 reconcile 再 admit; offset0 被截去。
新状态: replication=(2/1/1,2/0,0,0/0), physical=(0,0,0);
position=0, committed=none, visible=[], effects=[], receipt=none, error=NONE。第一次 LEADER receipt 不会被追溯撤销。

实验三: ALL 超时后稍晚可见。 测试记录为 key="all-key" (7 bytes)、value="late-visible-value" (18 bytes)、timestamp=3101, 占 57 bytes; consumer position 初始 0。

- TIMEOUT_NOT_VISIBLE: 以 Acks.ALL, timeoutMillis=0 追加, error=REQUEST_TIMEOUT; replication=(1/0/1,2,3/1,0,0/0), physical=(1,0,0), position=0、committed=none、visible=[], effects=[], receipt=none。追加不回滚。
- LATE_VISIBLE: 后续完成真实复制并报告三份副本。
新状态: replication=(1/0/1,2,3/1,1,1/1), physical=(1,1,1);
poll 后 position=1, committed=none, visible=[offset0], effects=[], receipt=none, error=NONE。
迟到的 HW 不会补发或创造原请求 receipt。

实验四: commit 失败后副作用重放。 测试记录 key="order-87" (8 bytes)、value="effect-after-poll" (17 bytes)、timestamp=4003, 占 57 bytes。起初 Kafka 日志已经复制到 offset0, HW=1; consumer group 尚无 commit。proxy 在第一次 COMMIT_OFFSET 转发前拒绝。

- COMMIT_FAILED: first consumer resume=0, poll/callback 后 position=1, error=REQUEST_TIMEOUT。
状态: replication=(1/0/1,2,3/1,1,1/1), physical=(1,1,1);
committed=none, visible=[offset0], effects=[0], receipt=none。proxy counts: rejected(COMMIT_OFFSET)=1、forwarded(COMMIT_OFFSET)=0。

- **REPLAY_COMMITTED**: 新 consumer 因没有持久位点再次 resume=0, 重放 callback 后 commit nextOffset=1 成功; error=NONE, replication=(1/0/1,2,3/1,1,1/1), physical=(1,1,1), position=1, committed=1, visible=[offset0], effects=[0,0], receipt=none。重复的是外部处理, 不是 Kafka 日志追加。

实验五: stale group side effect。 测试记录 key="inventory-13" (12 bytes)、value="old-owner-effect" (16 bytes)、timestamp=5007, 占 60 bytes; 消息已复制并在 offset0 可见。

- 成员 z 以 generation1 取得 partition、fetch offset0 并完成 callback。成员 a 加入后 generation2 把 partition 分给 a。z 用旧 token 的 commit 与 fetch 均得到 **ILLEGAL_GENERATION**; 观察 phase=STALE_COMMIT_REJECTED、error=ILLEGAL_GENERATION、replication=(1/0/1,2,3/1,1,1/1)、physical=(1,1,1)、position=1、committed=none、visible=[offset0]、effects=[0]、receipt=none。
- a 从无已提交位点处 fetch 同一 offset0, 再处理并提交 nextOffset1。phase=NEW_OWNER_COMMITTED、error=NONE、replication/LEO/HW 不变、position=1、committed=1、visible=[offset0]、effects=[0,0]、receipt=none。generation fencing 拒绝旧 broker 请求, 不能撤销 z 已经做的 callback。

分级提示 (不是答案)

- 一级** 逐个区别 physical LEO、authority reported LEO、HW、client position 与 durable committed nextOffset; 它们是不同的状态。
- 二级** 给五条故障时间线标注 append、复制、callback、commit 与错误发生的先后, 不把一个层次的成功推断为另一层次成功。
- 三级** 对照每阶段真实 snapshot/trace, 再独立检查调用方仍持有的 FaultProxy 计数和 EffectRecorder 列表; 不以 sleep 制造竞态。

可观测例子与测试点。先用 UNKNOWN→POISONED→EXPLICIT_RESEND 检查一次实际 append、一次本地拒绝、一次显式重发; 再分别重演 leader election、ALL 超时、commit 拒绝和 generation 改变。注意: LEADER receipt=[0,1] 可与 HW0/不可见并存; commit 拒绝可与 callback 已执行并存; group 当前 owner 可再次执行该副作用。通过的是状态演算, 不是“所有阶段都成功”的假设。

测试与完成标准。 Step29Test.producerOutcomeUnknownAndExplicitResendAreObserved 核对 reply drop、poison 与重复 offset; leaderAcknowledgmentDoesNotProtectAnUncommittedTail 核对 clean election、旧尾删除及 admission; allAcknowledgmentTimeoutCanBecomeVisibleLater 核对无 receipt 的 late visibility; consumerCommitFailureReplaysEffectAndCommitsOnFreshConnection 核对 commit 转发计数、position、offset 与 callback 重放; staleMemberSideEffectSurvivesGenerationFencing 核对旧 token 两个错误与旧副作用。另有 proxy 单故障、stamped input、非空日志和无效初态不改变状态的测试。单步命令定位 Step29; 累计命令运行 Steps1-29。

常见误区与真实 Kafka 的差异

receipt 只说明某种 ACK 的结果；LEADER 已确认的尾部仍可在选举中丢失。timeout 不证明未追加。position 在 commit 前推进；generation fencing 约束 broker 请求而非外部 callback。这些实验不重试普通 SimpleProducer，不声称端到端事务或 exactly-once。

命令与晋级。 ./gradlew :stepTest -Pstep=29 用于定位本步；./gradlew :test -Pstep=29 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 哪些观察能区分 LEADER 确认与复制提交后的可见性？
2. 为什么旧成员 commit 被拒，仍可能已有副作用？

自检答案。 检查 receipt、physical/reported LEO、HW 与 visible：LEADER receipt 可先于复制和可见。callback 与 commit 是两个动作，commit 失败只说明进度未持久化；它不会回滚已经发生的效果。

9.2 第 30 步：幂等生产者

目标与前置知识

本步目标：持久保存 producer batch 身份，使同一批次的显式重试返回原 offset 区间而不重复追加。

开始前：从基本身份概念出发，贯通 stamped record、日志恢复、复制 ACK、路由和显式 client retry。

背景与推理

先修第 29 步（三层故障实验）以及第 1 步（记录格式）、第 6 步（分段日志）与第 24 步（复制写入）。网络错误可能让 producer 不知道 broker 是否追加；若没有持久身份，直接重发就是一条新记录。幂等追加给一批逻辑记录稳定身份，broker 可以把完全相同的重试映射回第一次的 offset。四种编号不可混淆：leader epoch 是 broker 角色任期；producer epoch 是调用方的 producer 会话任期；firstSequence 是该 producer 在该分区中下一个 batch 序号；Kafka offset 是分区日志中的位置。eventId 则标识业务事件，第 31 步用它跨不同 offset 去重。

起点与编辑位置。

```
已给：provided 层提供 ProducerStamp (provided/src/main/java/io/simplekafka/model/
ProducerStamp.java) 与 RecordData (provided/src/main/java/io/simplekafka/model/
RecordData.java)，ProducerBatchFingerprint.fingerprint(List<RecordData>):byte
[] (provided/src/main/java/io/simplekafka/protocol/ProducerBatchFingerprint.ja
va)、ProducerBatchSupport.validateAppend(ProducerStamp,List<RecordData>):void
```

(provided/src/main/java/io/simplekafka/storage/ProducerBatchSupport.java)、Messages.IdempotentProduceRequest、MessageCodec.encodeRequest(Messages.Request):byte[]、decodeRequest(short,byte[]):Messages.Request (provided/src/main/java/io/simplekafka/protocol/Messages.java、provided/src/main/java/io/simplekafka/protocol/MessageCodec.java)、以及 IdempotentProduceBackend.produceIdempotent(List<RecordData>,Acks,int,long,long,int,long):AppendResult (provided/src/main/java/io/simplekafka/broker/IdempotentProduceBackend.java)。IdempotentProducer(MetadataRouter,long,int) 构造器/close() 与 IdempotentAppender(PartitionLog) 构造器在对应 exercise 类中已给, Appender 不会自动 recover。

请修改: 按依赖次序完成六个子任务; 实际编辑路径与方法签名如下。

1. **身份与格式。** provided 的 ProducerStamp(longproducerId,intproducerEpoch,longfirstSequence,intbatchSize,intbatchIndex,byte[]batchHash) 对每条记录保存生产者 ID、producer epoch、batch 首序号、批量大小、零起批内索引及 32-byte SHA-256 hash。id/epoch/sequence 非负、batchSize>0、index 属于 [0,batchSize)、sequence+size 不溢出。hash 输入为大端 count:int32, 逐记录 timestamp:int64|keyLength:int32(-1=null,0=empty)|key|valueLength:int32|value; 不包括 offset/leader epoch。普通记录字节维持原格式; stamped 记录多 64 bytes。磁盘格式为 length|crc32c|offset|timestamp|marker:int32=-2|producerId:int64|producerEpoch:int32|firstSequence:int64|batchSize:int32|batchIndex:int32|batchHash:32|keyLength:int32|valueLength:int32|key|value, CRC 覆盖 offset 至 value。普通记录最小总长 32、stamped 最小 96; length 字段上限 1,048,576、总编码最多 1,048,580 bytes (4-byte 前缀另计入总长)。null key 时 stamped value 最大 1,048,484 bytes, key/value 均计入限制。教学 wire 的 stamp 布局不同于磁盘: marker、identity、hash、keyLength/key、valueLength/value、timestamp。
2. **记录编解码与单段日志。** 在 exercises/src/main/java/io/simplekafka/storage/RecordCodec.java 扩展 publicstaticByteBufferencode(LogRecord) 和 publicstaticLogRecorddecode(ByteBuffer), 保留普通记录格式/CRC/原有 buffer 契约, 并以 marker 区分 stamped 记录。扩展 exercises/src/main/java/io/simplekafka/storage/SegmentLog.java 的 publicAppendResultappend(List<RecordData>) 与 publiclongrecover(): 写入、恢复时维护/重建尾 ProducerStamp, 验证记录结构与 CRC。provided helper voidtruncateToOffset(long) 只是 SegmentLog 的截尾操作, 不是 PartitionLog 的新 API。
3. **分区日志与前缀版本。** 在 exercises/src/main/java/io/simplekafka/storage/PartitionLog.java 扩展 publicsynchronizedAppendResultappend(List<RecordData>)、publicsynchronizedlongdeleteBefore(longoffset)、publicsynchronizedvoidtruncateTo(longnextOffset); 提供 longmutationVersion()、ProducerStamptailProducerStamp()、longprefixVersion()。append 要求非空、合法 batch, 连续分配 offset, 整批校验后按段追加并更新尾 stamp; tail 没有 stamped 记录时 tailProducerStamp() 为 null。deleteBefore 只删 endOffset<=offset 的非 active 封闭段、至少保留一个段, 负 offset 为 INVALID_REQUEST, 返回实际 logStart; truncateTo 的合法域是 [logStart,LEO], 越界为 OFFSET_OUT_OF_RANGE, 等于 LEO 不变。有效日志 mutation 在首次 I/O 前记一次; 实际前缀改写/保留删除增加 prefixVersion, no-op 不使缓存失效。I/O 失败可能已改磁盘, 错误状态按日志的 failed/reopen 约定处理。不可把 SegmentLog 的 truncateToOffset(long) 写

成 PartitionLog API。

4. **日志派生的 batch 去重。**在 `exercises/src/main/java/io/simplekafka/replication/IdempotentAppender.java` 实现 `publicAppendResultappend(longproducerId,intproducerEpoch,longfirstSequence,List<RecordData>records)` 和 `publicvoidrecover()`。构造器 `IdempotentAppender(PartitionLog)` 已给。`append` 接收普通 `unstamped` 输入, 验证完整 batch 后才追加; `recover` 从保留日志派生 `producer epoch`、`nextSequence`、完成批次范围与 `hash`, 不把易失 `cache` 当持久真相。日志 `start>0` 时报 `PRODUCER_STATE_EXPIRED`; 完整历史格式/序列/hash 损坏报 `CORRUPT_RECORD`。
5. **复制写入与 API 11。**扩展 `exercises/src/main/java/io/simplekafka/replication/ReplicatedPartition.java` 的 `publicAppendResultproduceIdempotent(List<RecordData>records,Acksacks,intleaderEpoch,longtimeoutMillis,longproducerId,intproducerEpoch,longfirstSequence)`; 扩展 `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java` 的 `publicMessages.Replyhandle(Messages.Requestrequest)` 分派 `API_IDEMPOTENT_PRODUCE=11`, 成功仍回 `ProduceBody(AppendResult)`; 扩展 `exercises/src/main/java/io/simplekafka/client/MetadataRouter.java` 的 `publicMessages.Replycall(TopicPartitiontp,Messages.Requestrequest)` 路由 `API11`。`provided` 的请求记录签名为 `IdempotentProduceRequest(TopicPartitiontp,intepoch,Acksacks,longtimeoutMillis,longproducerId,intproducerEpoch,longfirstSequence,List<RecordData>records)`, `record` 输入按序且未 `stamped`。单 `broker backend` 不支持该 `capability`, 须返回 `INVALID_REQUEST` 且不得追加。
6. **client 与显式 retry。**在 `exercises/src/main/java/io/simplekafka/client/IdempotentProducer.java` 实现 `publicProduceReceiptsend(TopicPartitiontp,List<RecordData>records,Acksacks,longtimeoutMillis)` 和 `publicProduceReceiptretry(TopicPartitiontp)`; `IdempotentProducer(MetadataRouterrouter,longproducerId,intproducerEpoch)` 与 `close()` 已提供。每分区维护 `nextSequence` 与最多一个 `pending batch`; 失败保留其冻结身份/数据, 显式 `retry` 仅调用一次 `RPC`。

错误、状态与副作用。 负 `producer id/epoch/sequence`、`null topic partition/acks`、`null/empty batch`、`batch` 内 `null`、带 `stamp` 的输入、序列加法溢出、每条记录超上限、负 `timeout` 或超帧均为 `INVALID_REQUEST`, 必须在追加前拒绝且不消耗 `sequence`。新 `producer` 或更高 `producer epoch` 必须从 `sequence0` 开始; 只有一条 `stamped` 记录持久化后新 `epoch` 才生效。当前 `epoch` 只接受精确 `nextSequence`; 旧 `epoch` 为 `FENCED_PRODUCER_EPOCH`, 跳号/未记录旧序号为 `OUT_OF_ORDER_SEQUENCE`。同一已完成 `producerId/producerEpoch/firstSequence` 的 `batchSize` 不同为 `OUT_OF_ORDER_SEQUENCE`; `batchSize` 相同但 `batchHash` 不同为 `DUPLICATE_SEQUENCE_CONFLICT`。相同 `identity`、边界和 `hash` 的完整重复返回原 `[firstOffset,nextOffset)`, 不追加、不改 `physical LEO`; 复制 `backend` 仍需检查当前 `leader epoch/minISR`, 并完整等待 `ALL` 的 `HW` 条件。若只有同身份尾 `batch` 前缀已持久化, 精确原请求校验已有前缀后只写缺失 `suffix` 并返回整个原范围; 其他 `producer/batch` 写入报 `INCOMPLETE_BATCH`, 同身份不同 `payload` 报 `DUPLICATE_SEQUENCE_CONFLICT`。保留起点 `>0` 返回 `PRODUCER_STATE_EXPIRED`。

教学协议的单帧上限。 API 11 仍使用第 11 步的课程 TCP frame: length 字段最小 10、最大 8,388,608 bytes (不含 4-byte length prefix), 最多 8,388,598-byte payload, 总帧最多 8,388,612 bytes。producer 须在发送前拒绝超过该帧限额的 batch 请求; 记录自身仍受上项 1,048,580-byte stamped record 上限约束。

派生状态和调用边界。 producer-state cache 由日志 stamped 记录推导。读取新尾部时增量扫描; prefixVersion 改变或 LEO 回退时丢弃 cache 并从 offset0 重建; recover 逐条执行 read(offset, 1, 1_048_580), 检查每 producer 序列、batch index 与 batch hash。部分 batch 只允许留在日志尾部; recover 可保留它等待原请求续写, 完整但不合法的历史拒绝且不改原日志。存储 mutation 后遇到不确定 I/O 错误时不要把内存状态当成持久成功, failed log 须重开恢复。leader epoch 与 producer epoch 不同: leadership 切换后同一个 producer 身份仍可用原 producer epoch 重试。

client 边界。 IdempotentProducer(router, producerId, producerEpoch) 构造时身份非负; 负 id/epoch 为 INVALID_REQUEST。在可用 producer 上, send 还拒绝 null tp、records、acks, 空批、null 记录、负 timeout、stamp 输入、超记录/帧界或 sequence 溢出。metadata 失败且数据 RPC 尚未发出时不构成未知追加。一次请求可能到达 broker 后失败, 因此失败留下 per-partition pending; 同 partition 另一 send 为 INVALID_REQUEST, 其他 partition 仍可发送。仅成功 receipt 且长度等于 batch 条数时推进 nextSequence 并清 pending。无 pending 的 retry(tp) 为 INVALID_REQUEST; retry 先显式刷新 metadata, 再用原 producer id/epoch、firstSequence、记录快照、acks、timeout 向新 route 只发一次。刷新、RPC 失败或非法 receipt 均保留 pending。close 清除 client 本地 pending、不关闭借用 router; 已关闭 client 的 send/retry 抛 IllegalStateException。

行为契约与不变量。 只对同一 producer identity、同一 batch 边界和同一内容返回原持久范围; 所有失败路径不得伪造 receipt。复制错误不会回滚已追加的 stamped 记录。

分级提示 (不是答案)

- 一级 先区分 leader epoch、producer epoch、batch sequence、Kafka offset 和业务 eventId, 各自不能互换。
- 二级 把持久日志看作真相、producer-state cache 看作带 prefixVersion 的派生结果; 说明普通 append、retention 与 truncate 分别如何影响它。
- 三级 由存储开始按六项顺序实现并观察磁盘: 先检查一条完整重复不改字节, 再检查部分 batch 只补 suffix, 最后在 API11 丢回复后切换 leader 并由显式 retry 验证一份持久批次。

可观测例子与测试点。 假设 partition 无保留、leader broker1/epoch0、producerId=7、producerEpoch=0、firstSequence=0; batch 是三条 ASCII 记录 ("retry-0", "value-zero", 1001)、("retry-1", "value-one", 1002)、("retry-2", "value-two", 1003)。key/value 长度分别 (7,10)、(7,9)、(7,9) bytes; 普通记录分别 49、48、48 bytes, stamped 磁盘记录分别 113、112、112 bytes。每条 batchIndex 为 0/1/2, batchSize=3 且共享同一 hash。本例调用 send(tp, batch, Acks.LEADER, 5_000); 若 API11 回复丢失, 客户端保留 pending、leader 物理 LEO=3。复制并报告所有副本后 HW=3; broker1 停机、broker2 成为 leader/epoch1 后, 显式

`retry(tp)` 保留原 LEADER ACK 与 `producerEpoch0/sequence0` 并返回原 `[0,3)`, 各日志仍三条。重新打开 `broker2` 磁盘, 原请求仍由日志恢复识别并返回 `[0,3)`, 字节与 LEO 不变。若该重复请求使用 `Acks.ALL`, `broker2` 的 `ISR=2<minISR=2`, 会在追加前报 `NOT_ENOUGH_REPLICAS`; `allAckDuplicateWaitsForHwAndStillRequiresMinIsrAndCurrentEpoch` 覆盖此边界。反例: 若一个三条 batch 只写 `offset0-1`, 普通新请求得到 `INCOMPLETE_BATCH`; 原身份/尺寸/hash/payload 补交时仅写 `offset2`, 返回完整 `[0,3)`。

测试与完成标准。 `Step30Test.producerRejectsInvalidBatchesBeforeChangingSequenceOrLogs` 与 `sequenceErrorsDoNotAppendOrInstallAnUnpersistedEpoch` 检查输入、epoch/sequence 错误无副作用; `stampedRecordDiskBytesAreIndependentAndStructurallyChecked` 检查磁盘格式/CRC/长度; `incompleteBatchSurvivesRecoveryAndCanOnlyBeCompletedByItsOriginalRequest`、`sameLeoRewriteCannotReuseOldProducerReceipt`、`recoveryRejectsCompleteBatchWithIncorrectFingerprint` 与 `retainedPrefixCannotServeCachedOrRecoveredProducerReceipts` 检查恢复、部分批次、前缀失效; `singleBrokerRejectsIdempotentProduceWithoutAppending` 与 `idempotentApiAndStampedFetchWireFormatsRoundTrip` 检查 API; `allAckDuplicateWaitsForHwAndStillRequiresMinIsrAndCurrentEpoch`、`lostResponseRetryAndDiskReopenPreserveOneDurableBatch`、`boundedReplicaFetchAndDiskReopenPreserveStampedSegments`、`reconciliationComparesProducerStampsAtCommittedAndUncommittedOffsets` 检查 ACK、真实 TCP、磁盘重开与修复。单步命令定位 Step30; 累计命令运行 Steps1-30, 但第 8 章累计完成还需 Step31。

常见误区与真实 Kafka 的差异

同一 producer `retry` 去重的是分区内带 sequence 的记录 batch, 不是不同 offset 上的业务事件。它不自动分配 producer id, 不是 Kafka PID/session 协议, 不是 transactional producer, 也不保证业务副作用 exactly-once。普通 `SimpleProducer` 仍不重试; 只有新 client 的调用方式式 `retry`。

命令与晋级。 `./gradlew :stepTest -Pstep=30` 用于定位本步; `./gradlew :test -Pstep=30` 从第 1 步累计运行至本步。仅当本章末尾编号 (4、8、12、16、20、24、28、31) 的累计测试全绿, 才算完成本章并进入下一章。

自检问题

1. 为什么已完成重复 batch 仍须检查当前 epoch、minISR 与 HW?
2. retention 删除日志前缀后, 为什么不能猜回 producer sequence 或旧 receipt?

自检答案。 重复请求不需要新追加, 但 ALL 承诺的是当前 leader 与副本已满足条件; 复制确认条件可能随选举/ISR 变化。被删前缀可能包含 producer 首次使用的 epoch、sequence 和原 offset 范围, 没有这些持久证据不能安全区分新写与重复。

9.3 第 31 步：消费副作用幂等

目标与前置知识

本步目标：Kafka record 被重复投递时，仍只将每个稳定业务事件应用到账本一次。

开始前：解释 offset 去重与 eventId 去重的差异，并实现可恢复的本地持久账本。

背景与推理

先修第 30 步（幂等生产者），并回看第 16 步（处理后提交）与第 29 步（副作用重放）。Kafka offset 表示一条日志记录的位置，不是业务事件身份：同一事件可被重发到另一个 offset；callback 次数也可能因 commit 失败而增加。稳定 eventId 来自业务数据，若相同事件在重放和重开后保持相同 ID，账本可以将它作为应用层去重键。这里实现的是一个本地强制落盘日志与内存余额的协调，不与 Kafka offset store 共享事务。

起点与编辑位置。

已给：provided 层提供 `BusinessEvent(StringeventId,Stringaccount,longdelta)` (`provided/src/main/java/io/simplekafka/model/BusinessEvent.java`) 与 `MessageCodec.encodeBusinessEvent(BusinessEvent):byte[] / decodeBusinessEvent(byte[]):BusinessEvent(provided/src/main/java/io/simplekafka/protocol/MessageCodec.java)`。

请修改：`exercises/src/main/java/io/simplekafka/client/DurableEffectStore.java`：现有骨架的 `publicDurableEffectStore(Pathdirectory)` 构造器打开 `PartitionLog` 并先调用 `recover`；实现 `publicsynchronizedbooleanapply(BusinessEventevent)` 与 `publicsynchronizedvoidrecover()`。已有 API 为 `publicsynchronizedlongbalance(Stringaccount)` 与 `publicsynchronizedvoidclose()`。

业务事件与持久格式。 `BusinessEvent` 的 `eventId/account` 都必须非空、严格 UTF-8 且各不超过 255 bytes；`delta` 为任意 signed long，正数加款、负数减款、0 也是合法。提供的 value 格式为 `version:uint8=1|eventId:length-prefixedidentifier|account:length-prefixedidentifier|delta:int64`；未知版本、截断、非法 UTF-8/标识或尾随字节在 codec 层为 `INVALID_REQUEST`。`BusinessEvent` 构造时拒绝非法标识并抛 `IllegalArgumentException`。`store` 每个账本事件恰写一条 `RecordData(null,encodedEvent,0)`，即 `key=null`、`timestamp=0`、`producerStamp=null`；普通记录字节有 32-byte 固定开销。

行为契约与不变量。`apply` 对新合法 `eventId` 先用 checked addition 计算账户余额，再强制追加一条记录，最后发布 `seen` 与 `balance` 内存状态，成功返回 `true`。完全相同的 `eventId/account/delta` 重放返回 `false`，不追加、不改余额；同 ID 但 `account` 或 `delta` 不同为 `INVALID_REQUEST`，余额溢出也为 `INVALID_REQUEST`，两者都不追加。

- 在 `store` 正常且日志完整时，`apply(null)` 返回 `INVALID_REQUEST`；无效 `eventId/account` 无法形成合法 `BusinessEvent`（提供构造器抛 `IllegalArgumentException`）。`balance(account)` 对不存在账户返回 0；`null`、空、非法 Unicode 或超过 255 UTF-8 bytes 的 `account` 返回 `INVALID_REQUEST`。

- 同一 store 实例的 apply/balance/recover/close 同步。构造时和显式 recover() 从 offset0 按序、逐条读取一条完整记录 (maxRecords=1,maxBytes=1_048_580) 重建事件身份与余额; 要求 logStart=0, 每记录 key null、timestamp0、没有 stamp。完整非法 payload、格式字段不符、同 ID 历史冲突、余额累计溢出或 scan 无进度为 CORRUPT_RECORD, 不得改写完整损坏日志。
- 底层磁盘末尾只有不完整记录时, PartitionLog/SegmentLog 恢复规则截到最后完整记录后再重建; 完整但 CRC/结构/业务编码错误不可按截断处理。若已存在重复的完全相同 event 历史, 恢复只计第一次余额但保留日志中既有记录。
- append I/O/返回范围不确定时不发布内存新余额并把实例标 failed; 截断账本 (logStart>0) 报 CORRUPT_RECORD 并 failed。failed 或 closed 之后 apply/balance/recover 为 STORAGE_ERROR; 必须 close 后新建实例由磁盘恢复。close 可重复调用。

分级提示 (不是答案)

- 一级** 用业务 eventId 去重, 不要使用 Kafka offset; 同一事件重投后 offset 可以不同但 eventId 仍相同。
- 二级** 每个新事件先检查 ID 冲突和 checked 余额, 再 force 追加; 只有追加成功才更新内存 seen/余额。
- 三级** 先验证同一事件第二次 apply 不增长日志, 再 close 并重开 store 重放历史, 比较余额、seen 行为和磁盘字节。

可观测例子与测试点。 设一个单分区输入日志按 offset 0、1、2 依序携带 A、A、B, 三条输入 record 都用 null key、timestamp0、value 为编码后的事件。A=BusinessEvent("event-A","acct",+10), 重复 A 的 eventId/account/delta 完全相同; B=BusinessEvent("event-B","acct",-3)。ASCII eventId 分别 7 bytes、account4 bytes; 每个编码 value 为 1+4+7+4+4+8=28 bytes, ledger 普通记录占 32+28=60 bytes。apply 依次返回 true、false、true; 账本仅有 [A,B] 两条 (120 bytes), 余额 7。若第一次 callback 处理全部三条后 commit 失败, callback history=[0,1,2] 但 committed 仍空; 新 consumer 从 0 重放并成功提交 3, history=[0,1,2,0,1,2], 账本仍 [A,B] 且余额 7, committed nextOffset=3。Kafka offset 没有变成 eventId, 两个系统也没有共同事务。

测试与完成标准。 Step31Test.appliesAndRecoversDurableBusinessEffects 检查首次/重复 apply、余额和重开恢复; rejectsConflictsInvalidIdentifiersAndOverflowWithoutAppending 检查冲突、UTF-8 边界与溢出; businessEventCodecRejectsMalformedPayloads 检查 value wire 格式; concurrentDuplicateApplicationsAppendOnceAndRecoverOnce 检查同步去重; recoversRepeatedHistoryAndTruncatesIncompleteFinalRecord 与 rejectsCorruptCompleteHistoryWithoutChangingItsLog 区分不完整尾与完整损坏; consumerCommitFailureReplaysCallbacksButNotDurableBusinessEffects 检查真 TCP commit 失败、6 次 callback、2 条账本记录、余额 7 及 committed=3。单步命令定位 Step31; 累计命令运行 Steps1-31, 是第 8 章完成边界。

常见误区与真实 Kafka 的差异

账本强制落盘只协调一个本地 store 的记录与内存发布；它不和 producer、Kafka log 或 consumer offset 形成分布式事务。相同事件可位于不同 Kafka offset；stale group token 能拒绝 commit，却不能回滚账本。课程例子是应用层本地去重，不宣称全局 transaction 或端到端 exactly-once。

命令与晋级。 `./gradlew :stepTest -Pstep=31` 用于定位本步；`./gradlew :test -Pstep=31` 从第 1 步累计运行至本步。仅当本章末尾编号（4、8、12、16、20、24、28、31）的累计测试全绿，才算完成本章并进入下一章。

自检问题

1. 为什么稳定 eventId 比 Kafka offset 或 callback 次数适合作为业务去重键？
2. 账本追加成功但 consumer offset commit 失败后，哪些状态尚未协调？

自检答案。 offset 区分日志位置，callback 计数只记录尝试；二者都不能识别跨位置的同一业务事件。账本已持久生效而 commit 仍未保存时，重放会重复调用 callback，但 eventId 让账本返回 false 并避免重复余额变更；消费进度与账本仍不是原子提交。

附录 A 格式、错误码与接口索引

本附录把各章反复使用的稳定约定集中列出，方便实现时查阅；若和练习里的公开签名不一致，以课程 manifest 和 Java 类型为准。本课程定义的布局与错误码只属于 simpleKafka 教学接口，除非明确引用官方资料，不应把它们称为 Kafka wire、存储格式或配置默认值。

A.1 磁盘与网络格式

A.1.1 记录与索引

记录为大端编码：

字段	宽度	语义
length	int32	不含自身的记录长度，含 CRC；28 至 1,048,576。
crc32c	int32	校验 offset 到 value 末尾，不含 length/CRC 字段自身。
offset	int64	非负逻辑位点。
timestamp	int64	非负时间戳。
keyLength	int32	-1 为 null，0 为存在的空 key，其余为 bytes 数。
valueLength	int32	非负 bytes 数，value 不为 null。
key/value	变长	按前述长度紧跟在字段后。

每条记录的固定长度部分（不含 length 前缀）包含 CRC、offset、timestamp、两个 length，共 28 bytes；长度前缀使最小总占用为 32 bytes。length 字段值下限仍是 28，因为其统计从 CRC 开始。每条 segment 的稀疏索引条目是 `offset:int64,position:int64`，共 16 bytes；索引每 intervalRecords 条记录写一个点，首条总有点。索引可以丢弃并由日志重建。普通记录保持原布局。stamped record 在 timestamp 后、keyLength 前插入以下 64-byte producer identity：
`marker:int32=-2|producerId:int64|producerEpoch:int32|firstSequence:int64|batchSize:int32|batchIndex:int32|batchHash:32bytes`。完整布局为 `length|crc32c|offset|timestamp|marker|producerId|producerEpoch|firstSequence|batchSize|batchIndex|batchHash|keyLength|valueLength|key|value`。最小总长为 96 bytes，1 MiB 上限包含 stamp。CRC32C 覆盖 offset 到最后一个 value byte，包含 stamp。普通 record 字节不变。已提供的 `ProducerStamp.ENCODED_OVERHEAD` 为 64 bytes，磁盘与 MessageCodec wire record 的 1 MiB 上限均包含这部分开销。

目录约定为 `<root>/<topic>/<partition>/<baseOffset:020d>.log`，配对索引后缀为 `.index`。保留范围是 `[logStartOffset,LEO)`，fetch 左闭右开；LEO 是下一条可写 offset，HW 是普通 consumer 能观察的最高提交边界（仅 `offset < HW` 可见）。group committed offset 也是 next offset，但不等于 HW。

A.1.2 TCP frame 与请求体

教学 frame 为 `length:int32|version:int16(1)|api:int16|correlationId:int32|error:int16|payload`，length 不含自身，范围 `[10,8,388,608]`。读到新 frame 首字节前 EOF 返回 null；frame 内 EOF 报 `INVALID_REQUEST`。请求 error 字段必须为 0。未知 version/API、过长 frame 和尾随 body 字节均拒绝。每条磁盘记录上限 1 MiB，不因 frame 上限而改变。

API id 固定如下：

ID	API	ID	API
1	METADATA	6	JOIN_GROUP
2	PRODUCE	7	HEARTBEAT
3	FETCH	8	LEAVE_GROUP
4	COMMIT_OFFSET	9	GROUP_ASSIGNMENT
5	FETCH_OFFSET	10	REPLICA_FETCH
11	IDEMPOTENT_PRODUCE		

MessageCodec 使用大端基础字段；字符串为 int32 UTF-8 byte 长度后接 bytes，集合为 int32 数量后逐项，Optional 值用 boolean 再接值，byte[] 用 int32 长度（仅 key 可 -1）。请求/成功响应字段按 provided Messages nested record 声明顺序编码；map 结果按 key 排序。Reply 头部 error=NONE 时 body 是相应成功 record；错误时 body 为 `ErrorBody(message)`，错误码只在 frame header 出现。字段 schema 由 `provided/.../protocol/Messages.java` 固定，学生不需重新定义十一种消息格式。FETCH 或 REPLICA_FETCH wire 上的 stamped RecordData 为 `marker=-2|producerId|producerEpoch|firstSequence|batchSize|batchIndex|batchHash[32]|keyLength|key|valueLength|value|timestamp`；普通 record 的 wire 编码不变。

A.1.3 消息 body 字段顺序

下面的请求与响应由 MessageCodec 按表内参数声明顺序编码。集合长度和字符串/byte 长度均为大端 int32；列表项无额外 tag，按各 value type 声明顺序内联。GroupToken nullable，用 boolean present 后跟 group、member、generation；Acks 为 int16，LEADER=1、ALL=-1。

API	请求记录与字段	成功响应记录与字段
METADATA	MetadataRequest: topic	MetadataBody: partitions
PRODUCE	ProduceRequest: tp, epoch, acks, timeoutMillis, records	ProduceBody: result
FETCH	FetchRequest: tp, epoch, offset, maxRecords, maxBytes, token	FetchBody: records, logStartOffset, logEndOffset, highWatermark, epoch

API	请求记录与字段	成功响应记录与字段
IDEMPOTENT_PRODUCE	IdempotentProduceRequest: tp, epoch, acks, timeoutMillis, producerId, producerEpoch, firstSequence, records	ProduceBody: result
COMMIT_OFFSET	CommitOffsetRequest: key, nextOffset, token	EmptyBody
FETCH_OFFSET	FetchOffsetRequest: key	OffsetBody: nextOffset (OptionalLong)
JOIN_GROUP	JoinGroupRequest: group, member, topic	GroupBody: assignment
HEARTBEAT	HeartbeatRequest: token	EmptyBody
LEAVE_GROUP	LeaveGroupRequest: group, member	GroupBody: assignment
GROUP_ASSIGNMENT	GroupAssignmentRequest: group, member	GroupBody: assignment
REPLICA_FETCH	ReplicaFetchRequest: tp, brokerId, epoch, fetchOffset, maxRecords, maxBytes, recoveryRead	ReplicaFetchBody: records, epoch, highWatermark, leaderLogEndOffset

Record	Fields in declaration order
TopicPartition	topic, partition
RecordData	key, value, timestamp, nullable producerStamp
LogRecord	offset, data
AppendResult	firstOffset, nextOffset
OffsetKey	group, tp
Endpoint	host, port
GroupAssignment	generation, assignments
PartitionMetadata	tp, leaderId, epoch, replicas, isr, leader
ProducerStamp	producerId, producerEpoch, firstSequence, batchSize, batchIndex, batchHash
BusinessEvent	eventId, account, delta

所有 byte array 都带长度前缀；只有 key 可以为 null（长度为 -1），value 始终存在，但允许是空数组。

OffsetStore 条目使用 MessageCodec.encodeOffsetEntry 编码，key 为 null、timestamp 为 0。记录在日志中的物理 offset 与已提交的消费 nextOffset 相互独立。BusinessEvent 的 value 使用 MessageCodec.encodeBusinessEvent 编码：version:uint8=1|eventId:identifier|account:identifier|delta:int64。两个标识均为非空、严格 UTF-8 且最长 255 bytes；delta 可正、可负或为零。

A.1.4 标识符与值类型

topic、group、member 是非空 UTF-8 标识且最长 255 bytes；topic 只能包含 ASCII 字母数字、点、下划线、连字符，并拒绝 . 与 ... TopicPartition 的 partition 非负。RecordData 的 key 可以 null、value 不可 null（允许空数组），timestamp 非负；LogRecord offset 非负。数组所有权移交值对象后由接收方持有，不需要反复复制。所有 offset 表示下一条时都使用 half-open 边界，具体区别见正文各章。

A.2 幂等追加与持久化副作用

API id 11 是独立的 IDEMPOTENT_PRODUCE 请求；普通 PRODUCE 仍为非幂等模式，并拒绝带 stamp 的 client 输入。请求携带 tp,epoch,acks,timeoutMillis,producerId,producerEpoch,firstSequence,records；broker 为每条记录加入请求的 batch identity，并对记录数量与 timestamp/key/value 内容流式计算 SHA-256 指纹。producer id 与 epoch 由调用方管理。新 producer epoch 从 sequence 0 开始，且只有记录持久化后才生效。完全相同的已完成请求返回原 offset 区间，不追加；相同 identity 携带不同内容会冲突；sequence gap 与未知的旧 sequence 会被拒绝。匹配的部分 batch 可只追加缺失后缀来完成。派生状态从保留日志历史恢复；当 logStartOffset 非零时，幂等请求报 PRODUCER_STATE_EXPIRED。本课程没有 producer-state snapshot 或 client session WAL。

IdempotentProducer 每次调用只发送一次。调用失败会保留 pending batch；只有调用方显式调用 retry 才刷新 metadata，并以同一 identity、sequence 和 payload 重发一次。重复请求仍通过普通 ACK precheck 并等待确认，包括 ALL 的 ISR/HW/deadline 条件。这不改变从不动重试的 SimpleProducer。

DurableEffectStore 将一条 BusinessEvent 作为一条本地日志记录 force 写入。首次 event id 会追加并更新账户余额；id 与内容都相同的重放返回 false，不再追加；同 id 不同内容或 checked balance 溢出会被拒绝。恢复从保留日志重建事件身份和余额，并要求日志起点为零。该本地账本不与 ProcessingLoop 的 offset commit 原子协调：commit 失败后 callback 可再次运行，而稳定 event id 可防止本地记账重复。group fencing 不能撤销已经发生的副作用。

A.3 固定错误码

Wire id 名称	用途
0 NONE	成功。
1 INVALID_REQUEST	参数、长度、版本或请求格式错误。
2 UNKNOWN_TOPIC_OR_PARTITION	topic/partition 不存在。
3 OFFSET_OUT_OF_RANGE	offset 超过当前日志起点/终点约束。
4 CORRUPT_RECORD	完整记录 CRC、格式或已提交历史冲突。
5 NOT_LEADER	请求者不是当前 leader。

Wire id 名称	用途
6 FENCED_EPOCH	请求使用旧 leader epoch。
7 NOT_ENOUGH_REPLICAS	当前 ISR 少于 minISR。
8 REQUEST_TIMEOUT	请求 deadline 到期，结果可能未知。
9 ILLEGAL_GENERATION	消费组 generation 已过期。
10 UNKNOWN_MEMBER	消费组 member 未登记。
11 NOT_ASSIGNED	操作涉及未分配 partition。
12 NO_ELIGIBLE_LEADER	无满足干净选举条件的候选。
13 STORAGE_ERROR	I/O 失败；保留底层 cause。
14 FENCED_PRODUCER_EPOCH	请求使用旧 producer epoch。
15 OUT_OF_ORDER_SEQUENCE	batch sequence 或边界无效。
16 DUPLICATE_SEQUENCE_C ONFLICT	重复 batch identity 的内容不同。
17 PRODUCER_STATE_EXPIR ED	所需 producer 历史已不在保留日志前缀中。
18 INCOMPLETE_BATCH	无关追加会跨越未完成的 producer batch。

未知 ID 不映射为成功；I/O 错误不吞 cause；网络错误 body 不回传服务器堆栈。

A.4 三十一歩与编辑入口

所有路径均相对于仓库根目录，学生修改 `exercises/src/main/java/io/simplekafka/` 下指定方法。每步累积命令为 `./gradlew :test -Pstep=N`；单步定位命令为 `./gradlew :stepTest -Pstep=N`，它不是晋级证明。

步 课程标题	编辑方法
1 记录编解码	storage/RecordCodec: encode, decode
2 日志追加	storage/SegmentLog: append
3 按位点读取	storage/SegmentLog: readFrom
4 崩溃恢复	storage/SegmentLog: recover
5 稀疏索引	storage/SparseIndex: add, floor, rebuild
6 分段日志	storage/PartitionLog: append, read
7 日志保留	storage/PartitionLog: deleteBefore
8 尾部截断	storage/PartitionLog: truncateTo
9 分区路由	client/Partitioner: choose
10 主题元数据	broker/PartitionCatalog: createTopic, metadata
11 TCP 帧协议	protocol/FrameCodec: write, read
12 单机 broker	broker/BrokerHandler: handle
13 批量生产者	client/SimpleProducer: send, flush

步 课程标题	编辑方法
14 拉取消费者	client/SimpleConsumer: assign, seek, poll
15 消费位点持久化	group/OffsetStore: commit, fetch; client/SimpleConsumer: resume, commitSync; broker/BrokerHandler: COMMIT_OFFSET, FETCH_OFFSET
16 至少一次处理	client/ProcessingLoop: runOnce
17 分区分配	group/RoundRobinAssignor: assign
18 成员与重平衡	group/GroupCoordinator: join, leave, assignment; broker/BrokerHandler: JOIN_GROUP, LEAVE_GROUP; GROUP_ASSIGNMENT
19 心跳与过期	group/GroupCoordinator: heartbeat, expire; broker/BrokerHandler: HEARTBEAT
20 消费组围栏	group/GroupCoordinator: validate; client/GroupConsumer: subscribe, poll, commitSync; broker/BrokerHandler: FETCH/COMMIT_OFFSET token fencing
21 拉取复制	broker/BrokerHandler: handle(REPLICA_FETCH); replication/FollowerReplicator: pollOnce
22 ISR 与高水位	replication/ReplicationTracker: report, expireLagging, advanceHighWatermark
23 生产确认	replication/AckPolicy: validateBeforeAppend, await
24 并发副本 broker	replication/ReplicatedPartition: produce, fetch
25 干净选举与 epoch	replication/LeaderElection: elect, checkLeader
26 分叉日志修复	replication/ReplicaReconciler: reconcile
27 元数据路由刷新	client/MetadataRouter: refresh, call
28 副本重新入 ISR	replication/ReplicaAdmission: tryAdd
29 三层一致性故障实验	lab/ConsistencyExperiments: producerOutcomeUnknown, leader-AckLoss, allAckTimeout, consumerReplay, staleGroupSideEffect
30 幂等生产者	client/IdempotentProducer: send, retry; replication/IdempotentAppender: append, recover; replication/ReplicatedPartition: produceIdempotent; broker/BrokerHandler: handle(IDEMPOTENT_PRODUCE); client/MetadataRouter: call; storage/RecordCodec: encode, decode; storage/SegmentLog: append, recover; storage/PartitionLog: append, deleteBefore, truncateTo
31 消费副作用幂等	client/DurableEffectStore: apply, recover

前七章各含四步；第 8 章含第 29–31 步。章节末累计通过点依次为 4、8、12、16、20、24、28、31。若前一步未通过，后一步测试通常也会失败，因为课程采用同一持续演进的系统而非互

不相干的演示程序。

A.5 课程简化与 Kafka 的差异

1. **路由**：key 使用 CRC32C unsigned modulo；Kafka 默认 key partition 算法不同。null key 采用实例级 round-robin，而非 Kafka 的 sticky batching。
2. **磁盘**：记录逐条编码并每 batch 显式 `force(false)`，不是 Kafka record batch 与默认 page-cache 路径；强制写入不保证媒介级断电安全。
3. **索引**：每固定记录数写一个稀疏点，不按 bytes 间隔。
4. **消费组**：仅单 topic、即时重分配的简化模型，不是完整 classic barrier、cooperative 重平衡或 KIP-848。token 围栏仅保护课程 broker 内 offset 提交，不原子化外部副作用。
5. **offset**：本地 OffsetStore 不是高可用复制压缩的 `__consumer_offsets`。
6. **复制与控制面**：各 broker 用真实 TCP 拉取并写独立目录；authority/coordinator 却在同 JVM 且单一可信，不是 KRaft，不保证 controller/authority 故障后的集群恢复或任意分区下共识。
7. **可见性假设**：本课程保留已提交前缀，要求干净候选 LEO 不低于 authority HW，使受控切换的已知提交边界不回退；这比对真实 Kafka 跨 epoch HW 行为更强，不可推广成 Kafka 永远单调的结论。
8. **重试、幂等与事务**：普通 SimpleProducer 从不自动重试，timeout 可能已追加。独立的 IdempotentProducer 支持由调用方管理 producer identity 并显式重试，batch stamp 持久化在日志中；但它不实现 Kafka PID 分配、transaction coordinator、client session WAL 或 producer-state snapshot，retention 可能使所需前缀过期。本地 DurableEffectStore 按稳定业务 event id 去重，但与 consumer offset commit 不原子协调。没有 Kafka 事务或 read_committed 保证、compaction、fire-and-forget 或 ELR。at-least-once 容许重复；本教学子集不声称端到端 exactly-once。
9. **恢复**：第 26 步全量比较前缀为 $O(n)$ 教学简化；真实系统利用 leader epoch 历史缩小分叉搜索范围。

A.6 官方资料与延伸阅读

以下资料用于解释 Kafka 概念与真实系统差异，并非课程自定义 wire/storage 布局的规范。版本标签反映本书所引用页面；不同版本配置和协议细节可能改变，阅读时以所列页面自身版本为准。

主题	官方资料	关联章节
Kafka 设计与持久化模型	Kafka 4.2 Design	日志、复制、消费者位置的官方概览；本课程简化项见上节。
消费位置与提交	KafkaConsumer API	第 14–16 步：position、seek、commit 与线程约束。

主题	官方资料	关联章节
最小 ISR 配置	Kafka 4.3 Topic Configs	第 22–24 步: min.insync.replicas 与写入确认。
消费组协议差异	Kafka 4.3 Rebalance Protocol	第 17–20 步: classic 与新协议背景; 课程不实现完整协议。
leader epoch 与截断	KIP-101	第 26 步: 为何仅靠 HW 不足以定位分叉。
JUnit 测试运行	JUnit 5.11.4 User Guide	阅读实际 Step 行为失败与测试报告。
Tectonic 编译选项	Tectonic compile	本书采用 XeTeX/CJK 与本地 cache 的构建入口。

A.7 排版与构建说明

默认主文件为英文版 `simpleKafka.tex`, 使用 book 文档类; 中文版 `simpleKafka-zh.tex` 使用 `ctexbook[UTF8,fontset=none,openany]`。共享 preamble 通过 `fontspec` 与 `xeCJK` 显式使用 TeX bundle 内的 Latin Modern 与 Fandol 字体文件, 不探测系统字体; Noto、`fontconfig`、`xelatex` 与 `latexmk` 都不是构建依赖。代码片段使用 `listings`, 不依赖 `minted`、`shell-escape` 或联网网页嵌入。TikZ 图由 TeX 本地绘制。EPUB 使用 Pandoc 3.12.1 自定义 reader 处理同一套展开后的 TeX 源码; Tectonic 渲染六张 TikZ 图, Poppler 的 `pdftoppm` 将其栅格化并嵌入 EPUB, 不维护第二份图形源码。

`./gradlew :setup` 解析经过 checksum 校验的 JUnit; `./gradlew :setupBook` 准备 checksum 校验的 Tectonic 0.17.0 与 Pandoc 3.12.1, 并检查 Poppler 的 `pdftoppm`。Poppler 需单独安装; `:setupBook` 不编译 Java 或解析 JUnit, 但 Wrapper 启动它仍需 Java 17+。`./gradlew :book` 默认生成英文版; `COURSE_LANG=zh ./gradlew :book` 生成中文版。`COURSE_LANG` 只接受 `en` 或 `zh`。英文 PDF/EPUB 位于 `build/book/en/simpleKafka.pdf` 与 `build/book/en/simpleKafka.epub`, 中文位于 `build/book/zh/simpleKafka-zh.pdf` 与 `build/book/zh/simpleKafka-zh.epub`。每次构建只更新所选语言在 `docs/book/dist/` 下的成品; 仓库克隆与源码 ZIP 已包含这四份文件, 读者无需 Java 或编译工具。首次构建在线获取 TeX bundle 与字体, `cache` 位于仓库 `.tools/tectonic-cache`; 两种语言都在线成功后, 可用 `BOOK_OFFLINE=1` 为 PDF 和图形构建启用 Tectonic only-cached 模式, Gradle `--offline` 则独立控制 Gradle 依赖解析。教材构建不编译 Java, 也不使用 JUnit。显式 `TECTONIC` 可提供兼容引擎; `PANDOC` 若设置必须为 3.12.1; `TEX_BUNDLE` 接受 URL、本地目录、`.zip` 或 `.ttb`, 不接受普通 `.tar`。首次 clone 离线构建不受保证; bundle URL 也不是完整资源集的内容摘要, PDF/EPUB 不承诺字节级可重复。

术语索引

术语	本书中的含义与位置
消息 / 记录 (message / record)	生产者交给系统保存和读取的一条数据；本书先在导读建立概念，再于 第 1 步（记录编解码） 解释字节格式。
生产者 (producer)	发送记录的程序角色；批量发送见 第 13 步（批量生产者） 。
broker	接收客户端请求并保存、提供记录的服务角色；单机请求见 第 12 步（单机 broker） 。
消费者 (consumer)	主动拉取记录并执行处理的程序角色；读取位置见 第 14 步（拉取消费者） 。
topic (主题)	相关记录的命名分类；注册和查询分区信息见 第 10 步（主题元数据） 。
partition (分区)	topic 内独立编号、独立保持顺序的一条日志；追加和读取多个分区见 第 6 步（分段日志） 。
key / value	记录可选的关联/路由字节与必需的内容字节；null key 与空 key 的编码区别见 第 1 步（记录编解码） 。
timestamp (时间戳)	随记录保存的时间值；本课程要求非负，但记录顺序由 offset 决定，见 第 1 步（记录编解码） 。
追加式日志 (append-only log)	新记录写到日志尾部，不因读取而删除；首次编码和追加见 第 2 步（日志追加） 。
offset (位点)	分区内记录的逻辑编号；追加结果用左闭右开区间表示，见 第 2 步（日志追加） 。
byte position (字节位置)	某个段文件中的物理字节位置，与逻辑 offset 不同；见 第 3 步（按位点读取） 。
LEO (日志末端位点)	下一条可追加记录的 offset，而不是最后一条记录的 offset；见 第 2 步（日志追加） 。
segment (日志段)	一个分区日志按文件切出的部分；段滚动见 第 6 步（分段日志） 。
sparse index (稀疏索引)	保存部分 offset 与文件位置、帮助缩短搜索的可重建数据；见 第 5 步（稀疏索引） 。
retention (日志保留)	按阈值删除已封闭的旧段，不代表消费者读过；见 第 7 步（日志保留） 。
truncation (截断)	删除某个 next offset 之后的日志尾部，与删除旧头部相反；见 第 8 步（尾部截断） 。

术语	本书中的含义与位置
metadata (元数据)	告诉客户端 topic 的分区及其 broker 位置的信息；见 第 10 步 (主题元数据) 。
frame (帧)	TCP 字节流中带长度边界的一条完整课程请求/响应；见 第 11 步 (TCP 帧协议) 。
batch (批次)	一次一起校验、写入或发送的多条记录；存储追加见 第 2 步 (日志追加) ，producer 缓冲见 第 13 步 (批量生产者) 。
consumer position (消费位置)	当前消费者下一次读取的分区 offset，主要在内存中推进；见 第 14 步 (拉取消费者) 。
committed offset (已提交消费位点)	持久保存、供重启后恢复的下一条消费 offset；不等于日志 LEO 或 HW，见 第 15 步 (消费位点持久化) 。
consumer group (消费组)	共同消费 topic、协调分区归属的一组成员；见 第 18 步 (成员与重平衡) 。
assignment (分配)	coordinator 当前交给每个成员的分区集合；基础轮询分配见 第 17 步 (分区分配) 。
rebalance (重平衡)	成员变化后重新计算分区归属；见 第 18 步 (成员与重平衡) 。
heartbeat (心跳)	成员定期向 coordinator 报告仍存活请求；超时边界见 第 19 步 (心跳与过期) 。
generation (代次)	消费组成员集合变化时递增的版本；旧版本请求会被拒绝，见 第 20 步 (消费组围栏) 。
fencing (围栏)	用 epoch / generation 拒绝已过期角色或成员请求；消费组示例见 第 20 步 (消费组围栏) 。
leader / follower (主 / 从副本)	对同一分区负责接收写入的副本，以及向它拉取记录的副本；见 第 21 步 (拉取复制) 。
replication (复制)	把 leader 的记录拉取并追加到其他副本日志的过程；见 第 21 步 (拉取复制) 。
ISR (同步副本集合)	满足课程同步条件、可参与提交边界判断的副本集合；见 第 22 步 (ISR 与高水位) 。
HW (高水位)	普通消费者可见前缀的右边界；仅 offset 小于 HW 的记录可见，见 第 22 步 (ISR 与高水位) 。
ACK (写入确认)	producer 选择 broker 何时回复写入成功；确认策略见 第 23 步 (生产确认) 。
minISR (最小同步副本数)	某些写入允许成功所需的 ISR 最小成员数；见 第 23 步 (生产确认) 。
deadline (截止时刻)	一次等待允许达到的绝对时刻；超时不保证此前追加已回滚，见 第 23 步 (生产确认) 。
leader epoch (leader 代次)	分区写入角色变化时递增、用于拒绝旧角色请求的版本；见 第 25 步 (干净选举与 epoch) 。

术语	本书中的含义与位置
authority (权威状态)	本课程同一 JVM 内可信、记录当前分区角色和复制进度的控制状态；见 第 24 步（并发副本 broker） 。
at-least-once (至少一次)	成功处理与进度提交之间失败时允许再次处理同一记录；见 第 16 步（至少一次处理） 。
结果未知 (outcome unknown)	请求可能已生效但回复未到达，调用者无法仅凭异常判定是否追加；见 第 29 步（三层一致性故障实验） 。
幂等 (idempotence)	相同 producer 身份、epoch 与序号的相同批次重试，不会重复追加；见 第 30 步（幂等生产者） 。
producer identity / epoch / sequence	区分写入者、其代次和批次顺序的持久身份字段；见 第 30 步（幂等生产者） 。
event ID (事件标识)	稳定标记一项业务事件，使不同 Kafka offset 的重放仍可识别为同一效果；见 第 31 步（消费副作用幂等） 。
side effect (副作用)	消费回调产生的外部或业务状态变化，例如账户记账；见 第 31 步（消费副作用幂等） 。
事务 (transaction)	把一组相关操作作为整体提交或失败的机制；课程没有跨 producer、broker 与外部业务状态的原子事务，见 第 29 步（三层一致性故障实验） 。
恰好一次 (exactly-once)	对同一输入只产生一次可观察业务结果的保证，不等同于 callback 只执行一次；本课程不提供端到端保证，见 第 29 步（三层一致性故障实验） 与 第 31 步（消费副作用幂等） 。